



**International
Standard**

ISO/IEC/IEEE 12207

**Systems and software
engineering — Software life cycle
processes**

*Ingénierie des systèmes et du logiciel — Processus du cycle de vie
du logiciel*

**Second edition
2026-04**



COPYRIGHT PROTECTED DOCUMENT

© ISO/IEC 2026

© IEEE 2026

All rights reserved. Unless otherwise specified, or required in the context of its implementation, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from either ISO or IEEE at the respective address below or ISO's member body in the country of the requester.

ISO copyright office
CP 401 • Ch. de Blandonnet 8
CH-1214 Vernier, Geneva
Phone: +41 22 749 01 11
Email: copyright@iso.org
Website: www.iso.org

Institute of Electrical and Electronics Engineers, Inc
3 Park Avenue, New York
NY 10016-5997, USA

Email: stds.ipr@ieee.org
Website: www.ieee.org

Published in Switzerland

Contents

Page

Foreword	v
Introduction	vii
1 Scope	1
2 Normative references	1
3 Terms, definitions and abbreviated terms	1
3.1 Terms and definitions	1
3.2 Abbreviated terms	13
4 Conformance	14
4.1 General	14
4.2 Full conformance	14
4.2.1 Full conformance to outcomes	14
4.2.2 Full conformance to tasks	15
4.3 Tailored conformance	15
5 Key concepts and application	15
5.1 General	15
5.2 Software system concepts	15
5.2.1 Software systems	15
5.2.2 Software system structure	16
5.2.3 Interfacing, enabling and interoperating systems	17
5.2.4 System of systems concepts	19
5.3 Organization and project concepts	20
5.3.1 Organizations	20
5.3.2 Organization and project-level adoption	21
5.3.3 Organization and collaborative activities	21
5.4 Life cycle concepts	22
5.4.1 Software life cycle stages	22
5.4.2 Life cycle models for the software system	22
5.5 Process concepts	24
5.5.1 Criteria for processes	24
5.5.2 Description of processes	24
5.5.3 General characteristics of processes	25
5.5.4 Process views	25
5.5.5 Tailoring	25
5.6 Process groups	25
5.6.1 Introduction	25
5.6.2 Agreement processes	27
5.6.3 Organizational project-enabling processes	27
5.6.4 Technical management processes	28
5.6.5 Technical processes	28
5.7 Process application	29
5.7.1 Selection and timing of process use	29
5.7.2 Process iteration, recursion, and concurrency	30
5.8 Process reference model	30
6 Software life cycle processes	33
6.1 Agreement processes	33
6.1.1 Acquisition process	33
6.1.2 Supply process	36
6.2 Organizational project-enabling processes	38
6.2.1 Life cycle model management process	38
6.2.2 Infrastructure management process	40
6.2.3 Portfolio management process	41
6.2.4 Human resource management process	43
6.2.5 Quality management process	44

ISO/IEC/IEEE 12207:2026(en)

6.2.6	Knowledge management process.....	46
6.3	Technical management processes.....	47
6.3.1	Project planning process.....	48
6.3.2	Project assessment and control process.....	50
6.3.3	Decision management process.....	53
6.3.4	Risk management process.....	55
6.3.5	Configuration management process.....	57
6.3.6	Information management process.....	61
6.3.7	Measurement process.....	63
6.3.8	Quality assurance process.....	65
6.4	Technical processes.....	67
6.4.1	Business or mission analysis process.....	68
6.4.2	Stakeholder needs and requirements definition process.....	71
6.4.3	System requirements definition process.....	76
6.4.4	System architecture definition process.....	80
6.4.5	Design definition process.....	84
6.4.6	System analysis process.....	88
6.4.7	Implementation process.....	90
6.4.8	Integration process.....	94
6.4.9	Verification process.....	97
6.4.10	Transition process.....	101
6.4.11	Validation process.....	105
6.4.12	Operation process.....	108
6.4.13	Maintenance process.....	112
6.4.14	Disposal process.....	116
Annex A (normative) Tailoring process.....		119
Annex B (informative) Examples of process information items.....		121
Annex C (informative) Process reference model for assessment purposes.....		127
Annex D (informative) Model-based systems and software engineering (MBSSE).....		129
Annex E (informative) Assurance and assurance cases.....		135
Bibliography.....		136
IEEE notices and abstract.....		141

Foreword

ISO (the International Organization for Standardization) and IEC (the International Electrotechnical Commission) form the specialized system for worldwide standardization. National bodies that are members of ISO or IEC participate in the development of International Standards through technical committees established by the respective organization to deal with particular fields of technical activity. ISO and IEC technical committees collaborate in fields of mutual interest. Other international organizations, governmental and non-governmental, in liaison with ISO and IEC, also take part in the work.

The procedures used to develop this document and those intended for its further maintenance are described in the ISO/IEC Directives, Part 1. In particular, the different approval criteria needed for the different types of documents should be noted. This document was drafted in accordance with the editorial rules of the ISO/IEC Directives, Part 2 (see www.iso.org/directives or www.iec.ch/members_experts/refdocs).

IEEE Standards documents are developed within IEEE Societies and subcommittees of IEEE Standards Association (IEEE SA) Board of Governors. IEEE develops its standards through an accredited consensus development process, which brings together volunteers representing varied viewpoints and interests to achieve the final product. IEEE standards are documents developed by volunteers with scientific, academic, and industry-based expertise in technical working groups. Volunteers are not necessarily members of IEEE or IEEE SA and participate without compensation from IEEE. While IEEE administers the process and establishes rules to promote fairness in the consensus development process, IEEE does not independently evaluate, test, or verify the accuracy of any of the information or the soundness of any judgments contained in its standards.

ISO and IEC draw attention to the possibility that the implementation of this document may involve the use of (a) patent(s). ISO and IEC take no position concerning the evidence, validity or applicability of any claimed patent rights in respect thereof. As of the date of publication of this document, ISO and IEC had not received notice of (a) patent(s) which may be required to implement this document. However, implementers are cautioned that this may not represent the latest information, which may be obtained from the patent database available at www.iso.org/patents and <https://patents.iec.ch>. ISO and IEC shall not be held responsible for identifying any or all such patent rights.

Any trade name used in this document is information given for the convenience of users and does not constitute an endorsement.

For an explanation of the voluntary nature of standards, the meaning of ISO specific terms and expressions related to conformity assessment, as well as information about ISO's adherence to the World Trade Organization (WTO) principles in the Technical Barriers to Trade (TBT), see www.iso.org/iso/foreword.html. In the IEC, see www.iec.ch/understanding-standards.

This document was prepared by Joint Technical Committee ISO/JTC 1, *Information technology*, Subcommittee SC 7, *Software and systems engineering*, in cooperation with the Systems and Software Engineering Standards Committee of the IEEE Computer Society, under the Partner Standards Development Organization cooperation agreement between ISO and IEEE.

This second edition cancels and replaces the first edition (ISO/IEC/IEEE 12207:2017), which has been technically revised.

The main changes are as follows:

- clarifications and updates to reflect current practices in selected technical processes, including business or mission analysis, system architecture definition, implementation, integration, operations, and maintenance;
- improvements to selected technical management processes, including risk management and configuration management;
- updates to [Clause 5](#), key concepts, including a more precise description of iteration, recursion, system-of-systems, and quality characteristics;

ISO/IEC/IEEE 12207:2026(en)

- new content in [Clause 5](#) on concept and system definition, and expanded content on agile methods, process application, and system concepts.
- revised [Annex D](#) on model-based systems and software engineering (MBSSE)

Any feedback or questions on this document should be directed to the user's national standards body. A complete listing of these bodies can be found at www.iso.org/members.html and www.iec.ch/national-committees.

Introduction

The complexity of software systems continues to increase to unprecedented levels. Technology change has led to new opportunities, but also to increased challenges for the organizations that create and utilise systems. There is a continuum of human-made systems from those that use little or no software to those in which software is the primary interest. It is rare to encounter a complex system without software, and all software systems require physical system elements (hardware) to operate, either as part of the software system-of-interest (SoI) or as an enabling system or infrastructure. These challenges exist throughout the life cycle of a system and at all levels of architectural detail.

The purpose of this document is to provide a defined set of software life cycle processes. This document provides a common process framework for engineering the life cycle of systems created by humans, adopting a software engineering approach. Software engineering is an interdisciplinary approach and means to enable software. This document concerns software systems that are configured with software elements and with one or more of the following: hardware elements, data, humans, processes, services, procedures, and facilities.

This document provides processes for use by acquirers, suppliers, and other stakeholders in the life cycle of a software system, such as developers, integrators, operators, maintainers, managers, quality managers, and users of software services and products. It covers defining stakeholder needs, concerns, priorities, and constraints for the required functionality and non-functional characteristics early in the life cycle, establishing requirements, and concurrent design synthesis and system validation while considering the complete problem. It integrates disciplines and specialties into a team effort, forming a structured set of process that proceeds from concept through production operation and maintenance and sustainment to disposal. It considers both the business and the technical needs of stakeholders with the goal of providing a quality product that meets the needs of users and other applicable stakeholders. It provides the processes for acquiring and supplying systems. It helps to improve communication and cooperation among the parties that create, utilise, and manage modern software systems so they can work in an integrated, coherent fashion. Finally, this document provides the framework for assessment and improvement of the life cycle processes.

There is a wide variety of software systems in terms of their purpose, domain, complexity, size, novelty, adaptability, quantity, location, life span, and evolution. The processes in this document form a comprehensive set from which an organization can construct software life cycle models appropriate to its products and services. An organization can select and apply an appropriate subset of these processes to fulfil its specific objectives.

This document can be used in one or more of the following situations:

- By an organization — to help establish an environment of desired processes. These processes can be supported by an infrastructure of methods, procedures, techniques, tools, and trained personnel. The organization can then employ this environment to perform and manage its projects and progress systems through their life cycle stages. In this mode, this document can be used to assess conformance of a declared, established environment to its provisions. It can be used by a single organization in a self-imposed mode or in a multi-party situation. Parties can be from the same organization or from different organizations, and the situation can range from an informal agreement to a formal contract.
- By a project: to help select, structure, and employ the elements of an established environment to provide products and services. In this mode this document is used in the assessment of conformance of the project to the declared and established environment.
- By an acquirer and a supplier: to help develop an agreement concerning processes and activities. Via the agreement, the processes and activities in this document are selected, negotiated, agreed to, and performed. In this mode this document is used for guidance in developing the agreement.
- By process assessors: to serve as a process reference model for use in the performance of process assessments that can be used to support organizational process improvement.

This document is related to ISO/IEC/IEEE 15288, which covers system engineering processes. The choice of whether to apply this document for the software life cycle processes, or ISO/IEC/IEEE 15288:2023, depends

on the system-of-interest (SoI). Processes in both documents have the same process purpose and process outcomes, but differ in activities and tasks to perform software engineering or systems engineering, respectively.

The requirements in this document are intended to be compatible with the requirements of the quality management system provided by ISO 9001, the service management system provided by ISO/IEC 20000-1, the IT asset management system provided by ISO/IEC 19770-1, and the information security management system provided by ISO/IEC 27001.

Systems and software engineering — Software life cycle processes

1 Scope

This document establishes a common framework for software life cycle processes. Its terminology can be referenced and applied across the software industry. It contains processes, activities and tasks that can be applied during the acquisition of a software system, product, or service and during the supply, development, operation, maintenance, and disposal of software products and services. This is accomplished through the involvement of stakeholders, with the goal of achieving customer satisfaction. This document includes those aspects of system definition needed to provide the context for software systems and services. This document also provides processes that can be employed for defining, controlling, and improving software life cycle processes within an organization or a project.

This document is applicable to one-of-a-kind software systems, software systems for wide commercial or public distribution, and customised, adaptable software systems. Software includes the software portion of firmware. It applies to a complete stand-alone software system and to software systems that are embedded and integrated into larger more complex and complete systems of systems (SoS). The processes, activities, and tasks of this document can also be applied during the acquisition of a system that contains software.

This document applies to the full life cycle of software systems, products, and services, including conception, development, operations, support, and retirement, and to their acquisition and supply, whether performed internally or externally to an organization. The life cycle processes of this document can be applied concurrently, iteratively, and recursively to a software system and incrementally to its elements.

This document can be applied in organizations and software projects using a variety of formal engineering approaches. It is applicable for agile approaches and methods, which are most widely used for software development, sustainment, and maintenance, and which are believed to be more affordable and to deliver usable products more quickly.

This document does not identify or require any specific software life cycle model, development methodology, method, modelling approach, or techniques for selecting a life cycle model for the organization or project and mapping the processes, activities, and tasks in this document into that model. Using engineering judgment to help achieve the desired level of quality is also outside the scope of this document.

This document does not detail information items in terms of name, format, explicit content, and recording media. ISO/IEC/IEEE 15289 identifies the content for life cycle process information items (documentation).

2 Normative references

There are no normative references in this document.

3 Terms, definitions and abbreviated terms

3.1 Terms and definitions

For the purposes of this document, the following terms and definitions apply.

ISO, IEC, and IEEE maintain terminology databases for use in standardization at the following addresses:

- ISO Online browsing platform: available at <https://www.iso.org/obp>
- IEC Electropedia: available at <https://www.electropedia.org/>

— IEEE Standards Dictionary Online: available at: <https://dictionary.ieee.org/>

NOTE Definitions for other system and software engineering terms can be found in ISO/IEC/IEEE 24765, available at www.computer.org/sevocab.

3.1.1

acquirer

stakeholder (3.1.76) that acquires or procures a *system* (3.1.78), *product* (3.1.49), or *service* (3.1.65) from a *supplier* (3.1.77)

Note 1 to entry: Other terms commonly used for an acquirer are buyer, *customer* (3.1.19), owner, purchaser, or internal organizational sponsor.

3.1.2

acquisition

process (3.1.45) of obtaining a *system* (3.1.78), *product* (3.1.49) or *service* (3.1.65)

3.1.3

activity

set of cohesive *tasks* (3.1.83) of a *process* (3.1.45)

3.1.4

agile development

development approach based on iterative development, frequent inspection and adaptation, and incremental deliveries, in which *requirements* (3.1.56) and solutions evolve through collaboration in cross-functional teams and through continual *stakeholder* (3.1.76) feedback

3.1.5

agreement

mutual acknowledgement of terms and conditions under which a working relationship is conducted

EXAMPLE Contract, memorandum of agreement.

3.1.6

architecture

fundamental concepts or properties of an entity in its *environment* (3.1.25) and governing principles for the realization and evolution of this entity and its related *life cycle* (3.1.34) *processes* (3.1.45)

[SOURCE: ISO/IEC/IEEE 42010:2022, 3.2]

3.1.7

architecture description framework

conventions, principles and practices for the description of *architectures* (3.1.6) established within a specific domain of application or community of *stakeholders* (3.1.76)

EXAMPLE Generalized Enterprise Reference Architecture and Methodologies (GERAM) (ISO 15704:2019, Annex B), Reference Model of Open Distributed Processing (RM-ODP) (ISO/IEC 10746-1), Reference Model of Open Distributed Processing (RM-ODP), Unified Architecture Framework (UAF), and NATO Architecture Framework (NAF).

[SOURCE: ISO/IEC/IEEE 42010:2022, 3.5, modified — The abbreviated term and note 1 to entry have been removed.]

3.1.8

architecture view

information part comprising portion of an *architecture* (3.1.6) description

[SOURCE: ISO/IEC/IEEE 42010:2022, 3.7, modified — EXAMPLE has been removed.]

3.1.9

architecture viewpoint

set of conventions for the creation, interpretation and use of an *architecture view* (3.1.8) to frame one or more *concerns* (3.1.16)

[SOURCE: ISO/IEC/IEEE 42010:2022, 3.8, modified — Notes to entry have been removed.]

3.1.10

artefact

work *product* (3.1.49) that is produced and used during a *project* (3.1.50) to capture and convey information

EXAMPLE Models, *stakeholder* (3.1.76) *requirements* (3.1.56), *system* (3.1.78) *requirements*, *architecture* (3.1.6) *descriptions*, *design* (3.1.20) *descriptions*, source code, implemented *system elements* (3.1.79), verified or validated system.

3.1.11

audit

independent examination of a work *product* (3.1.49) or set of work products to assess compliance with contractual *agreements* (3.1.5) or conformance to specifications, standards, or other criteria

3.1.12

baseline

formally approved version of a *configuration item* (3.1.17), regardless of media, formally designated and fixed at a specific time during the configuration item's *life cycle* (3.1.34)

[SOURCE: IEEE Std 828-2012]

3.1.13

business process

partially ordered set of enterprise *activities* (3.1.3) that can be executed to achieve some desired end-result in pursuit of a given objective of an *organization* (3.1.41)

3.1.14

capability

ability to do something useful under a particular set of conditions

[SOURCE: ISO/IEC/IEEE 24641:2023, 3.1.3, modified — Note 1 to entry has been removed.]

3.1.15

concept of operations

verbal and graphic statement, in broad outline, of an *organization's* (3.1.41) assumptions or intent regarding an operation or series of operations of new, modified, or existing organizational *systems* (3.1.78)

Note 1 to entry: The concept of operations frequently is embodied in long-range strategic plans and annual operational plans. In the latter case, the concept of operations in the plan covers a series of connected operations to be carried out simultaneously or in succession to achieve an organizational performance objective. See also *operational concept* (3.1.39).

Note 2 to entry: The concept of operations provides the basis for bounding the operating space, system *capabilities* (3.1.14), *interfaces* (3.1.33) and operating *environment* (3.1.25).

[SOURCE: ANSI/AIAA G-043B-2018, 5.2, modified — The second definition has been used; the last two sentences of Note 1 to entry have been removed; Note 2 to entry has been added.]

3.1.16

concern

matter of interest or importance to a *stakeholder* (3.1.76)

Note 1 to entry: A concern pertains to any influence on a *system* (3.1.78) in its *environment* (3.1.25), including developmental, technological, business, operational, organizational, political, economic, legal, regulatory, ethical, ecological, and social influences.

[SOURCE: ISO/IEC/IEEE 42020:2019, 3.8, modified — EXAMPLE has been removed; Note 1 to entry has been added.]

3.1.17

configuration item

item or aggregation of *hardware* (3.1.29) or *software* (3.1.66) with associated information and data

Note 1 to entry: *Information items* (3.1.31) can be considered as part of the software and managed as part of the configuration item, or can be managed using the information management *process* (3.1.45).

3.1.18

constituent system

independent *system* (3.1.78) that forms part of a *system of systems (SoS)* (3.1.81)

Note 1 to entry: Constituent systems can be part of one or more SoS. Each constituent system is a useful system by itself, having its own development, *management* (3.1.38), utilization, goals, and *resources* (3.1.60), but interacts within the SoS to provide the unique *capability* (3.1.14) of the SoS.

[SOURCE: ISO/IEC/IEEE 21839:2019, 3.1.1]

3.1.19

customer

organization (3.1.41) or person that receives a *product* (3.1.49) or *service* (3.1.65)

EXAMPLE Consumer, client, *user* (3.1.88), *acquirer* (3.1.1), buyer or purchaser.

Note 1 to entry: A customer can be internal or external to the organization.

3.1.20

design

specification of *system elements* (3.1.79) and their relationships, that is sufficiently complete to support a compliant implementation of the *architecture* (3.1.6)

Note 1 to entry: Design provides the detailed implementation-level physical structure, behaviour, temporal relationships, and other attributes of system elements.

3.1.21

design characteristic

design (3.1.20) attribute or distinguishing feature that pertains to a measurable description of a *product* (3.1.49), or *service* (3.1.65)

3.1.22

digital engineering

software engineering (3.1.69) discipline that uses the underlying data, modelling, simulation, and analytical functions for physical or logical *systems* (3.1.78) conceptualization, development, test and evaluation, and integrated control of operational performance

3.1.23

enabling system

system (3.1.78) that supports a *system-of-interest* (3.1.80) during its *life cycle* (3.1.36) *stages* (3.1.75) but does not necessarily contribute directly to its function during operation

EXAMPLE A configuration management system used to control *software elements* (3.1.68) during *software* (3.1.66) development.

Note 1 to entry: Each enabling system has a life cycle of its own.

3.1.24

engineering

application of a systematic, disciplined, quantifiable approach to structures, machines, *products* (3.1.49), systems, or *processes* (3.1.45)

3.1.25

environment

context determining the setting and circumstances of all influences upon a *system* ([3.1.78](#))

3.1.26

facility

physical means or equipment for facilitating the performance of an action, e.g. buildings, instruments, tools

3.1.27

firmware

ordered set of instructions and associated data stored in a way that is functionally independent of main storage, usually in read-only memory

3.1.28

governance

practice of establishing and enforcing strategic goals and objectives, organizational policies and performance parameters

3.1.29

hardware

physical equipment used to *process* ([3.1.45](#)), store, or transmit computer programs or data

[SOURCE: ISO/IEC 19770-1:2017, 3.21]

3.1.30

incident

anomalous or unexpected event or set of events at any time during the *life cycle* ([3.1.36](#)) of a *project* ([3.1.50](#)), *product* ([3.1.49](#)), *service* ([3.1.65](#)), or *system* ([3.1.78](#))

Note 1 to entry: An incident is elevated and treated as a *problem* ([3.1.44](#)) when the cause of the incident is analysed and corrected to prevent recurrence, or to avoid or minimise loss of life, or damage to property or natural *resources* ([3.1.60](#)). Some incidents do not involve further analysis, as they result from known errors where workarounds or solutions are already known.

3.1.31

information item

information product

separately identifiable body of information that is produced, stored, and delivered for human use

Note 1 to entry: A document produced to meet information *requirements* ([3.1.59](#)) can be an information item, part of an information item, or a combination of several information items.

Note 2 to entry: An information item can be produced in several versions during a *project* ([3.1.50](#)) or *system* ([3.1.78](#)) *life cycle* ([3.1.36](#)).

[SOURCE: ISO/IEC/IEEE 15289:2019, 3.1.12, modified — The preferred term "information product" has been changed to an admitted term.]

3.1.32

infrastructure

hardware ([3.1.29](#)) and *software* ([3.1.66](#)) *environment* ([3.1.25](#)) to support *system* ([3.1.78](#)) and software *design* ([3.1.20](#)), development, operation and modification

3.1.33

interface

point at which two or more logical or physical *system elements* ([3.1.79](#)) meet to act on or communicate with each other

3.1.34

interoperating system

system ([3.1.78](#)) that exchanges information with the *system-of-interest* ([3.1.80](#)) and uses the information that has been exchanged

3.1.35 iteration

repeating the application of the same *process* (3.1.45) or set of processes at the same level of abstraction on the same *system* (3.1.78) or *system element* (3.1.79)

3.1.36 life cycle

evolution of a *system* (3.1.78), *product* (3.1.49), *service* (3.1.65), *project* (3.1.50) or other human-made entity from conception through *retirement* (3.1.61)

3.1.37 life cycle model

framework of *processes* (3.1.45) and *activities* (3.1.3) concerned with the *life cycle* (3.1.36) which can be organized into *stages* (3.1.75), acting as a common reference for communication and understanding

3.1.38 management

oversight and direction of *activities* (3.1.3) and controls to achieve the strategic objectives set by the *organization's* (3.1.41) governing body

3.1.39 operational concept

verbal and graphic statement of an *organization's* (3.1.41) assumptions or intent in regard to an operation or series of operations of a specific *system* (3.1.78) or a related set of specific new, existing or modified systems

Note 1 to entry: The operational concept is designed to give an overall picture of the operations using one or more specific systems, or set of related systems, in the organization's operational *environment* (3.1.25) from the *users'* (3.1.88) and *operators'* (3.1.40) perspectives.

Note 2 to entry: The operational concept is about systems, while a *concept of operations* (3.1.15) typically refers to organizations.

[SOURCE: ANSI/AIAA G-043B-2018, 5.2, modified — The third definition has been used; the first sentence in Note 1 to entry has been removed; Note 2 to entry has been added.]

3.1.40 operator

individual or *organization* (3.1.41) that performs the operations of a *system* (3.1.78)

Note 1 to entry: The role of operator and the role of *user* (3.1.88) can be vested, simultaneously or sequentially, in the same individual or organization.

Note 2 to entry: An individual operator combined with knowledge, skills and procedures can be considered as an element of the system.

Note 3 to entry: An operator may perform operations on a system that is operated, or of a system that is operated, depending on whether operating instructions are placed within the system boundary.

3.1.41 organization

person or group of people that has its own functions with responsibilities, authorities, and relationships to achieve its objectives

EXAMPLE Company, corporation, firm, enterprise, manufacturer, institution, charity, sole trader, association, or parts or combination thereof.

Note 1 to entry: An identified part of an organization (even as small as a single individual) or an identified group of organizations can be regarded as an organization if it has responsibilities, authorities and relationships. A body of persons organized for some specific purpose, such as a club, union, corporation or society, is an organization.

3.1.42

party

organization ([3.1.41](#)) entering into an *agreement* ([3.1.5](#))

Note 1 to entry: The agreeing parties are called the *acquirer* ([3.1.1](#)) and the *supplier* ([3.1.77](#)).

3.1.43

portfolio

collection of *projects* ([3.1.50](#)) or *systems* ([3.1.78](#)) that address the strategic or business objectives of the *organization* ([3.1.41](#))

3.1.44

problem

difficulty, uncertainty, or otherwise realised and undesirable condition or situation that is investigated and can receive corrective action

3.1.45

process

set of interrelated or interacting *activities* ([3.1.3](#)) that transform inputs into outputs

3.1.46

process outcome

observable result of the successful achievement of the *process purpose* ([3.1.47](#))

3.1.47

process purpose

high level objective of performing the *process* ([3.1.45](#)) and the likely outcomes of effective implementation of the process

Note 1 to entry: The purpose of implementing the process is to provide benefits to the *stakeholders* ([3.1.76](#))

3.1.48

process reference model

PRM

model comprising definitions of *processes* ([3.1.45](#)) in a domain of application described in terms of *process purpose* ([3.1.47](#)) and outcomes, together with an *architecture* ([3.1.6](#)) describing the relationships between the processes

[SOURCE: ISO/IEC 33001:2015, 3.3.16, modified — The abbreviated term "PRM" has been added.]

3.1.49

product

result of a *process* ([3.1.45](#))

Note 1 to entry: ISO/IEC/IEEE 15288 defines a product as "output of an *organization* ([3.1.41](#)) that can be produced without any transaction taking place between the organization and the *customer* ([3.1.19](#))".

Note 2 to entry: There are four generic product categories: *hardware* ([3.1.29](#)), e.g. engine mechanical part; *software* ([3.1.66](#)), e.g. computer program procedures, and possibly associated documentation and data; *services* ([3.1.65](#)), e.g. transport; and processed materials, e.g. lubricant. Hardware and processed materials are generally tangible products, while software or services are generally intangible.

3.1.50

project

endeavour with defined start and finish criteria undertaken to create a *product* ([3.1.49](#)) or *service* ([3.1.65](#)) in accordance with specified *resources* ([3.1.60](#)) and *requirements* ([3.1.59](#))

Note 1 to entry: A project is sometimes viewed as a unique *process* ([3.1.45](#)) comprising coordinated and controlled *technical management* ([3.1.85](#)) and *technical activities* ([3.1.3](#)).

Note 2 to entry: Continuous development approaches such as agile and DevOps can create products and services without the defined start and finish criteria of projects.

3.1.51

qualification

process ([3.1.45](#)) of determining whether an entity is capable of fulfilling specified *requirements* ([3.1.59](#))

3.1.52

quality assurance

QA

process ([3.1.45](#)) that is focused on providing confidence that *quality requirements* ([3.1.56](#)) are fulfilled

3.1.53

quality characteristic

inherent characteristic of a *product* ([3.1.46](#)), *service* ([3.1.65](#)), *process* ([3.1.45](#)) or *system* ([3.1.78](#)) related to a *requirement* ([3.1.59](#))

Note 1 to entry: Critical quality characteristics commonly include those related to health, *safety* ([3.1.63](#)), *security* ([3.1.64](#)) assurance, reliability, availability, and supportability.

[SOURCE: ISO 9000:—, 3.11.2, modified — ‘object’ has been replaced with ‘product, service, process or system’; notes to entry have been replaced by a new one.]

3.1.54

quality control

QC

tasks ([3.1.83](#)) to evaluate the quality of *services* ([3.1.65](#)) as delivered or the quality of developed or manufactured *products* ([3.1.46](#))

3.1.55

quality management

QM

coordinated *activities* ([3.1.3](#)) to direct and control an *organization* ([3.1.41](#)) with regard to quality

3.1.56

quality requirement

requirement ([3.1.59](#)) for quality properties or attributes of a *product* ([3.1.46](#)), data, or *service* ([3.1.65](#)) that satisfy needs associated with its the purpose

3.1.57

recursion

repeating the application of the same *process* ([3.1.45](#)) or set of processes to progressively more detailed *system elements* ([3.1.79](#))

3.1.58

release

version of a *configuration item* ([3.1.17](#)) that is made available for a specific purpose

EXAMPLE Test release.

3.1.59

requirement

statement which translates or expresses a need and its associated constraints and conditions

[SOURCE: ISO/IEC/IEEE 29148:2018, 3.1.19, modified — Notes to entry have been removed.]

3.1.60

resource

asset that is utilised or consumed during the execution of a *process* ([3.1.45](#))

Note 1 to entry: Resources include those that are reusable, renewable, or consumable.

Note 2 to entry: Resource includes diverse entities such as funding, personnel, *facilities* ([3.1.26](#)), capital equipment, tools and utilities such as power, water, fuel and communication *infrastructures* ([3.1.32](#)).

3.1.61

retirement

withdrawal of active support by the operation and maintenance *organization* (3.1.41), partial or total replacement by a new *system* (3.1.78), installation of an upgraded system, or final decommissioning and disposal

3.1.62

risk

effect of uncertainty on objectives

Note 1 to entry: An effect is a deviation from the expected — positive or negative. A positive effect is also known as an opportunity.

Note 2 to entry: Objectives can have different aspects [such as financial, health and *safety* (3.1.63), and environmental goals] and can apply at different levels, such as strategic, organization-wide, *project* (3.1.50), *product* (3.1.46) and *process* (3.1.45).

Note 3 to entry: Risk is often characterized by reference to potential events and consequences, or a combination of these.

Note 4 to entry: Risk is often expressed in terms of a combination of the consequences of an event (including changes in circumstances) and the associated likelihood of occurrence.

Note 5 to entry: Uncertainty is the state, even partial, of deficiency of information related to understanding or knowledge of an event, its consequence, or likelihood.

3.1.63

safety

expectation that a *system* (3.1.78) does not, under defined conditions, lead to a state in which human life, health, property, or the *environment* (3.1.25) is endangered

3.1.64

security

protection against intentional subversion or forced failure

Note 1 to entry: Security includes authenticity, accountability, confidentiality, integrity, availability, non- repudiation, and reliability, all of which have the related issue of their assurance.

[SOURCE: NATO AEP-67, modified — Note 1 to entry has been updated.]

3.1.65

service

performance of *activities* (3.1.3), work, or duties

Note 1 to entry: The dominant elements of a service are generally intangible.

Note 2 to entry: A service is coherent, discrete, and can be composed of other services.

Note 3 to entry: In ISO/IEC/IEEE 15288, service is defined as "output of an *organization* (3.1.41) with at least one *activity* (3.1.3) necessarily performed between the organization and the *customer* (3.1.19).

3.1.66

software

computer programs, procedures and possibly associated documentation and data pertaining to the operation of a computer *system* (3.1.78)

3.1.67

software bill of materials

SBOM

list of uniquely identified *software* (3.1.66) items, including their sources in the supply chain, version, license and copyright data, and possibly *security* (3.1.64)-related data

Note 1 to entry: Each SBOM item can have its own SBOM.

3.1.68

software element

system element (3.1.79) that is *software* (3.1.66)

Note 1 to entry: A software element can be part of a *software system* (3.1.72) or a *system* (3.1.78) that is not predominantly software.

3.1.69

software engineering

application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of *software* (3.1.66)

Note 1 to entry: Software engineering is the application of *engineering* (3.1.24) to software.

3.1.70

software item

source code, object code, control code, control data or collection of these items

Note 1 to entry: A *software* (3.1.66) item can be viewed as a *system element* (3.1.79). Software items are typically *configuration items* (3.1.17).

3.1.71

software product

set of computer programs, procedures, and possibly associated documentation and data

Note 1 to entry: A *software* (3.1.66) *product* (3.1.46) is a *software system* (3.1.72) viewed as the output product resulting from a *process* (3.1.45).

3.1.72

software system

system (3.1.78) for which *software* (3.1.66) is of primary importance to the *stakeholders* (3.1.76)

Note 1 to entry: In the most general case, a software system is comprised of *hardware* (3.1.29), software, people and manual procedures.

Note 2 to entry: In a software system, software is the leading driver in meeting *system requirements* (3.1.59).

3.1.73

software system element

member of a set of elements that constitute a *software system* (3.1.72)

Note 1 to entry: A software system element can include one or more *software units* (3.1.74), *software elements* (3.1.68), *hardware* (3.1.29) units, hardware elements, *services* (3.1.65), and other *system elements* (3.1.79) and *systems* (3.1.78).

Note 2 to entry: A software system element can be viewed as a system element.

3.1.74

software unit

atomic-level *software* (3.1.66) component of the *software architecture* (3.1.6) that can be subjected to standalone testing

Note 1 to entry: Some software units are separately compilable pieces of code.

[SOURCE: ISO 26262-1:2018, 3.159, modified —Note 1 to entry has been added.]

3.1.75

stage

period within the *life cycle* (3.1.36) of an entity that relates to the state of its description or realization

Note 1 to entry: Stages relate to major progress and achievement milestones of the entity through its life cycle.

Note 2 to entry: Stages often overlap.

3.1.76

stakeholder

individual or *organization* (3.1.41) having a right, share, claim or interest in a *system* (3.1.78) or in its possession of characteristics that meet their needs and expectations

EXAMPLE *Users* (3.1.88), end user organizations, supporters, developers, *customers* (3.1.19), producers, trainers, maintainers, disposers, *acquirers* (3.1.1), *suppliers* (3.1.77), regulatory bodies, and people influenced positively or negatively by a system.

Note 1 to entry: Some stakeholders can have interests that oppose each other or oppose the system.

Note 2 to entry: Stakeholders include those who are affected or impacted by a system.

3.1.77

supplier

organization (3.1.41) or an individual that enters into an *agreement* (3.1.5) with the *acquirer* (3.1.1) for the supply of a *product* (3.1.46) or *service* (3.1.65)

Note 1 to entry: Other terms commonly used for supplier are contractor, producer, seller or vendor.

Note 2 to entry: The acquirer and the supplier sometimes are part of the same organization.

3.1.78

system

combination of interacting elements organized to achieve one or more stated purposes

Note 1 to entry: A system is sometimes considered as a *product* (3.1.46) or as the *services* (3.1.65) it provides.

Note 2 to entry: In practice, the interpretation of its meaning is frequently clarified by an associative noun, e.g. database management system. Alternatively, the word "system" is substituted simply by a context-dependent synonym e.g. database, though this potentially obscures a system principles perspective.

Note 3 to entry: A system can include the associated equipment, *facilities* (3.1.26), material, *software* (3.1.63), *firmware* (3.1.27), technical information, services and personnel required for operations and support to the degree necessary for use in its intended *environment* (3.1.25).

Note 4 to entry: In ISO/IEC/IEEE 15288, system is defined as "arrangement of parts or elements that together exhibit a stated behaviour or meaning that the individual constituents do not".

3.1.79

system element

discrete part of a *system* (3.1.75) that can be implemented to fulfil specified *requirements* (3.1.59)

EXAMPLE *hardware* (3.1.29), software, data, humans, *processes* (3.1.45), e.g. processes for providing *service* (3.1.65) to *users* (3.1.88), procedures e.g. *operator* (3.1.40) instructions, *facilities* (3.1.26), materials, and naturally occurring entities or any combination.

Note 1 to entry: A system element that is *software* (3.1.66) is a *software element* (3.1.68).

3.1.80

system-of-interest

SoI

system (3.1.75) whose *life cycle* (3.1.36) is under consideration

3.1.81

system of systems

SoS

set of *systems* (3.1.75) or *system elements* (3.1.79) that interact to provide a unique *capability* (3.1.14) that none of the *constituent systems* (3.1.18) can accomplish on its own

Note 1 to entry: Each constituent system is a useful system by itself, having its own *management* (3.1.38), goals, and *resources* (3.1.60), but coordinates within the SoS to provide the unique capability of the SoS.

[SOURCE: ISO/IEC/IEEE 21839:2019, 3.1.4, modified — The original note 1 to entry has been replaced by a new one.]

3.1.82

systems engineering

transdisciplinary and integrative approach to enable the successful realization, use and *retirement* (3.1.61) of engineered *systems* (3.1.75) using systems principles and concepts and scientific, technological and *management* (3.1.38) methods

3.1.83

task

required, recommended or permissible action, intended to contribute to the achievement of one or more outcomes of a *process* (3.1.45)

3.1.84

technical debt

deferred cost of work not identified or not done at an earlier point in the *product* (3.1.46) *life cycle* (3.1.36)

3.1.85

technical management

application of technical and administrative *resources* (3.1.60) to plan, organize and control *engineering* (3.1.24) functions

3.1.86

traceability

association established between two or more logical entities, especially entities having a predecessor-successor or primary-secondary relationship to one another, such as *requirements* (3.1.56), *system elements* (3.1.79), *verifications* (3.1.90) or *tasks* (3.1.83)

EXAMPLE *Software* (3.1.66) features and test cases are typically traceable to software requirements.

3.1.87

trade-off

decision-making actions that select from various *requirements* (3.1.56) and alternative solutions on the basis of net benefit to the *stakeholders* (3.1.76)

3.1.88

user

individual or group that interacts with a *system* (3.1.75) or benefits from a system during its utilization

Note 1 to entry: The role of user and the role of *operator* (3.1.40) are sometimes vested, simultaneously or sequentially, in the same individual or *organization* (3.1.41).

3.1.89

validation

confirmation, through the provision of objective evidence, that the *requirements* (3.1.56) for a specific intended use or application have been fulfilled

Note 1 to entry: In a *life cycle* (3.1.36) context, validation involves the set of *activities* (3.1.3) for gaining confidence that a *system* (3.1.75) can accomplish its intended use, goals, and objectives in an *environment* (3.1.25) like the operational environment. The right system was built.

[SOURCE: ISO 9000:—, 3.12.13, modified — Notes 1 to 3 to entry have been removed; a new note 1 to entry has been added.]

3.1.90

verification

confirmation, through the provision of objective evidence, that specified *requirements* (3.1.56) have been fulfilled

Note 1 to entry: Verification is a set of *activities* (3.1.3) that compares a *system* (3.1.75) or *system element* (3.1.79) against the required characteristics. Verification includes specified requirements, *design* (3.1.20) descriptions, and the system itself. The system was built right.

[SOURCE: ISO 9000:—, 3.12.11, modified — Notes 1 to 3 to entry have been removed; a new note 1 to entry has been added.]

3.1.91

view

representation of a *system* (3.1.75) from the perspective of a related set of *concerns* (3.1.16)

Note 1 to entry: A view can be an operational, functional or architectural representation of a system.

[SOURCE: ISO/IEC/IEEE 24774:2021, 3.21, modified — ‘whole’ has been removed from the definition; the original note 1 to entry has been replaced by a new one.]

3.1.92

viewpoint

specification of the conventions for constructing and using a *view* (3.1.91)

[SOURCE: ISO/IEC/IEEE 24774:2021, 3.22, modified — Notes 1 to 3 to entry have been removed.]

3.2 Abbreviated terms

API	application program interface
CM	configuration management
COTS	commercial-off-the-shelf
FCA	functional configuration audit
GUI	graphical user interface
MBSSE	model-based systems and software engineering
NDI	non-developmental items
OTS	off the shelf
PCA	physical configuration audit
PMP	project management plan
SCM	software configuration management
SDP	software development plan
SEMP	systems engineering management plan
WBS	work breakdown structure

4 Conformance

4.1 General

[Clause 6](#) contains requirements for software engineering processes. Projects or organizations can use selections from the processes provided by this document. Therefore, implementation of this document typically involves selecting and declaring a set of processes suitable to the organization or project. There are two ways that an implementation can be claimed to conform to the provisions of this document: full conformance ([4.2](#)) and tailored conformance ([4.3](#)).

There are two criteria for claiming full conformance. Achieving either criterion suffices for conformance, although the chosen criterion (or criteria) shall be stated in the claim. Claiming “full conformance to tasks” asserts that all the required activities and tasks of the declared set of processes are achieved. Alternatively, claiming “full conformance to outcomes” asserts that all the required outcomes of the declared set of processes are achieved. Full conformance to outcomes permits greater freedom in the implementation of conforming processes and can be useful for implementing processes to be used in the context of an innovative life cycle model.

NOTE ISO/IEC/IEEE 24774:2021, Clause 7 describes how to state claims of conformance to process outcomes or to tasks.

Options for conformance are provided for needed flexibility in the application of this document. Each process has a set of objectives (phrased as “outcomes”) and a set of activities and tasks that represent one way to achieve the objectives.

Users who implement the activities and tasks of the declared set of processes can assert full conformance to tasks of the selected processes. Some users, however, can have innovative process variants that achieve the objectives (i.e. the outcomes) of the declared set of processes without implementing all the activities and tasks. These users can assert full conformance to the outcomes of the declared set of processes. The two criteria, conformance to task and conformance to outcome, are necessarily not equivalent since specific performance of activities and tasks can require, in some cases, a higher level of capability than just the achievement of outcomes.

When this document is used to help develop an agreement between an acquirer and a supplier, clauses of this document can be selected for incorporation in the agreement with or without modification. In this case, it is more appropriate for the acquirer and supplier to claim compliance with the agreement than conformance with this document.

An organization (e.g. national, industrial association, company) imposing this document as a condition of trade can specify and make public the minimum set of required processes, outcomes, activities, and tasks, which constitute suppliers' compliance with the conditions of trade.

4.2 Full conformance

4.2.1 Full conformance to outcomes

A claim of full conformance declares the set of processes for which conformance is claimed. Full conformance to outcomes is achieved by demonstrating that all the outcomes of the declared set of processes have been achieved. In this situation, the provisions for activities and tasks of the declared set of processes are guidance rather than requirements, regardless of the verb form that is used in the provision.

One intended use of this document is to facilitate process assessment and improvement. For this reason, each process in this document is written in the form of purpose and outcomes compatible with the provisions of ISO/IEC/IEEE 24774. Users intending process assessment and improvement may use the process outcomes in this document as the process reference model required by ISO/IEC 33002, as detailed in [Annex C](#).

4.2.2 Full conformance to tasks

A claim of full conformance declares the set of processes for which conformance is claimed. Full conformance to tasks is achieved by demonstrating that all the requirements of the activities and tasks of the declared set of processes have been achieved. In this situation, the provisions for the outcomes of the declared set of processes are guidance rather than requirements, regardless of the verb form that is used in the provision.

NOTE A claim of full conformance to tasks can be appropriate in contractual situations where an acquirer or a regulator requires detailed understanding of the suppliers' processes.

4.3 Tailored conformance

When this document is used as a basis for establishing a set of processes that do not qualify for full conformance, the processes in [Clause 6](#) shall be selected or modified in accordance with the tailoring process prescribed in [Annex A](#). The tailored set of processes, for which tailored conformance is claimed, is declared. Tailored conformance is achieved by demonstrating that the outcomes, activities, and tasks, as tailored, have been achieved.

Tailoring can diminish the perceived value of a claim of conformance to this document because it is difficult for other organizations to understand the extent to which tailoring can have deleted desirable provisions.

An organization asserting a claim of conformance to this document can find it advantageous to claim full conformance to a smaller list of processes rather than tailored conformance to a larger list of processes.

An organization may choose to claim full conformance to a selected set of processes as well as tailored conformance to other processes.

5 Key concepts and application

5.1 General

This clause highlights and explains essential concepts on which this document is based.

NOTE ISO/IEC/IEEE 24748-1, ISO/IEC/IEEE 24748-2, ISO/IEC/IEEE 24748-3, and ISO/IEC/IEEE 24748-4 provide additional detail regarding systems concepts, including processes, life cycle models and stages. The life cycle concepts and models described in this clause can be applied not only to software systems but also to other systems as described in ISO/IEC/IEEE 15288.

5.2 Software system concepts

5.2.1 Software systems

The software systems considered in this document are human-created and utilized to provide products or services in defined environments for the benefit of users and other stakeholders. These software systems can include the following system elements: hardware, software, data, humans, processes (e.g. processes for providing service to users), procedures (e.g. operator instructions), facilities, services, materials and naturally occurring entities. As viewed by the user, they are thought of as products or services.

This document applies to systems for which software is of primary or significant importance to the stakeholders. It is based upon the general principles of systems engineering and software engineering. A fundamental premise of this document is that software always exists in the context of a system. Since software does not operate without hardware, the processor upon which the software is executed can be considered as part of the system. Alternatively, hardware or services hosting the software system and handling communications with other systems can also be viewed as the platform, infrastructure systems or external systems in the operating environment. In this case, the software is the system-of-interest (SoI).

The perception and definition of a particular software system, its architecture, and its elements depend on a stakeholder's interests and responsibilities. One stakeholder's SoI can be viewed as a system element in

another stakeholder's SoI. Furthermore, an SoI can be viewed as being part of the environment for another stakeholder's SoI.

The following are key points regarding the characteristics of SoI:

- a) Defined boundaries encapsulate meaningful needs and practical solutions.
- b) There is a logical, functional, or other relationship between system elements.
- c) An entity at any level in the SoI can be viewed as a system.
- d) A system comprises an integrated, defined set of subordinate system elements.
- e) Humans can be viewed as both users of a system and as system elements (i.e. operators) within a system.
- f) A system can be viewed in isolation as an entity, i.e. a product, or as means of producing products, or as a collection of functions capable of interacting with its surrounding environment, i.e. a set of services.

Whatever the boundaries chosen to define the system, the concepts in this document are generic and permit a practitioner to correlate or adapt individual instances of life cycles to its system principles.

5.2.2 Software system structure

The life cycle processes in this document are described in relation to a software system. A software system is composed of a set of interacting system elements, including software elements. Each element of the software system fulfils one or more stated purposes, as expressed in its respective specified requirements. The software elements of the system are of primary importance to the stakeholders. Each element of the software system can be implemented within the system boundary ([Figure 1](#)).

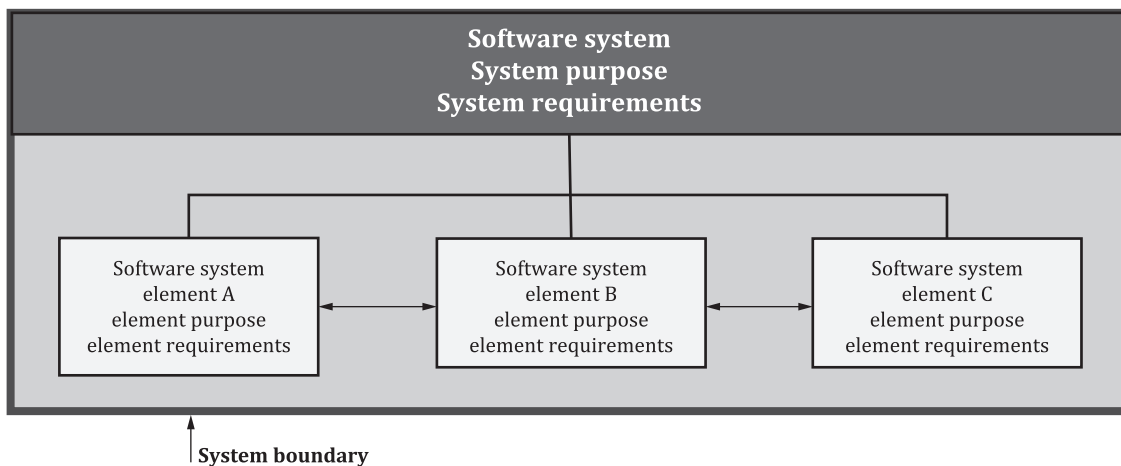


Figure 1 — Software system and software system element relationship

The relationship between the software system and its complete set of system elements can typically be represented showing relationships between the elements – often depicted as a hierarchy for the simplest of systems-of-interest. Decomposition is one approach to some software activities. Other approaches include the object-oriented approach, where the system elements are laid out in a flat (non-hierarchical) description such as in a network diagram. For more complex software systems-of-interest, a prospective system element may need to be considered as a system (that in turn is comprised of system elements) before a complete set of system elements can be defined with confidence. The appropriate system life cycle processes are applied recursively to an SoI to resolve its structure to the point where understandable and manageable software system elements can be implemented: created, modelled, adapted, acquired, or reused ([Figure 2](#)).

NOTE 1 Responsibility for the implementation of any system element can be delegated to another party through an agreement.

NOTE 2 [Annex D](#) contains more information on modelling software systems and describes model-based systems and software engineering (MBSSE) and the relationships of software engineering, systems engineering, and digital engineering.

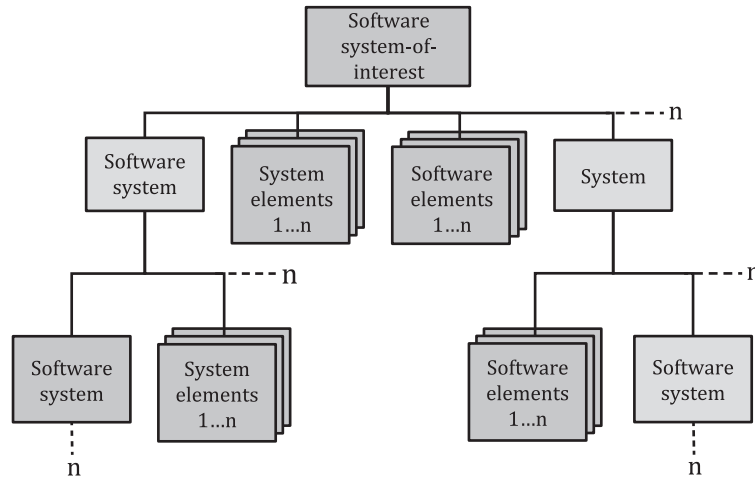


Figure 2 — Example of software system-of-interest structure

While [Figures 1](#) and [2](#) imply a hierarchical relationship among software system elements, many software systems, from one or more aspects, are not hierarchical, such as networks, digital twins, and other distributed or loosely coupled systems. Systems of systems (SoS) usually do not have a hierarchical structure.

NOTE 3 Decomposition, or analysis, is an activity fundamental to many software activities. Not all decompositions imply the designation of new software system elements and the corresponding recursive application of the activity. Designation of a decomposed construct as an element is necessary only when it is appropriate to apply distinct requirements, design, or implementation activities to its development. One example of an appropriate situation is when the element is to be developed by a distinct organization. Another example is when management determines that it is appropriate to distinctly monitor the status of the development or customization of the element.

5.2.3 Interfacing, enabling and interoperating systems

Interfacing, enabling and interoperating systems are outside the boundaries of the SoI, but are directly involved with it.

Interfaces among software systems are primarily sending, receiving, or exchanging data or information or controlling or monitoring functional performance. The most common types of software system interfaces are graphical user interfaces (GUI) and application programming interfaces (API). Indeed, a major part of software implementation and integration is accomplished through interfaces between existing software objects, libraries, programs, and systems. Interfaces with humans can be external or internal interfaces, i.e. human operators considered as part of the system.

Interoperating systems are a subset of interfacing systems. Systems that interact to perform a function are called interoperating systems. Interoperating systems can perform communication and networking functions, or exchange information to enable an SoI to operate reliably, securely, usefully, or efficiently, or to improve accessibility or usability. Interoperability is an important aspect in the context of SoS (see [5.2.4](#)). An interoperating system can also receive information from the SoI for use by other systems or SoS. In software systems, interoperation typically involves the exchange and use of information, or the ability of multiple software applications to function on the same software platform and operating system. Interoperating systems, including both software and hardware, can form the infrastructure for the software SoI. Infrastructure management is so important that it is one of the organizational project-enabling processes in this document ([6.2.2](#)).

Throughout the life cycle of an SoI, essential services are obtained from systems that are not directly a part of the SoI, e.g. a modelling system, training system, release management system, security monitoring system, or diagnostic maintenance system. Enabling systems in software development include software development and test environments for target platforms. Enabling systems comprise the infrastructure and

facilitate progression of the SoI through one or more stages in its life cycle. Almost all the life cycle processes in this document include activities to obtain the needed enabling systems.

An SoI and its interoperating and enabling systems can be considered as a solution addressing various stakeholder concerns. The various concerns of different stakeholders, based on their business models, drive the need to use different interoperating and enabling systems to make the SoI viable in their own system solution context. For example, the concerns of an SoI supplier, an acquirer and a user are different.

EXAMPLE A software SoI receives information about a user's account number and validity, allows deductions from the account value for payments, and sends data about the transaction back to a system of record. It interoperates with the primary system of record for customer accounts of a credit card issuer or other financial institution. It also interoperates with systems that accept authentication data (credit card readers). Enabling systems are used to update the SoI when new security controls are deployed, to monitor the SoI for excessive numbers of large transactions from unusual locations, or to update the SoI when communication protocols for financial transactions (e.g. blockchain) are revised.

The relationship between the services delivered to the operational environment by the SoI and the services delivered by the enabling systems to the SoI is shown in [Figure 3](#). Enabling systems X, Y, and Z can be seen to contribute indirectly to the services provided by the SoI. The interrelationships between the SoI and the enabling systems can be bidirectional or a one-way relationship. An interoperating or enabling system for the SoI can be primarily a software system or primarily a non-software (predominantly hardware) system. In addition to interacting with enabling systems, the SoI can also interact with interoperating and interfacing systems in the operating environment, shown as systems A, B and C. Requirements for interfaces with enabling systems and other systems in the operational environment are included in the requirements for the SoI.

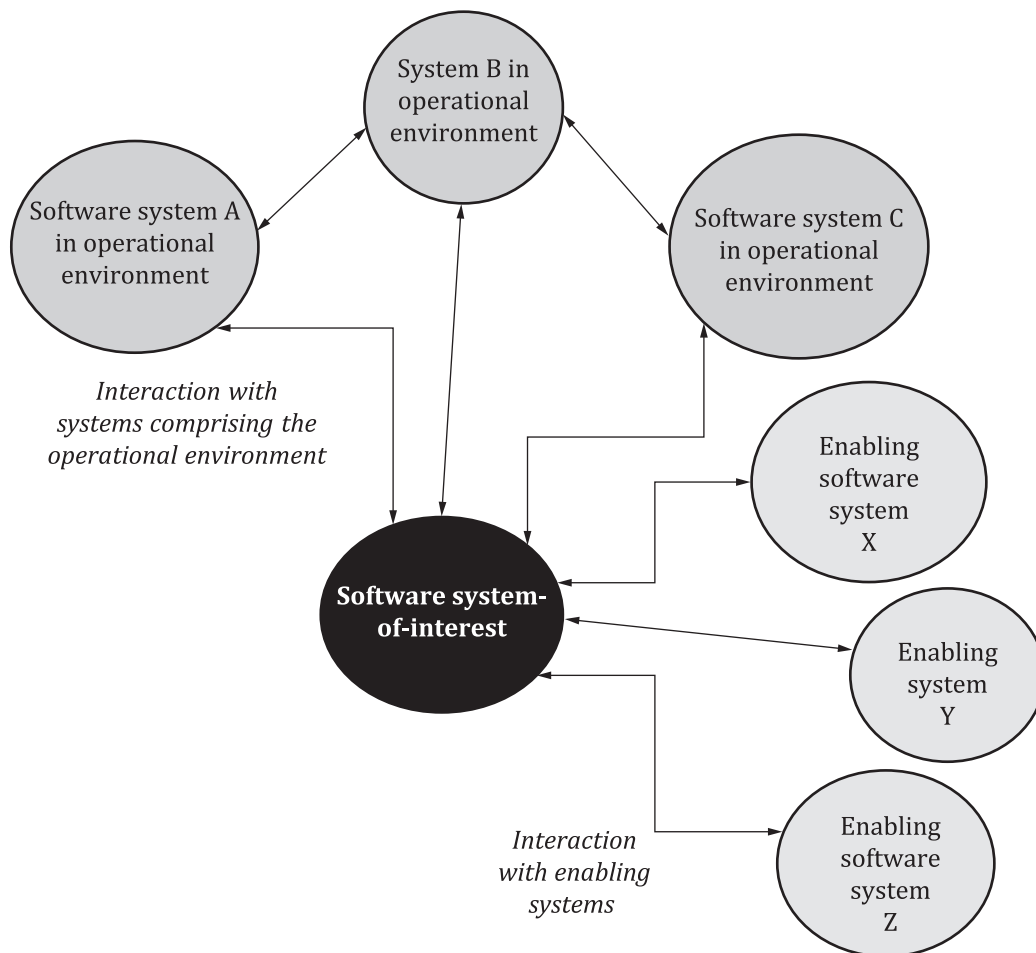


Figure 3 — Software system-of-interest, its operational environment and enabling systems

During a stage in the software life cycle, the relevant enabling systems and the SoI are considered together. Since they are interdependent, they can also be viewed as a system (or a system of systems. SoS). Creating the enabling system can be viewed as a separate project and subsequently as another SoI. When the software is provided as a service (SaaS) it can be considered as a distinct software product (system) with its own enabling systems, or the software with its infrastructure can be part of the deliverable product or service.

Variants and options can be specified for the SoI to address the set of solution contexts. Products and services are often grouped in system families and product lines with identification of elements common to different projects, and variants and options per project (see ISO/IEC 26550 for more details). Product line engineering has more nuanced concepts of system elements and system boundaries. A product line is comprised of a combination of common core elements along with product-specific configurations and components. For example, a software product line can include product variants to operate on different operating system platforms or include different functional options for different missions or market segments. Product line engineering is about managing those core components, such as choosing to bring a specific implementation from a product into the core set.

Development of systems, products and services often benefits from the identification of reuse opportunities between projects, including the establishment of product lines, families of products, systems and SoS. These assets available for projects are managed through the application of the configuration management, information management and knowledge management processes.

5.2.4 System of systems concepts

5.2.4.1 Differences between systems and SoS

An SoS is a set of systems that interact to provide a unique capability that none of the constituent systems can accomplish on its own. In the context of SoS, the relevant pieces of the SoI are, by definition, systems themselves. An SoS consists of some number of constituent systems, plus any inter-system infrastructure, facilities, and processes necessary to enable the constituent systems to integrate or interoperate.

Within an SoS, each constituent system is an independent system that forms part of an SoS. A constituent system can interact as part of one or more SoS in support of multiple capabilities. An SoS is often comprised of systems to enable it to interact in the anticipated SoS configurations. Each constituent system is a useful system by itself, having its own development, management, utilization, goals and resources, but interacts within the SoS to provide the unique capability of the SoS. These additional attributes are what distinguish SoS from a collection of systems.

In this document, when the interaction of an SoI with an SoS is discussed, this may include one or more SoS in support of one or more capabilities. The differences between systems and SoS are not in the structure or arrangement of the individual elements, but rather in the behavioural and managerial characteristics of those elements.

5.2.4.2 Managerial and operational independence

Systems operate within a context of managerial control subject to governance. Organizations govern a portfolio of programmes through goals and objectives, subject to laws, regulations and external agreements such as contracts. Programmes manage some number of projects to achieve those goals and objectives. Relationships between constituent systems affect the SoS. Systems that do not have any interactions with the constituent systems of a subject SoS are not part of that SoS.

An essential characteristic of the SoS is that constituent systems within the SoS are operationally independent. That is, the constituent systems fulfil some purposes on their own, separate from the SoS. While constituent systems operate independently from each other for their own purposes, they also operate interdependently with each other and other elements to produce the SoS outputs. Constituent systems are never totally independent, yet they are also never totally subservient to the SoS.

Another essential characteristic is that constituent systems within the SoS are both managerially independent and interdependent. Managerial independence suggests that the constituent systems can be managed by organizations that retain some degree of independence even though they are interdependent while participating in SoS. The implication is that these organizations can have goals and objectives for the

constituent systems that differ from those of the SoS and the other constituent systems. If so, there is likely some degree of independence and interdependence of governance, as well as some degree of independence and interdependence of management.

Regardless of the means of managing the organizations, alignment (or lack thereof) in the goals and objectives affects the SoS. While some constituent systems are directed or influenced to belong to SoS, owners of some constituent systems can be unaware of the SoS. Owners of some constituent systems choose to belong on a cost/benefits basis, also to cause greater fulfilment of their own purposes, and because of their belief in the overarching SoS purpose.

5.2.4.3 Taxonomy of SoS

ISO/IEC/IEEE 21841 can be useful when describing an SoS or comparing SoS. Using essential characteristics to partition the various types of SoS provides an abbreviated nomenclature for thinking about SoS. ISO/IEC/IEEE 21841 defines a normalised taxonomy for SoS to facilitate communications among stakeholders. It also briefly explains what a taxonomy is and how it applies to the SoS to aid in understanding and communication. There are many characteristics such as scale and scope, around which taxonomies can be derived.

The SoS taxonomy in ISO/IEC/IEEE 21841 organizes the relevant aspects or essential characteristics of SoS, providing specific viewpoints that align with stakeholder concerns. This organization facilitates communications between the various stakeholders that are involved with activities like governance, engineering, operation and management of these SoS, and provides a reference for other related standards.

5.2.4.4 SoS considerations in life cycle stages of a system

When the SoI is part of an SoS, the application of the processes in this document is dependent on the type of SoS life cycle and impact of the SoS interactions on the SoI. ISO/IEC/IEEE 21839 can be useful as an augmentation to this document when the SoI is a constituent system within an SoS, which is the typical situation for many systems. ISO/IEC/IEEE 21839 provides a set of SoS considerations to be addressed at key points in the life cycle of an SoI which is part of an SoS. The considerations and key points align with those which are introduced in this document. Selected subsets of these considerations can be applied throughout the life of systems through the involvement of stakeholders.

SoS considerations for the SoI include the technical impact on interacting systems and their stakeholders and infrastructure. ISO/IEC/IEEE 21839 states that “this includes both systems/services on which the SoI depends and systems/services that depend on the SoI”. The SoS configurations can impose new requirements or constraints on the SoI in support of required or desired business or mission capabilities. The goal is to achieve stakeholder satisfaction for the overall mission, so that when delivered, the SoI operates effectively in the operational environment which is typically characterized as one or more SoS.

5.2.4.5 Application of this document to SoS

ISO/IEC/IEEE 21840 provides guidance for the utilization of life cycle processes in the context of SoS. However, ISO/IEC/IEEE 21840 is not a self-contained SoS replacement for this document. ISO/IEC/IEEE 21840 can be used in conjunction ISO/IEC/IEEE 15288, ISO/IEC/IEEE 21839, and ISO/IEC/IEEE 21841.

In some cases, there can be “waves” of SoS revision. In other cases, SoS changes occur continually, with many processes operating on a continuous basis to implement evolutionary change.

5.3 Organization and project concepts

5.3.1 Organizations

For the purpose of this document, any project is assumed to be conducted within the context of an organization. This context is important, because a project is dependent upon various outcomes produced by the business processes of the organization, e.g. employees to staff the project and facilities to house the project. For this purpose, this document provides a set of organizational project-enabling processes. These processes are not assumed to be adequate to operate a business; instead, the processes, considered

as a collection, are intended to state the minimum set of dependencies that the project places upon the organization.

In informal terms, the organization that is responsible for executing a process is sometimes referred to by the name of that process. For example, the organization executing the acquisition process is sometimes called the acquirer. Other examples include supplier, implementer, integrator, maintainer and operator.

A few other terms are applied to organizations in this document. “User” can be the organization or individuals that directly engage with or benefit from the utilization of the product or service. “Customer” can refer to the user or to an acquirer, from the supplier's perspective. “Stakeholder” can refer to an individual or organization with an interest in the system.

The processes and organizations are only related functionally. This document does not dictate or imply a structure for an organization, nor does it specify that particular processes should be executed by particular parts of the organization. An organization that implements the life cycle processes in this document has the responsibility of defining a suitable structure for the organization and assigning appropriate roles for the execution of processes. An organization can be as small as a single individual, if the individual is assigned responsibilities and authorities.

The processes in this document form a comprehensive set to serve various organizations. An organization, small or large, depending on its business purpose or its acquisition strategy, can select an appropriate set of the processes (and associated activities and tasks) to fulfil that purpose. An organization can perform one process or more than one process.

This document is intended to be applied by an organization internally or externally by two or more organizations. Parties entering into an agreement can be from the same organization or from separate organizations. When applied internally, the two agreeing parties typically act under the terms of an agreement that may vary in formality under different circumstances. When applied externally, the two agreeing parties typically act under the terms of a contract. This document uses the term “agreement” to apply to either situation.

5.3.2 Organization and project-level adoption

An organization can choose to establish one or more projects for the life cycle of a system, product, or group of systems or product lines. Each project has its own life cycle of initiation, operation, and completion or transition.

Organizations using this document can strive to develop a robust set of life cycle processes that are applied repeatedly to their projects and services. Therefore, this document is intended to be useful for adoption at either the organization level or at the project level. An organization can adopt the document and supplement it with appropriate procedures, practices, tools, and policies. In turn, a project of the organization typically conforms to the organization's processes rather than conforming directly to this document.

In some cases, projects may be executed by an organization that does not have an appropriate set of processes adopted at the organizational level, or whose organizational processes are specialized for non-software production, such as hardware security or manufacturing. Such a project may apply the provisions of this document directly to the project.

5.3.3 Organization and collaborative activities

Ongoing and recurring activities for one or more life-cycle processes can also be handled by a team or group on a level-of-effort basis without established projects. Organizations that use agile methods can apply the life cycle processes identified in this document, including organizational, technical management, and technical processes (and can operate under the agreement processes). Due to the increasing complexities of system solutions, it is often useful to employ collaborative and concurrent engineering approaches across the system life cycle. The following are some considerations:

- System life cycle activities are performed in a collaborative manner by involving stakeholders and subject matter experts concurrently, as practical.

- A collaborative framework can be defined within an organization or between organizations to facilitate the involvement of the range of stakeholders and experts. A collaborative framework includes shared methods, tools, and other resources, and establishes an environment for better communication and shared vision and values.
- Collaborative engineering is more important for iterative or incremental development approaches to help ensure timely feedback and communication across the stakeholders.

DevOps can be considered as an extended use of collaboration, supported by integrated tools, among development and operational groups, which can increase the velocity and the reliability of software. Site reliability engineering implements DevOps goals of collaboration and tooling, but is concerned with reliability for very large operational environments, where reliability applies to the entire infrastructure as an SoS, rather than to individual SoI.

NOTE ISO/IEC/IEEE 32675 includes principles for the implementation of DevOps to define, control, and improve software life cycle processes.

5.4 Life cycle concepts

5.4.1 Software life cycle stages

A system life cycle model is segmented by stages to facilitate planning, provisioning, operating, and supporting the SoI. This segmentation by stages provides an orderly progression of a system through established decision-making gates. These decision gates are used by organizations to understand and manage the inherent uncertainties and risks associated with costs, schedule and functionality when creating or utilizing a software system. The stages relate to the state of the software system description or the software system itself. The stages describe the major progress and achievement milestones of the software system through its life cycle. Using stages thus provides organizations with a framework within which organization management has high-level visibility and control of project and technical processes.

Organizations define and employ stages differently to satisfy contrasting business and risk mitigation strategies. In ISO/IEC/IEEE 24748-1, the typical system life cycle stages include concept, development, production, utilization, support, and retirement. Use of these terms to define stages is not normative. A common set of stages for a software system is concept exploration, development, sustainment, and retirement, with transitions between stages for the system as a whole and for its elements. Often, the development stage is subdivided more finely and in different ways. Life cycle models vary according to the nature, purpose, use and prevailing circumstances of the software system.

This document does not prescribe a specific life cycle model with a fixed set or sequence of processes or stages. The sequence of the stages and processes is determined by project objectives and by selection of the software system life cycle model. Using stages concurrently and in different orders can lead to life cycle forms with distinctly different characteristics.

The life cycle processes defined in this document are not aligned to any specific stage in a software life cycle. All the life cycle processes involve planning, performance, and evaluation activities that should be considered for activities in every stage.

Further elaboration of these concepts can be found in the ISO/IEC/IEEE 24748 series, on the application of life cycle management.

5.4.2 Life cycle models for the software system

Every software system has a life cycle. A life cycle can be described using an abstract functional model that represents the conceptualization of a need for the system, its realization, utilization, evolution, and disposal. Software system elements, hardware system elements, and constituent systems can have different life cycles and life cycle models. A software system progresses through its life cycle as the result of activities, performed and managed by people in organizations, using life cycle processes for defining the activities. The detail in the life cycle model is expressed in terms of these processes, their outcomes, relationships, and sequence.

This document does not prescribe any specific life cycle model. Instead, it defines a set of processes that can be used in the definition of the system's life cycle. The processes described in this document support sequential as well as iterative and incremental development models.

NOTE 1 ISO/IEC/IEEE 24748-1 includes a set of steps and guidance for life cycle model selection and adaptation.

NOTE 2 ISO/IEC/IEEE 21839 provides additional guidance, grouped by life cycle stage, for SoIs that are an SoS or a constituent system in an SoS.

One oft-cited set of stages associated with software development includes elicitation, requirements, design, construction, and testing, known as the predictive sequential, or “waterfall” model. If the stages are considered as sequential, then each stage is expected to produce correct results before proceeding to the next stage. In practice, this is extremely difficult to achieve unless the requirements are known well and the initial cost estimates are accurate. In performing a waterfall, one risks performing extensive rework that does not properly fall within any of the planned stages, hence probably does not fall within any budget.

NOTE 3 Winston Royce, commonly recognized as an early analyst of life cycle process models, described the need for rework stages rather than the “waterfall” (a term that he did not use). Unfortunately, the rework stages were dropped from the “waterfall” model as it was popularly understood. Royce argued that a highly sequential model is one that “invites failure.”^[95]

To deal with conditions such as incompletely known requirements and inaccurate estimates, other types of life cycle models have been proposed: incremental, spiral, iterative, and evolutionary (adaptive). These life cycle models can incorporate agile techniques and methods. These models can typically involve repeated performance of the life cycle processes and stages during the life cycle, for example, for different increments of the software product, for more precise handling of exceptions to common functions, or for requirements that were not fully defined at the outset. These models can be applied across stages, such as development and utilization or deployment. Use of these life cycle models can affect software release strategies and acquisition strategies for software services.

The incremental life cycle model includes initial planning, initial requirements analysis, initial architectural definition, and initial validation, but allocates design, implementation, verification (and sometimes delivery) activities to a series of stages, each of which provides a portion of the intended functionality. The approach provides for some flexibility to respond to inaccurate cost or schedule estimates by moving functionality to later increments.

The spiral life cycle model orders the development and utilization of functionality based on risk, with the riskiest problems considered in the early increments. This approach provides some protection against cost and schedule surprises occurring late in the development cycle.

When the same process or set of processes is repeated on the same system element, the process model is referred to as iterative. The iterative life cycle model performs initial planning and then consists of repeated cycles of prototyping, testing, analysing, and refining the requirements and the solution. Iterative life cycle models repeatedly apply the life cycle processes to deliver prioritized system functions sooner, with refined or more complex elements of the system coming in later iterations.

NOTE 4 Applying the same process or set of processes to progressively more detailed system components is called a recursive process model.

The evolutionary life cycle model is intended to deal with incomplete knowledge of requirements. It provides for initial planning and initial system architecture definition, but allocates requirements analysis, design, construction, verification, validation, delivery, and utilization to a series of stages. Delivered capabilities that do not meet user needs can be added or reworked in subsequent stages of the system evolution.

In large software development efforts as well as in smaller projects, many agile methods can be used with various life cycle models, and different methods can be used at different points in the lifecycle. The process requirements in this document are appropriate for commonly used agile techniques.

Agile methods can be applied within a variety of models. While agile methods are common in executing an evolutionary lifecycle model, they can be used in incremental lifecycle models at various stages. As stated in [5.4.1](#), this document does not prescribe any sequence of processes within a life cycle model. The sequence of the processes is determined by project objectives and by selection of the life cycle model. An agile project,

because it transforms or combines activities while creating or improving working software, may find it more appropriate to claim full conformance to outcomes (4.2.1) rather than full conformance to activities and tasks (4.2.2).

NOTE 5 ISO/IEC 33202 contains additional information on agile practices.

Agile methods succeed in part because of the nature of software, which accommodates flexibility in design while the software is being constructed. In agile practice, software design, implementation (construction), and continuous integration are frequently performed concurrently. This practice is contrasted with a formal top-down approach to traceability in which construction cannot begin until design is approved so that the constructed software is traced to a previously approved detailed design. Thus, agile projects take full advantage of the approach in this document in which processes occur concurrently, as contrasted with sequentially staged (idealized waterfall) projects.

Similarly, DevOps, which is often combined with agile, emphasizes continuous integration, continuous delivery, and frequent transitions (releases) of software.

In agile projects, concept exploration, development, construction, testing, transition, and retirement of previous software can be performed concurrently for successive iterations. Agile projects typically perform replanning concurrently with development activities. In such approaches, using the end of a stage as a management checkpoint or control is not very useful. Other agile approaches perform replanning at points between designated iterations (e.g. sprints, iterations, or pre-defined time-boxed cadences) so that each of these iterations can be treated as a stage.

Agile projects can have closely tied cycles for development and software release (e.g. with daily, weekly, monthly scheduled releases) or can separate completion of development iterations from management of scheduled software releases, for the convenience of the customer or according to organizational strategy.

EXAMPLE Software elements can be developed incrementally, and then held for block operational release at a convenient time in the organization's business cycle.

What the models and methods have in common is an emphasis on continuous inspection and collaboration in the rapid production of working software, in an environment where changes, including changes to requirements, are expected. These methods help counter the risk of poor communication among multiple teams.

NOTE 6 Selecting the name of a type of model does not satisfy the requirement to define a model comprised of stages, with defined purpose and outcomes accomplished via the processes of this document.

5.5 Process concepts

5.5.1 Criteria for processes

The determination of the life cycle processes in this document is based upon three basic principles:

- a) Each life cycle process has strong relationships among its outcomes, activities, and tasks.
- b) The dependencies among the processes are reduced to the greatest feasible extent.
- c) A process is capable of execution by a single organization in the life cycle.

5.5.2 Description of processes

Each process of this document is described in terms of the following attributes:

- a) The title conveys the scope of the process as a whole.
- b) The purpose describes the goals of performing the process.
- c) The outcomes express the observable results expected from the successful performance of the process.
- d) The activities are sets of cohesive tasks of a process.

- e) The tasks are requirements, recommendations, or permissible actions intended to support the achievement of the outcomes.

NOTE 1 Additional detail regarding this form of process description can be found in ISO/IEC/IEEE 24774.

NOTE 2 Outputs are an optional attribute for a process description. They are artefacts or information items. Typical outputs of the processes are identified in [Annex B](#).

The processes and process groups in this document are identical in their purpose and outcomes with those in ISO/IEC/IEEE 15288, to assist in the application of life cycle processes in software engineering and systems engineering. Software-specific activities, tasks, and work products (artefacts and information items) are applied to achieve the outcomes of the processes in this document.

The processes described in this document are not intended to preclude or discourage the use of additional processes that organizations find useful. The order of the subclauses in which the processes are defined in this document does not determine the order in which the processes are performed during the system life cycle or any of its stages.

5.5.3 General characteristics of processes

In addition to the basic attributes described in the previous subclause, processes may be characterized by other attributes common to all processes. ISO/IEC 33020 identifies common process attributes that characterize six levels of achievement within a measurement framework for process capability.

5.5.4 Process views

There are cases where a unified focus is needed for activities and tasks that are selected from disparate processes to provide visibility to a significant concept or thread that cuts across the processes employed across the life cycle. For this purpose, the concept of a process view has been formulated. Like a process, the description of a process view includes a statement of purpose and outcomes. Unlike a process, the description of a process view does not include unique activities and tasks. Instead, the description includes guidance explaining how the outcomes can be achieved by employing the activities and tasks of the various life cycle processes. The detailed information about process views is available in ISO/IEC/IEEE 24748-1 and ISO/IEC/IEEE 24774. The following International Standards provide details on some of the technical viewpoints:

- ISO/IEC/IEEE 24748-1: specialty engineering, interface management, and security;
- ISO/IEC/IEEE 15026-4: systems assurance and software assurance.

5.5.5 Tailoring

If an organization decides to tailor any of the processes in this document, the activities of the tailoring process defined in [Annex A](#) shall be performed. Tailoring can diminish the perceived value of a claim of conformance to this document, because it is difficult for other organizations to understand the extent to which tailoring may have deleted desirable provisions. An organization asserting a single-party claim of conformance to this document may find it advantageous to claim full conformance to a smaller list of processes rather than tailored conformance to a larger list of processes.

NOTE See ISO/IEC/IEEE 24748-1 for additional information about selection and adaptation of life cycle models, as well as selection and tailoring of processes.

5.6 Process groups

5.6.1 Introduction

This document groups the activities that can be performed during the life cycle of a software system into four process groups. Each of the life cycle processes within those groups is described in terms of its purpose and desired outcomes with a set of related activities and tasks that can be performed to achieve those

outcomes. The four process groups and the processes included in each group are depicted in [Figure 4](#) as follows:

- a) agreement processes;
- b) organizational project-enabling processes;
- c) technical management processes;
- d) technical processes.

A description of each process group is provided in [5.6.2](#), [5.6.3](#), [5.6.4](#), and [5.6.5](#).

NOTE Additional interpretation of software concepts and processes can be found in ISO/IEC/IEEE 24748-3. More information about process application for the SoS is provided in ISO/IEC/IEEE 21839 and ISO/IEC/IEEE 21840.

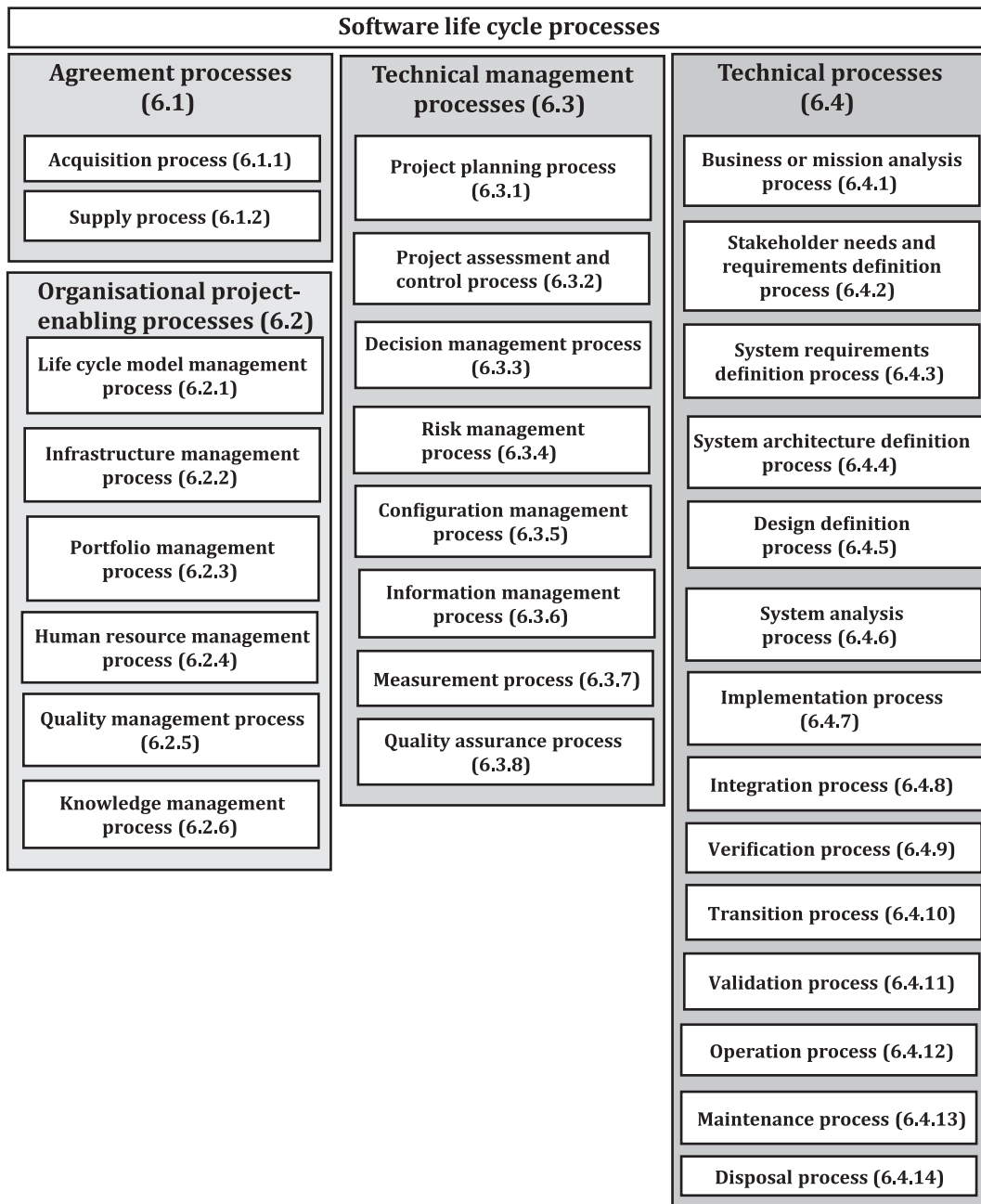


Figure 4 — Software life cycle processes

5.6.2 Agreement processes

Organizations are producers and users of software systems. Using agreements, one organization (acting as an acquirer) can task another (acting as a supplier) for products or services. Agreements allow both acquirers and suppliers to realize value and support business strategies for their organizations.

Generally, organizations act simultaneously or successively as both acquirers and suppliers of systems. The agreement processes can be used with less formality when the acquirer and the supplier are in the same organization. Similarly, they can be used within the organization to agree on the respective responsibilities of organization, project, and technical functions.

These processes define the activities necessary to establish an agreement between two organizations. If the acquisition process is invoked, it provides the means for interacting with a supplier. The agreement can include products that are supplied for use as an operational system, services in support of operational activities, or elements of a system being provided by a supplier. If the supply process is invoked, it provides the means for an agreement for a product or service that is provided to the acquirer.

NOTE 1 Security is an increasing concern in systems engineering. See the ISO/IEC 27036 series for requirements and guidance for suppliers and acquirers on how to secure information in supplier relationships. Specific aspects of information security supplier relationships are addressed in ISO/IEC 27036-3 and ISO/IEC 27036-4.

NOTE 2 ISO/IEC/IEC 41062 includes detailed processes for software acquisition.

When an SoI participates as part of a managed SoS, its resource and capability dependencies can be reflected in the agreement processes. As needed, the agreements can include clauses for the SoS interactions and dependencies. The processes apply for the agreements between the stakeholders of the SoI and interoperating systems. If there is an external entity with some type of responsibility that spans an SoS in which the SoI is a constituent system, then management and support arrangements can be required with that external entity. The agreements establish the responsibilities and modes of support and control across the life cycle stages among the stakeholder organizations from the context of the SoI's participation in the SoS. This participation is of greater importance if the organization holds primary objectives for their constituent system that are not directly aligned with those of the SoS.

5.6.3 Organizational project-enabling processes

The organizational project-enabling processes establish the environment and activities supporting software engineering. This process group is concerned with providing the resources to enable the project to meet the needs and expectations of the organization's stakeholders. The organizational project-enabling processes are typically concerned at a strategic level with the management and improvement of the organization's business or undertaking, with the provision and deployment of resources and assets, and with its management of risks in competitive or uncertain situations. The organizational project-enabling processes apply outside the span of a project's life, as well as during a project's lifespan. The organizational project-enabling processes are not intended to be a comprehensive set of organizational processes that enable strategic management or ongoing governance or administrative operations of the organization.

Concepts, needs and requirements evolve at various levels resulting from the establishment of or changes to the organizational concept of operations, strategy, or environment. The concept of operations addresses the leadership's intended way of operating the organization. This evolution is accomplished through the application of the technical processes: business or mission analysis, stakeholder needs and requirement definition, system requirements definition, system architecture definition, and design definition processes, with the support of other processes as needed. Through the iterative and concurrent use of all these processes, stakeholders are identified and insights are gained into the relationships between the concepts, needs, and requirements of the organization, stakeholders, and systems, as well as the emergent properties and behaviours of the system that arise from the interactions and relations among the system elements and environment.

The organizational project-enabling processes do not necessarily imply commercial or profit-making motives. The organizational project-enabling processes are equally relevant to non-profit organizations, since they are also accountable to stakeholders, are responsible for resources, and encounter risk in their undertakings. This document can be applied to non-profit organizations as well as to profit-making organizations. [Figure 4](#) lists the processes contained in this process group.

In a typical Sol, organizational project-enabling processes establish the environment and provide the necessary resources for the conduct of projects to address system solutions. The organization establishes the processes and life cycle models to be used by projects; establishes, redirects, or cancels projects; provides resources required, including human, material and financial; and sets and monitors the quality measures for systems and other deliverables that are developed by projects for internal and external customers. These processes also provide the environment and resources for the Sol to be able to support capabilities provided by an SoS in which the Sol participates. The organizations responsible for the constituent systems implement these processes for their Sol independent of the SoS. These processes can be influenced by regulations, interface standards, or agreements for those parts of the Sol that contribute to the overall SoS capabilities.

5.6.4 Technical management processes

The technical management processes are concerned with managing the resources and assets allocated by organization management and with applying them to fulfil the agreements into which the organization or organizations enter. Technical management processes are applied during the performance of each technical process. The technical management processes relate to the technical effort of projects, in particular to planning in terms of cost, timescales, and achievements, to the checking of activities to help ensure that they comply with plans and performance criteria, and to the identification and selection of corrective actions that recover shortfalls in progress and achievement. These processes are used to establish and perform technical plans for the project, manage information across the technical team, assess technical progress against the plans for the software system, products, or services, control technical tasks through to completion, and aid in decision-making.

In this document, the project is chosen as the context for describing processes. The same processes can also be applied in the development and performance of services, the development and support of products and product lines, or ongoing level-of-effort activities. Individual technical management processes may be invoked at any time in the life cycle and at any level in a hierarchy of projects, as required by plans or unforeseen events. The technical management processes are applied with a level of rigor and formality that depends on the agreements as well as risk and complexity of the project.

The technical management processes do not comprise a management system or a comprehensive set of processes for project management; they are included so that system-specific technical processes can be conducted effectively.

Typically, several projects co-exist in any one organization. The technical management processes can be employed at various organizational levels to meet internal needs. [Figure 4](#) lists the processes contained in this process group.

Project planning and project assessment and control are key to all management practices. Software projects or approaches applying the technical management processes all have plans and are driven by those plans, including agile approaches with planning for each sprint and each release. These processes establish the general approach for managing a project or a process.

The other processes in this group provide a specific focused set of tasks for performing to a specialised management objective. The technical management processes are applied to the management of any undertaking, ranging from a complete organization down to a single life cycle process and its tasks.

Technical management processes are concerned with managing the resources and assets allocated by organization management and with applying them to fulfil the agreements into which the organization or organizations enter. The technical management processes need to include the considerations of the expected SoS interactions. The considerations span the planning, assessment and management of resources, risks, and other factors associated with dependencies from other interacting systems. For example, the planning, assessment, and control activities need to include SoS-related cost and schedule considerations for the Sol, such as monitoring the progress in the areas of cross-system dependencies. The agreement processes are executed to negotiate the required resources across stakeholders.

5.6.5 Technical processes

The technical processes are concerned with technical activities throughout the life cycle of the Sol. Technical processes transform the needs of stakeholders into a product or service. By applying that product or

operating that service, technical processes, provide sustainable performance, when and where needed to meet the stakeholder requirements and achieve customer satisfaction. The technical processes are applied to create and use a software system, whether it is in the form of a model or is an operational product. The technical processes apply to any element in a software system and at any stage in the life cycle. [Figure 4](#) lists the processes contained in this process group.

The technical processes define the activities that enable organization and project functions to optimise the benefits and reduce the risks that arise from technical decisions and actions. These activities enable products and services to possess the timeliness and availability, the cost effectiveness, and the functionality, reliability, maintainability, producibility, usability and other quality characteristics required by acquiring and supplying organizations. They also enable products and services to conform to the expectations, ethical perspectives, or legislated requirements of society.

These processes are often performed concurrently, iterating between one another to establish a solution that is balanced with respect to requirements, critical performance measures, critical quality characteristics, and SoS considerations. At any level of abstraction, system requirements and models are made consistent via iterations of applicable technical processes. When requirements and models are not directly capable of being implemented, the same processes are repeated recursively throughout the system.

The technical processes are performed for the provision and support of technical solutions across the life cycle stages. The SoS technical considerations apply to all the technical processes across the life cycle stages and play an especially important role in the business or mission analysis, stakeholder needs and requirements definition, system requirements definition, system architecture definition, and design definition processes.

NOTE Interface management is a set of activities that cut across the systems engineering processes. These are cross-cutting activities of the technical and technical management processes that apply and track as a specific view of the processes and system. See ISO/IEC/IEEE 24748-1 for an example interface management process view and INCOSE-TP-2003-002-5, Part III, section 3.2.4 for more information.

5.7 Process application

5.7.1 Selection and timing of process use

The life cycle processes defined in this document can be used by any organization when acquiring, using, creating, or supplying a software system. The functions these processes perform are defined in terms of specific purposes, outcomes, and the set of activities and tasks that constitute the process.

Each life cycle process in [Figures 4](#) and [5](#) can be invoked, as required, at any time throughout the life cycle. The order that the processes are presented in this document does not imply any prescriptive order in their use. However, sequential relationships are introduced by the definition of a life cycle model. The detailed purpose and timing of use of these processes throughout the life cycle are influenced by multiple factors, including social, trading, organizational and technical considerations, each of which can vary during the life of a software system. An individual software life cycle is thus created through a selection and application of processes that normally possess concurrent, iterative, recursive, and time-dependent characteristics.

Concurrent use of processes can exist within a project (e.g. when design actions and preparatory actions for building a software system are performed at the same time) and between projects (e.g. when system elements are designed at the same time under different project responsibilities).

The recursive use of processes, i.e., the repeated application of the same process or set of processes applied to successive levels of system elements in a system's structure, is a key aspect of the application of this document. The outputs of processes at any level, whether information, artefacts, or services, are inputs to the processes used at the level below (e.g. during top-down design) or at the level above (e.g. during software system realization). The outcomes from one application are used as inputs to the next lower (or higher) system in the system structure to arrive at a more detailed or mature set of outcomes. Such an approach adds value to successive systems in the system structure.

The changing nature of the influences on the software system (e.g. operational environment changes, new opportunities for system element implementation, modified structure, and responsibilities in organizations)

requires continual review of the selection and timing of process use. Process use in the life cycle can be dynamic, responding to the many external influences on the software system. The life cycle approach also allows for incorporating the changes in the next stage. The life cycle stages assist the planning, execution, and management of life cycle processes in the face of this complexity in life cycles by providing comprehensible and recognizable high-level purpose and structure. The set of processes within a life cycle stage are applied with the common goal of satisfying the exit criteria for that stage or the entry criteria of the formal progress reviews within that stage.

The discussion in this clause on iterative and recursive use of software life cycle processes is not meant to imply any specific hierarchical, vertical, or horizontal structure for the SoI, enabling system, organization, or project.

Where justified by product quality risks, detailed descriptions of process instances in the context of the specific product may also be created. Instantiation of processes involves identifying specific success criteria for a process instance, derived from the product requirements, and identifying the specific activities and tasks needed to achieve the success criteria, derived from the activities and tasks identified in this document. Creating detailed descriptions of process instances enables better management of product quality risks by establishing the link between the process and the specific product requirements.

Further elaboration of these concepts can be found in the ISO/IEC/IEEE 24748 series on the application of life cycle processes.

5.7.2 Process iteration, recursion, and concurrency

When the application of the same process or set of processes is repeated on the same system, the application is referred to as iterative. The iterative use of processes is important for the progressive refinement of process outputs, for example, the interaction between successive verification tasks and integration tasks can incrementally build confidence in the conformance of the product. Iteration is not only appropriate but also expected. New information is created by the application of a process or set of processes. Typically, this information takes the form of questions with respect to requirements, analysed risks or opportunities. Iterative application of a process or set of processes should continue until such questions are resolved

Iteration between business or mission analysis, stakeholder needs and requirements definition, system requirements definition, system architecture definition, and design definition processes can help achieve a common understanding of the problem to be solved and the identification of a satisfactory solution ([Figure 5](#)). These are heavily supported by the system analysis and decision management processes. Information produced by the processes should be shared and used by all other system life cycle processes.

The repeated application of the same process or set of processes applied to successively more detailed system elements is the recursive use of processes. From a view of relations between a system and its system elements, the outputs of processes applied for a system or system element, whether information, artefacts, or services, are inputs to other processes or other system elements for additional analysis or more generalised synthesis of its system elements, to arrive at a more detailed or mature set of outcomes. Such an approach adds value to systems at successive levels in the system.

Concurrent use of processes can exist within a project (e.g. when design and preparatory activities for building a system are performed at the same time), and between projects (e.g. when system elements are designed at the same time under different project responsibilities). All processes can be used in parallel with other processes. As an example, the operation and maintenance processes need to provide input to the system requirements definition, system architecture definition, design definition, and implementation processes.

5.8 Process reference model

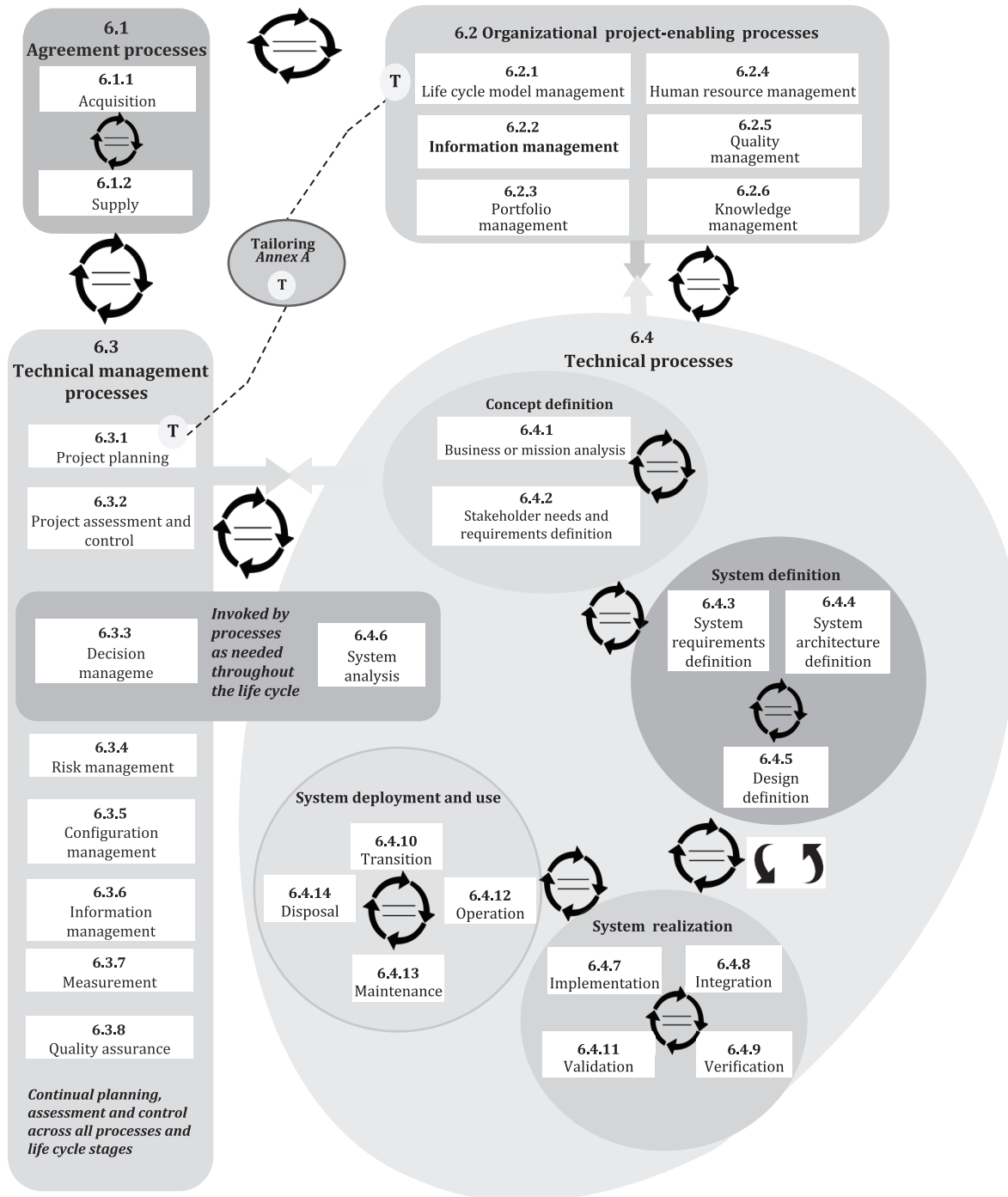
In this document, a process reference model (PRM) is specified that is consistent with ISO/IEC 33001. [Annex C](#) defines the PRM in this document at a level of abstraction higher than the detailed requirements contained in [Clause 6](#). The PRM in [Annex C](#) is characterized in terms of the process purpose and the process outcomes that result from the successful execution of the activity tasks. This document can therefore be used as a reference model to support process assessment as specified in ISO/IEC 33002. The PRM is applicable to an organization that is assessing its processes to determine the capability of these processes. The purpose and

outcomes are a statement of the goals of the performance of each process. This statement of goals permits assessment of the effectiveness of the processes in ways other than simple conformity assessment.

[Figure 5](#) illustrates the interrelationships among processes defined in this document. The arrows in [Figure 5](#) show general relationships that include iterative, recursive, and concurrent application. All possible relationships are not included. The actual flows or interactions between the processes for a project are determined by the project tailoring and needs. Also, the arrows are not intended to indicate any specific temporal relationships, sequences, or scheduling. The concurrency iteration icons between process groups or aggregations of processes within process groups are intended to indicate that the project can apply the processes in any order, can iterate between processes, and can implement them concurrently.

The technical management processes are continually applied to manage and control the processes and life cycle stages. The system analysis process provides data and information that are essential for each iteration of the systems life cycle processes, which are performed to support any of the technical management processes ([6.3](#)) and the technical processes ([6.4](#)).

The technical processes described in [Clause 6](#) interact with other technical and technical management processes to provide necessary inputs and information. For example, the system analysis process provides analysis results to support trade-offs that are managed by the decision management process. Additionally, the establishment of the concepts, requirements, architecture, and design are informed by the other technical processes, such as the operations and maintenance processes.



Key



concurrency, iteration



recursion

Figure 5 — Interrelationships among processes

6 Software life cycle processes

6.1 Agreement processes

This subclause specifies the requirements for the establishment of agreements with organizational entities external and internal to the organization.

The agreement processes consist of the following:

- a) acquisition process, used by organizations for obtaining products or services;
- b) supply process, used by organizations for providing products or services.

These processes define the activities necessary to establish an agreement between two organizations. If the acquisition process is invoked, it provides the means for conducting business with a supplier. Acquisition can include products that are supplied for use as an operational software system, services in support of operational activities, software elements of a system, or elements of a software system being provided by a supplier. If the Supply process is invoked, it provides the means for an agreement in which the result is a product or service that is provided to the acquirer. The agreement processes can also accommodate jointly developed software where both parties are both acquirer and supplier, with provisions specified for the ownership or rights to use the resulting work product.

NOTE Security is an increasing concern in systems and software engineering. See the ISO/IEC 27036 series for requirements and guidance for suppliers and acquirers on how to secure information in supplier relationships. Specific aspects of information security supplier relationships are addressed in ISO/IEC 27036-3 and ISO/IEC 27036-4.

6.1.1 Acquisition process

6.1.1.1 Purpose

The purpose of the acquisition process is to obtain a product or service in accordance with the acquirer's requirements.

6.1.1.2 Outcomes

As a result of the successful performance of the acquisition process:

- a) A request for supply is prepared.
- b) One or more suppliers are selected.
- c) An agreement is established between the acquirer and supplier.
- d) A product or service complying with the agreement is accepted.
- e) Acquirer obligations defined in the agreement are satisfied.
- f) Responsibility for the acquired product or service is transferred, as directed by the agreement.

NOTE Agreements vary in formality of the agreement (written or not) and procedures for communications between the acquirer and the supplier, and as to whether they are legally enforceable contracts, and whether they are made within an organization or with external suppliers.

6.1.1.3 Activities and tasks

The acquirer shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the acquisition process.

NOTE 1 The activities and resulting agreement from this process often apply to suppliers in the supply chain, including subcontracted suppliers.

NOTE 2 ISO/IEC/IEEE 41062 contains detailed activities for software acquisition alternatives, including custom-developed, off-the-shelf, and software as a service. It also provides software acquisition guidance for technical data rights and protection of intellectual property, safety assurance or information security.

a) Prepare for the acquisition. This activity consists of the following tasks:

1) Define a strategy for how the acquisition is to be conducted.

NOTE 3 This strategy describes or references the life cycle model, risks and issues mitigation, a schedule of milestones, and selection criteria if the supplier is external to the acquiring organization. It also includes key drivers and characteristics of the acquisition, such as responsibilities and liabilities; specific models, methods, or processes; level of criticality; formality; and priority of relevant trade-off factors.

NOTE 4 The decision management process and the system analysis process are often used to support trade-offs for the acquisition strategy. Examples include determining a make or buy decision, as well as the suitability of specific COTS or modified OTS solutions, and supplier evaluation.

NOTE 5 If the strategy calls for acquisition of specific COTS software or open-source software, acquisition can be limited to identifying the supplier, accepting or negotiating the conditions in a pre-defined license or lease or maintenance agreement, determining rights to intellectual property and data rights in the software system, and agreeing on the price.

NOTE 6 Factors to consider in agreements between suppliers include data rights and lateral access to constituent data and intellectual property. As an example, suppliers for one system element can collaborate with suppliers for another element and share source code. Agreements can enable this collaboration.

2) Prepare a request for the supply of a product or service that includes the requirements.

NOTE 7 If a supplier is external to the organization, then the request includes the business practices and can refer to the applicable laws and regulations with which a supplier is expected to comply. Suppliers can be requested to supply evidence of their capabilities, experience, and qualifications. The request also indicates the criteria for selecting a supplier.

NOTE 8 The requirements are the defined stakeholder or the system requirements (6.4.2 or 6.4.3), depending on the acquisition approach. Security requirements are typically included, such as processes or features for data integrity, provenance data (software bill of materials, SBOM), and incident response procedures. When an external supplier proposes operations, maintenance, and support services for a proposed software product, the acquirer can agree to acquire the proposed software product with or without the proposed services. If both the proposed product and services are selected, they typically are the subjects of separate related agreements.

NOTE 9 The acquirer develops the requirements by itself or retains a supplier to develop them. If the acquirer retains a supplier to develop requirements, the acquirer retains approval authority for the requirements developed by the supplier.

b) Advertise the acquisition and select the supplier. This activity consists of the following tasks:

1) Communicate the request for the supply of a product or service to potential suppliers.

NOTE 10 The supplier's review of the request can result in questions and responses, which can lead to the acquirer's modifying its request or suppliers modifying their proposals.

2) Select one or more suppliers.

NOTE 11 To obtain competitive solicitations, proposals to supply are evaluated and compared against the selection criteria and ranked. The justification for rating each proposal is declared and suppliers commonly are informed why they were or were not selected. If no supplier's proposal is found suitable, the acquisition can be terminated.

c) Establish and maintain an agreement. This activity consists of the following tasks:

NOTE 12 Project cost, schedule, and performance are monitored through the project assessment and control process (6.3.2). Any identified issues that require agreement modifications are referred to this activity. Any proposals for changes to system elements or information are controlled through the change management activity of the configuration management process.

- 1) Develop an agreement with the supplier that includes acceptance criteria.

NOTE 13 This agreement ranges in formality from a written contract to a verbal agreement. Appropriate to the level of formality, the agreement establishes requirements, development and delivery milestones, verification, validation and acceptance conditions, exception-handling procedures, agreement change management procedures, and payment schedules, so that both parties of the agreement understand the basis for executing the agreement. Other provisions for agreements include rights and restrictions associated with technical data and intellectual property, acceptance test preparation and test environment details; and the extent of supplier involvement. The agreement identifies process requirements specified for participating subcontractors, such as security, configuration management, reporting of risks, and reporting of measures and measurement analysis.

NOTE 14 Acceptance criteria, such as acceptance testing, relate to how the product or service satisfies its intended use in its operational environment. Acceptance testing can be performed using the validation process. Exceptions that arise during the conduct of the agreement or with the delivered product or service are resolved according to the procedures established in the agreement.

- 2) Identify necessary changes to the agreement.

NOTE 15 In requesting a change to the agreement, the acquirer or the supplier details its specifications, rationale, and background. The agreement is modified when a change request affecting the terms of the agreement is agreed to by both the acquirer and supplier.

- 3) Evaluate impact of changes on the agreement.

NOTE 16 Any change is investigated for impacts to project plans, schedule, cost, technical capability, and quality. A change can be handled within the existing agreement, can require a modification to the agreement, or can require a new agreement.

- 4) Negotiate the agreement with the supplier.

NOTE 17 Agreement terms are negotiated between the acquirer and supplier. Negotiation occurs for the initial agreement, and as required for any changes. Changed agreements are based on the required change and identified impacts. Details are discussed and changed during negotiation, after which the acquirer and supplier accept the terms of an agreement and the agreement commences. For a written contract, this occurs when the contract is signed or as specified in the agreement.

- 5) Update the agreement with the supplier, as necessary.

NOTE 18 The result of the agreement modification is incorporated into the project plans and communicated to all affected parties.

NOTE 19 Agreements can specify the conditions under which the agreement may be terminated by either party, e.g. unexpected changes in strategy or available funding, or lack of satisfactory progress.

- d) Monitor the agreement. This activity consists of the following tasks:

- 1) Assess the execution of the agreement.

NOTE 20 Suppliers' processes and products can be audited, re-qualified or periodically reviewed using the quality assurance, verification and validation processes. This evaluation can include confirmation that all parties are meeting their responsibilities according to the agreement. The project assessment and control process is used to evaluate projected cost, schedule, performance, and the impact of undesirable outcomes on the organization. This information is combined with other assessments of the execution of the terms of the agreement. If execution of the agreement does not result in an acceptable product or service, the acquirer or supplier can terminate the agreement as allowed in its terms.

- 2) Provide data needed by the supplier and resolve issues in a timely manner.

- e) Accept the product or service. This activity consists of the following tasks:

- 1) Confirm that the delivered product or service complies with the agreement.

NOTE 21 Acceptance testing can be performed using the validation process. Exceptions that arise during the conduct of the agreement or with the delivered product or service are resolved according to the procedures established in the agreement.

NOTE 22 If the agreed requirements are satisfied and the acceptance criteria are met, the supplier has fulfilled its obligation. Unaddressed exceptions, for example, disputes over conduct of acceptance testing or product suitability for intended use, are a matter for agreement provisions relating to disputes, arbitration or applicable law and regulation.

- 2) Provide payment or other agreed consideration.
- 3) Accept the product or service from the supplier, or other party, as directed by the agreement.

NOTE 23 Iterative or incremental delivery and acceptance of and payment for products or services can occur as specified in the agreement

- 4) Close the agreement.

NOTE 24 The project is closed by the portfolio management process.

6.1.2 Supply process

6.1.2.1 Purpose

The purpose of the supply process is to provide an acquirer with a product or service that meets agreed requirements.

6.1.2.2 Outcomes

As a result of the successful performance of the supply process:

- a) An acquirer for a product or service is identified.
- b) A response to the acquirer's request is produced.
- c) An agreement is established between the acquirer and supplier.
- d) A product or service is provided.
- e) Supplier obligations defined in the agreement are satisfied.
- f) Responsibility for the acquired product or service is transferred as directed by the agreement.

6.1.2.3 Activities and tasks

The supplier shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the supply process.

- a) Prepare for the supply. This activity consists of the following tasks:
 - 1) Determine the existence and identity of an acquirer who has a need for a product or service.

NOTE 1 Potential acquirers can be identified through the business or mission analysis process. For a product or service developed for consumers, an agent, for example, a marketing function within the supplier organization, often represents the acquirer.

- 2) Define a supply strategy.

NOTE 2 This strategy describes or references the life cycle model, risks and issues mitigation, and a schedule of milestones. It also includes key drivers and characteristics of the acquisition, such as responsibilities and liabilities; specific models, methods, or processes; level of criticality; formality; and priority of relevant trade-off factors.

- b) Respond to a request for supply of products or services. This activity consists of the following tasks:
 - 1) Evaluate a request for the supply of a product or service to determine feasibility and how to respond.

- 2) Prepare a response that satisfies the solicitation.
- c) Establish and maintain an agreement. This activity consists of the following tasks:
 - 1) Negotiate an agreement with the acquirer that includes acceptance criteria.

NOTE 3 This agreement ranges in formality from a written contract to a verbal agreement. The supplier confirms that the requirements, delivery milestones, and acceptance conditions are achievable, that exception handling and agreement change management procedures and payment schedules are acceptable, and that they establish a basis for executing the agreement without unnecessary risks. The agreement can define the supplier's responsibility for applying appropriate quality management, assurance and control processes to produce suitable systems, products, or services. Issues are discussed and resolved during negotiation, after which the acquirer and supplier accept the terms of an agreement and the agreement commences. For a contract, this occurs when the contract is signed.

- 2) Identify necessary changes to the agreement.

NOTE 4 In requesting a change to the agreement, the acquirer or the supplier details its specifications, rationale, and background.

- 3) Evaluate impact of changes on the agreement.

NOTE 5 Any change is investigated for impacts to project plans, schedule, cost, technical capability, or quality. A change can be handled within the existing agreement, can require a modification to the agreement, or can require a new agreement.

- 4) Negotiate the agreement with the acquirer, as necessary.

NOTE 6 As part of this process, the agreement is modified when a change request affecting the terms of the agreement is agreed to by both the acquirer and supplier. Modifications to agreements can be due to changing market context. Negotiation occurs for the initial agreement, and as required for any changes. Changed agreements are based on the required change and identified impacts.

- 5) Update the agreement with the acquirer, as necessary.

NOTE 7 The result of the agreement modification is incorporated into the project plans and communicated to all affected parties.

- d) Execute the agreement. This activity consists of the following tasks:

- 1) Execute the agreement according to the established project plans.

NOTE 8 A supplier sometimes adopts, or agrees to use, acquirer processes.

- 2) Assess the execution of the agreement.

NOTE 9 This evaluation includes confirmation that all parties are meeting their responsibilities according to the agreement. The project assessment and control process is used to evaluate projected cost, schedule, performance, and the impact of undesirable outcomes on the organization. The change management activity of the configuration management process is used to control changes to the system elements. This information is combined with other assessments of the execution of the terms of the agreement. If execution of the agreement does not result in an acceptable product or service, the acquirer or supplier can terminate the agreement as allowed in its terms.

- e) Deliver and support the product or service. This activity consists of the following tasks:

- 1) Deliver the product or service in accordance with the agreement criteria.

NOTE 10 As stated in the agreement, acceptance criteria, such as acceptance testing, relate to how the product or service satisfies its intended use in its operational environment. Unaddressed exceptions, for example, disputes over conduct of acceptance testing or product suitability for intended use, are a matter for agreement provisions relating to disputes, arbitration or applicable law and regulation.

NOTE 11 Delivery often is accomplished through a series of partial deliveries, acceptances, and compensation.

- 2) Assist the acquirer in support of the delivered product or service, per the agreement.
- 3) Accept and acknowledge payment or other agreed consideration.
- 4) Transfer the product or service to the acquirer, or other party, as directed by the agreement.
- 5) Close the agreement.

NOTE 12 The project is closed by the portfolio management process.

NOTE 13 Agreements can specify the conditions under which the agreement can be terminated by either party, for example, unexpected changes in strategy or available funding, or lack of satisfactory progress.

6.2 Organizational project-enabling processes

The organizational project-enabling processes help ensure the organization's capability to acquire and supply products or services through the initiation, support, and control of projects. These processes provide resources and infrastructure necessary to support projects and help ensure the satisfaction of organizational objectives and established agreements. They are not intended to be a comprehensive set of business processes that enable strategic management of the organization's business.

The organizational project-enabling processes consist of the following:

- a) life cycle model management process;
- b) infrastructure management process;
- c) portfolio management process;
- d) human resource management process;
- e) quality management process;
- f) knowledge management process.

6.2.1 Life cycle model management process

6.2.1.1 Purpose

The purpose of the life cycle model management process is to define, maintain, and help ensure availability of policies, life cycle processes, life cycle models, and procedures for use by the organization with respect to the scope of this document.

This process provides policies, life cycle processes, life cycle models, and procedures that are consistent with the organization's objectives. These life cycle assets are defined, adapted, improved, and maintained to support individual project needs in a way that they are capable of being applied using effective, proven methods and tools.

NOTE 1 Regulated domains sometimes can require specific life cycle management process standards, e.g. ANSI/AAMI/IEC 62304. ISO/IEC 5338 presents adaptations of the processes in this document for AI systems.

NOTE 2 The SWEBOK v4 discusses life cycles and life cycle management in its software engineering process knowledge area.

6.2.1.2 Outcomes

As a result of the successful performance of the life cycle model management process:

- a) Organizational policies and procedures for the management and deployment of life cycle models and processes are established.

- b) Roles, responsibility, accountability, and authority within life cycle policies, processes, models, and procedures are defined.
- c) Policies, life cycle processes, life cycle models, and procedures for use by the organization are selected.
- d) Policies, life cycle processes, life cycle models, and procedures for use by the organization are assessed.
- e) Prioritized process, model, and procedure improvements are implemented.

6.2.1.3 Activities and tasks

The organization shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the life cycle model management process.

- a) Establish the life cycle processes. This activity consists of the following tasks:

NOTE 1 The detail of the life cycle implementation within a project is dependent upon the complexity of the work, the methods used, and the skills and training of personnel involved in performing the work. A project tailors policies, processes, models, and procedures according to its requirements and needs, while maintaining alignment with regulations and organizational policies. [Annex A](#) contains information on tailoring.

- 1) Establish policies and procedures for life cycle process management and deployment that are consistent with organizational strategies.
- 2) Establish the life cycle processes that implement the requirements of this document and that are consistent with organizational strategies.
- 3) Define the roles, responsibilities, accountabilities, and authorities to facilitate implementation of life cycle processes and the strategic management of life cycles.

EXAMPLE The role of security champion or security lead focuses on security concerns throughout the life cycle.

- 4) Define business criteria that control progression through the life cycle.

NOTE 2 The decision-making criteria regarding entering and exiting each life cycle stage and key milestones are established. These are sometimes expressed in terms of business achievement.

- 5) Establish standard life cycle models for the organization that are comprised of stages and define the entry and exit criteria for each stage.

NOTE 3 The life cycle model comprises one or more stage models, as needed. It is assembled as a sequence of stages that can overlap or iterate, as appropriate for the Sol's scope, magnitude, complexity, changing needs and opportunities. Stages are illustrated in ISO/IEC/IEEE 24748-1, using a commonly encountered example of life cycle stages. Specific examples for systems and software are provided in ISO/IEC/IEEE 24748-2 and ISO/IEC/IEEE 24748-3. The life cycle processes and activities are selected, tailored as appropriate and employed in a stage to fulfil the purpose and outcomes of that stage.

- b) Assess the life cycle processes. This activity consists of the following tasks:

NOTE 4 The quality assurance process can help support the assessment of policies, procedures, and the life cycle process model. IEEE 730 and ISO/IEC 33002 provide a more detailed set of process assessment activities and tasks that can be aligned with the tasks shown below.

- 1) Monitor process execution across the organization.

NOTE 5 Monitoring the execution of processes includes monitoring performance, analysing process measures, and reviewing trends with respect to compliance with regulations, organizational policies, business criteria, and feedback from the projects regarding the effectiveness and efficiency of the processes.

- 2) Conduct periodic reviews of the life cycle models used by the projects.

NOTE 6 Periodic reviews include confirming the continuing suitability, adequacy and effectiveness of the life cycle models used by the projects and making improvements as appropriate. Reviews include the stages, processes and achievement criteria that control progression through the life cycle.

- 3) Identify improvement opportunities from assessment results.

NOTE 7 Improvements can affect the stages, processes, and achievement criteria that control progression through the life cycle.

- c) Improve the life cycle processes. This activity consists of the following tasks:

- 1) Prioritize and plan improvement opportunities.
- 2) Implement improvement opportunities and inform relevant stakeholders.

NOTE 8 Process improvement includes improvements to any of the processes in the organization. Lessons learned are captured and available.

6.2.2 Infrastructure management process

6.2.2.1 Purpose

The purpose of the infrastructure management process is to provide the infrastructure and services to projects to support organization and project objectives throughout the life cycle.

This process defines, provides, and maintains the facilities, tools, and communications and information technology assets needed for the organization with respect to the scope of this document.

6.2.2.2 Outcomes

As a result of the successful performance of the infrastructure management process:

- a) The needs for infrastructure are defined.
- b) The infrastructure elements are specified.
- c) Infrastructure elements are obtained.
- d) The infrastructure is available.
- e) Prioritized infrastructure improvements are implemented.

6.2.2.3 Activities and tasks

The organization shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the infrastructure management process.

- a) Establish the infrastructure. This activity consists of the following tasks:
 - 1) Define project infrastructure requirements.

NOTE 1 Infrastructure elements can include facilities, tools, hardware, software, services, and standards. In addition to the general infrastructure resources common to an organization to support its business processes, an organization can also provide projects with unique or shared enabling systems to support the project's technical processes.

NOTE 2 The infrastructure resource needs for the project are considered in context with other projects and resources within the organization, as well as within the policies and strategic plans of the organization. Business constraints and timelines that influence and control provision of infrastructure resources and services for the project are also evaluated. Project plans and future business needs contribute to the understanding of the resource infrastructure. Physical factors, for example, facilities, logistics needs, and human factors (including health and safety aspects), are also considered.

NOTE 3 A secured or segmented development infrastructure can be required to reduce internal and external threats. The ISO/IEC 27036 series provides guidance for addressing security of outsourced infrastructure.

- 2) Identify, obtain, and provide infrastructure resources and services that are needed to implement and support projects.

NOTE 4 An inventory asset registry or asset management system is often established to track infrastructure elements and support reuse of infrastructure assets.

- b) Maintain the infrastructure. This activity consists of the following tasks:
 - 1) Evaluate the degree to which delivered infrastructure resources satisfy project needs.
 - 2) Identify and provide improvements or changes to the infrastructure resources as the project requirements change.

6.2.3 Portfolio management process

6.2.3.1 Purpose

The purpose of the portfolio management process is to initiate and sustain necessary, sufficient, and suitable projects to meet the strategic objectives of the organization.

This process commits the investment of adequate organization funding and resources, and sanctions the authorities needed to establish selected projects. It performs continued assessment of projects to confirm they justify, or can be redirected to justify, continued investment.

Portfolio management can be performed at various levels or scales above the project level, such as for programs, departments, lines of business, or the entire enterprise, depending on the size and complexity of the organization and its strategic and business objectives

For software systems, portfolio management can also refer to the management of a product line, which may include a portfolio of assets, such as architecture models; products; a service catalogue; and enabling systems. This portfolio of assets can be used to meet organizational or customer needs and objectives and support changes in technology. Portfolio management for product lines can include identifying the need for a new product line, adding new products to existing product lines, and sustaining or retiring existing product lines. Management of assets is achieved through management of projects.

Agile projects for complex systems attempt to manage cost by prioritizing the most important capabilities for early realization. If an organization manages the development and maintenance of its entire portfolio of software systems as a single system, managed by spend rate rather than total spending, then the organization can, in principle, manage that portfolio of systems as a continuing agile development, analogous to managing a highly iterative “Kanban” maintenance effort.

6.2.3.2 Outcomes

As a result of the successful performance of the portfolio management process:

- a) Strategic venture opportunities, investments, or necessities are prioritized.
- b) Projects are identified.
- c) Resources and budgets for each project are allocated.
- d) Project management responsibilities, accountability, and authorities are defined.
- e) Projects meeting agreement and stakeholder requirements are sustained.
- f) Projects not meeting agreement or satisfying stakeholder requirements are redirected or terminated.
- g) Projects that have completed agreements and satisfied stakeholder requirements are closed.

6.2.3.3 Activities and tasks

The organization shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the portfolio management process.

a) Define and authorize projects. This activity consists of the following tasks:

- 1) Identify potential new or modified capabilities or missions.

NOTE 1 The organization business strategy, concept of operations, or gap analysis or opportunity analysis is reviewed for current gaps, problems, or opportunities. A new capability or enterprise need is usually determined in the business or mission analysis process, further defined in the stakeholder needs and requirements definition process, and managed through this process.

- 2) Prioritize, select and establish new business opportunities, ventures, or undertakings.

NOTE 2 These are usually consistent with the business strategy and action plans of the organization. The potential projects are prioritized, and thresholds established, to determine which projects are executed. The characteristics of identified projects are often determined, including stakeholder value, risks and barriers to success, dependencies and inter-relationships, constraints, resource needs and mutual contention for resources. Each potential project is then assessed with respect to likelihood of success and cost-benefit. The decision management and system analysis processes provide details on performing an analysis of alternatives.

- 3) Define projects, accountabilities, and authorities.
- 4) Identify the expected goals, objectives, and outcomes of each project.
- 5) Identify and allocate resources for the achievement of project goals and objectives.
- 6) Identify multi-project interfaces and dependencies to be managed or supported by each project.

NOTE 3 Managed interfaces and dependencies include the use or reuse of enabling systems used by more than one project and the use or reuse of common system elements, including software elements, by more than one project. Reuse of existing, well-secured capabilities to reduce the likelihood of introducing new security vulnerabilities is discussed by NIST SP 800-218.

NOTE 4 Understanding each project in the context of the enterprise architecture helps to ensure interfaces and constraints are identified.

- 7) Specify the project reporting requirements and review milestones that govern the execution of each project.
- 8) Authorize each project to commence execution of project plans.

NOTE 5 Refer to the project planning process for additional information on developing project plans. Project plans are most useful when developed and approved early in the project life cycle and iteratively refined.

b) Evaluate the portfolio of projects. This activity consists of the following tasks:

- 1) Evaluate projects to confirm ongoing viability.

NOTE 6 Viability includes the following criteria:

- i) The project is making progress towards achieving established goals and objectives.
- ii) The project is complying with project directives.
- iii) The project is being conducted according to approved project life cycle policies, processes, and procedures.

- iv) The project remains viable, as indicated by, for example, continuing need for the service, practicable product implementation, and acceptable investment benefits.
- 2) Act to continue or redirect projects that are satisfactorily progressing or can be expected to progress satisfactorily by appropriate redirection.
- c) Terminate projects. This activity consists of the following tasks:
 - 1) Where agreements permit, act to cancel or suspend projects whose disadvantages or risks to the organization outweigh the benefits of continued investments.

NOTE 7 Capture of lessons learned from cancelled or failing projects can be especially useful for organizational improvement or use on other projects.
 - 2) After completion of the agreement for products and services, act to close the projects.

NOTE 8 Closure is accomplished in accordance with organizational policies and procedures, and the agreement.

6.2.4 Human resource management process

6.2.4.1 Purpose

The purpose of the human resource management process is to provide the organization with necessary human resources and to maintain their competencies, consistent with strategic needs.

This process provides a supply of skilled and experienced personnel qualified to perform life cycle processes to achieve organization, project, and stakeholder objectives.

NOTE The SWEBOK v4 discusses aspects of human resource management in its Software Engineering Professional Practice knowledge area, including accreditation, certification, qualification, licensing, employment contracts, and group dynamics.

6.2.4.2 Outcomes

As a result of the successful performance of the human resource management process:

- a) Skills required by projects are identified.
- b) Necessary human resources are provided to projects.
- c) Skills of personnel are developed, maintained, or enhanced.
- d) Personnel conflicts are resolved.

NOTE Skills management enabling systems can provide processes and support for organizational planning, identification, development, management and maintenance of skills, personnel competencies, resources and assets to satisfy near and long-term project and product line human resource needs.

6.2.4.3 Activities and tasks

The organization shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the human resource management process:

- a) Identify skills. This activity consists of the following tasks:
 - 1) Identify skill needs based on current and expected projects, including skills in secure software development and operational practices.

- 2) Identify and record skills of personnel.
- b) Develop skills. This activity consists of the following tasks:
 - 1) Establish skills development strategy.

NOTE 1 The strategy can be included in a plan that defines types and levels of training, categories of personnel, schedules, personnel resource requirements, training needs, and methods to verify competency or proficiency.

- 2) Obtain or develop training, education, or mentoring resources.

NOTE 2 These resources include training materials that are developed by the organization or external parties, training courses that are available from external suppliers, and computer-based instruction.

- 3) Provide planned skill development.
- 4) Maintain records of skill development.

- c) Acquire and provide skills. This activity consists of the following tasks:

NOTE 3 Providing skills includes the recruitment and retention of personnel with experience levels and skills necessary to properly staff projects; staff assessment and review, for example, their proficiency, motivation, ability to work in a team environment, as well as the need to be retrained, reassigned, or reallocated.

- 1) Obtain qualified personnel when skill deficits are identified.

NOTE 4 Obtaining competent personnel can include using outsourced resources.

- 2) Maintain and manage the pool of skilled personnel necessary to staff ongoing projects.
- 3) Make project assignments based on project and staff-development needs.
- 4) Motivate personnel, e.g. through career development and reward mechanisms.
- 5) Control multi-project management interfaces to resolve personnel conflicts.

NOTE 5 Areas of conflict can include conflicts of capacity in organizational infrastructure and in supporting services and personnel resources among ongoing projects; or from project personnel being over-committed.

6.2.5 Quality management process

6.2.5.1 Purpose

The purpose of the quality management process is to assure that products, services, and implementations of the quality management process meet organizational and project quality objectives and achieve customer satisfaction.

Quality management includes both evaluation of quality assurance (QA) activities that have been performed and guidance identifying and implementing QA activities which should be performed to deliver and operate software products and services with suitable quality, consistent with business needs and constraints.

NOTE The SWEBOK v4 discusses quality management, including software quality improvement, software quality measurement, and corrective and preventive actions in its Software Quality knowledge area.

6.2.5.2 Outcomes

As a result of the successful performance of the quality management process:

- a) Organizational quality management policies, objectives, and procedures are implemented.
- b) Quality evaluation criteria and methods are established.

- c) Resources and information are provided to projects to support the operation and monitoring of project QA activities.
- d) QA evaluation results are analysed.
- e) Quality management policies and procedures are improved, based upon project and organizational results.

6.2.5.3 Activities and tasks

The organization shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the quality management process.

NOTE 1 Refer to ISO 9001 for information and requirements to establish a quality management system. ISO/IEC/IEEE 90003 provides guidance for applying the quality requirements of ISO 9001 to software.

- a) Plan quality management. This activity consists of the following tasks:

- 1) Establish quality management objectives, policies and procedures.

NOTE 2 ISO 9004 contains guidelines for performance improvements.

NOTE 3 The policies, objectives, and procedures are based on the business strategy for customer satisfaction and risk management considerations.

- 2) Define responsibilities and authority for implementation of quality management.

NOTE 4 Resources for QM are often assigned from distinct organizations for independence from project management. An organization's QM function can provide continuity across projects, while being separate from direct control of project management.

- 3) Define quality evaluation criteria and methods.

- 4) Provide resources and information for quality management.

- b) Evaluate quality management. This activity consists of the following tasks:

- 1) Gather and analyse quality assurance evaluation results, in accordance with the defined criteria.
- 2) Assess customer satisfaction.

NOTE 5 ISO 10004 contains guidelines for monitoring and measuring customer satisfaction. The quality of the software system is also demonstrated by user satisfaction.

- 3) Conduct periodic reviews of project QA activities for compliance with the QM policies, objectives, and procedures.

NOTE 6 Quality evaluation criteria and methods are established. QM reviews address the adequacy and appropriateness of QA methods and procedures, compliance with the project procedures, and product conformance with specified functionality and quality characteristics.

- 4) Monitor the status of quality improvements on processes, products, and services.

- c) Perform corrective and preventive action. This activity consists of the following tasks:

- 1) Plan corrective actions when QM objectives are not achieved.
- 2) Plan preventive actions when there is a sufficient risk that QM objectives cannot be achieved.
- 3) Monitor corrective and preventive actions to completion and inform relevant stakeholders.

NOTE 7 Implementation of corrective and preventive action is performed in other relevant processes, such as Life Cycle Model Management or Project Assessment and Control.

NOTE 8 ISO 9001:2015, 0.3.3 and A.4 describe preventive action to eliminate potential nonconformities as part of risk-based thinking.

6.2.6 Knowledge management process

6.2.6.1 Purpose

The purpose of the Knowledge Management process is to create the capability and assets that enable the organization to exploit opportunities to re-apply existing knowledge.

This encompasses knowledge, skills, and knowledge assets, including system elements.

NOTE The re-application of existing knowledge is known as knowledge reuse and includes the reuse of knowledge about or from software elements, such as reusable objects, code assets, style guides, large language models, and data stores.

6.2.6.2 Outcomes

As a result of the successful performance of the knowledge management process:

- a) A taxonomy for the application of knowledge assets is identified.
- b) The organizational knowledge, skills, and knowledge assets are organized.
- c) The organizational knowledge, skills, and knowledge assets are available.
- d) The organizational knowledge, skills, and knowledge assets are communicated across the organization.
- e) Knowledge management usage data is analysed.

6.2.6.3 Activities and tasks

The organization shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the knowledge management process:

- a) Plan knowledge management. This activity consists of the following tasks:

- 1) Define the knowledge management strategy.

NOTE 1 This knowledge management strategy generally includes the following:

- i) identifying domains and their potential for the reapplication of knowledge;
- ii) plans for obtaining and maintaining knowledge, skills, and knowledge assets for their useful life;
- iii) characterization of the types of knowledge, skills, and knowledge assets to be collected and maintained;
- iv) criteria for accepting, qualifying, and retiring knowledge, skills, and knowledge assets;
- v) procedures for controlling changes to the knowledge, skills, and knowledge assets;
- vi) plans, mechanisms and procedures for protection, control and access to classified or sensitive data and information, based on the principle of least privilege;
- vii) mechanisms for storage and retrieval.

NOTE 2 Knowledge management includes knowledge shared both locally within the organization and knowledge that is shared outside the organization (and by federation) with designated stakeholders, acquirers, and business partners, subject to intellectual property and non-disclosure agreements.

- 2) Identify the knowledge, skills, and knowledge assets to be managed.

- 3) Identify projects that can benefit from the application of the knowledge, skills, and knowledge assets.
- b) Share knowledge assets throughout the organization. This activity consists of the following tasks:
 - 1) Establish a taxonomy to organize knowledge assets.

NOTE 3 The taxonomy includes the following:

 - i) definition of the boundaries of domains and their relationships to others;
 - ii) domain models capturing essential common and different features, capabilities, concepts, and functions;
 - iii) an architecture for a family of systems within the domain, including their common and different features.

NOTE 4 See ISO/IEC 26550 for more information on product line models. Refer to ISO/IEC/IEEE 42010 for requirements on architecture frameworks, viewpoints, model kinds, views, and models.
 - 2) Develop or acquire knowledge assets.

NOTE 5 Knowledge assets include system elements or their representations (e.g. reusable code libraries, reference architectures) architecture or design elements (e.g. architecture or design patterns), processes, criteria, or other technical information (e.g. training materials) related to domain knowledge, and lessons learned.
 - 3) Share knowledge assets across the organization.

NOTE 6 Automated search capabilities improve access to knowledge assets.
- c) Manage knowledge, skills, and knowledge assets. This activity consists of the following tasks:
 - 1) Monitor and record the reuse of knowledge, skills, and knowledge assets.
 - 2) Periodically reassess the currency of technology and market needs for the knowledge assets.
 - 3) Assess the business benefits which the organization gained through knowledge management practices.

6.3 Technical management processes

The technical management processes are used to establish and evolve plans, to execute the plans, to assess actual achievement and progress against the plans, and to control execution through to fulfilment. Individual technical management processes may be invoked at any time in the life cycle and at any level in a hierarchy of projects, as required by plans or unforeseen events. The Technical Management processes are applied with a level of rigor and formality that depends on the risk and complexity of the project.

The scope of a technical management process is the technical management of a project or an ongoing effort practice, or its products, to include the software product or SoI.

NOTE This set of technical management processes is performed so that software system-specific technical processes can be conducted effectively. They do not comprise a management system or a comprehensive set of processes for project management, as that is not the scope of this document.

The technical management processes consist of the following:

- a) project planning process;
- b) project assessment and control process;
- c) decision management process;
- d) risk management process;

- e) configuration management process;
- f) information management process;
- g) measurement process;
- h) quality assurance process.

Project planning and project assessment and control processes are key to all management practices. These processes establish the general approach for managing a project or a process. The other processes in this group provide a specific focused set of tasks for achieving a specialized management objective. They are all evident in the management of any undertaking, ranging from a complete organization down to a single life cycle process and its tasks. In this document, the project has been chosen as the context for describing processes. The same processes can also be applied in the performance of services.

6.3.1 Project planning process

6.3.1.1 Purpose

The purpose of the project planning process is to produce and coordinate effective and workable plans.

This process determines the scope of the project management and technical activities, identifies process outputs, tasks, and deliverables, establishes schedules for task conduct, including achievement criteria, and required resources to accomplish tasks. This is an ongoing process that continues throughout a project, with regular revisions to plans.

NOTE 1 The strategies defined in each of the other processes provide inputs and are integrated in the Project planning process. The Project Assessment and Control process is used to assess whether the plans are integrated, aligned, and feasible.

NOTE 2 The SWEBOK v4 discusses project planning, including determining deliverables, estimation (effort, schedule, and cost), and resource allocation in the Software Engineering Management knowledge area. ISO/IEC/IEEE 16326 provides additional information on project planning.

6.3.1.2 Outcomes

As a result of the successful performance of the project planning process:

- a) Objectives and plans are defined.
- b) Roles, responsibilities, accountabilities, and authorities within the project are defined.
- c) Performance and achievement criteria are defined.
- d) Resources and services necessary to achieve the objectives are committed.
- e) Plans for the execution of the project are activated.

6.3.1.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the project planning process.

- a) Define the project. This activity consists of the following tasks:

- 1) Identify the project objectives and constraints.

NOTE 1 Objectives and constraints include performance and other quality aspects, cost, time and customer and user satisfaction. Each objective is identified with a level of detail that permits selection, tailoring and implementation of the appropriate processes and activities.

NOTE 2 The ISO/IEC/IEEE 15026 series, ISO/IEC 27001, and the ISO/IEC 27036 series provide additional guidance on objectives and constraints related to assurance and security.

- 2) Define the project scope as established in the agreement.

NOTE 3 The definition of project scope includes the relevant activities required to satisfy business decision criteria and complete the project successfully. A project can have responsibility for one or more stages in the complete software system life cycle. Project planning includes defining appropriate actions for maintaining project plans, performing assessments, and controlling the project.

- 3) Define and maintain a life cycle model that is comprised of stages using the defined life cycle models of the organization.

NOTE 4 ISO/IEC/IEEE 24748-1 provides detailed information regarding life cycle stages and the definition of an appropriate life cycle model. It defines a general set of exemplar system life cycle stages, including concept, development, production, utilization, support, and retirement. It also identifies a generic exemplar set of software life cycle stages, including needs determination, concept exploration and definition, demonstration and evaluation, engineering/development, production/manufacturing, deployment/sales, operations, maintenance and support, and retirement.

- 4) Establish a work breakdown structure (WBS) based on the deliverable products or the evolving architecture of the software system.

NOTE 5 Each element of the software system architecture, and appropriate processes and activities are described with a level of detail that is consistent with identified risks. Related tasks in the work breakdown structure are grouped for performance. Project tasks identify work items being developed or produced. For projects with agile or iterative methods, a WBS element can correspond to the primary features, from a user perspective, to be produced during iterations.

NOTE 6 ISO 21511 includes detailed guidance for project WBS.

- 5) Define and maintain the processes that are applied on the project.

NOTE 7 These processes are based on the defined processes of the organization (see life cycle model management process). [Annex A](#) contains information on tailoring that can be used to address project-specific needs. The definition of the processes includes the entry and exit criteria, inputs, process sequence constraints (predecessor/successor relationships), process concurrency requirements (what processes and tasks are worked concurrently with other process area tasks or activities), Measures of effectiveness/measures of performance attributes, and scope and cost parameters (for critically important cost estimation).

NOTE 8 Identifying interfaces with other projects or organizational units is addressed through the Portfolio Management process.

- b) Plan project and technical management. This activity consists of the following tasks:

- 1) Define and maintain a project schedule based on management and technical objectives and work estimates.

NOTE 9 The project schedule defines the duration, relationship, dependencies and sequence of activities, achievement milestones, resources employed, and the reviews and schedule reserves for risk management necessary to achieve timely completion of the project.

- 2) Define achievement criteria for the life cycle stage decision gates, delivery dates and major dependencies on external inputs or outputs.

NOTE 10 The time intervals between internal reviews are defined in accordance with organizational policy on issues such as business and system criticality, schedule, and technical risks.

- 3) Define the costs and plan a budget.

NOTE 11 Budgeted costs are based on the schedule, software size and complexity estimates, labour estimates, infrastructure costs, procurement items, acquired service and enabling system estimates, and budget reserves for risk management.

- 4) Define roles, responsibilities, accountabilities, and authorities.

NOTE 12 This framework for use of human resources includes defining the project organization, staff acquisitions, and the development of staff skills. Authorities include, as appropriate, the legally responsible roles and individuals, for example, design authorization, safety authorization, and those responsible for applicable certifications or accreditations.

5) Define the infrastructure and services required.

NOTE 13 Infrastructure planning includes defining the capacity needed, its availability and its allocation to project tasks. Infrastructure includes facilities, services, tools, communications, and information technology assets. The requirements for enabling systems and services for each life cycle stage are also specified.

6) Plan the acquisition of materials and enabling systems and services supplied from outside the project.

NOTE 14 Acquisition of external resources includes, as necessary, plans for solicitation, supplier selection, acceptance, contract administration and contract closure. The agreement processes are used for the planned acquisitions.

NOTE 15 The ISO/IEC 27036 series provides guidance for secure acquisition of infrastructure and services.

7) Generate and communicate a plan for project and technical management and execution, including reviews.

NOTE 16 Technical planning for the software system is often captured in a system engineering management plan (SEMP) or a software engineering management plan or a software development plan (SDP). ISO/IEC/IEEE 24748-5 provides more detail on software engineering technical management planning and includes an annotated outline for an SDP. Planning for the project is often captured in a project management plan. ISO/IEC/IEEE 16326 provides more detail on project planning.

NOTE 17 The strategy activities and tasks from each of the other processes provide inputs and are integrated in the project planning process. The project assessment and control process is used to help ensure that the plans are integrated, aligned, and feasible.

c) Activate the project. This activity consists of the following tasks:

1) Obtain approval to start the project.

NOTE 18 Approval to start (authorization to proceed) is provided through the portfolio management process.

2) Submit requests and obtain commitments for necessary resources to perform the project.

NOTE 19 Commitment of resources includes access to enabling systems or services.

3) Implement project plans.

6.3.2 Project assessment and control process

6.3.2.1 Purpose

The purpose of the project assessment and control process is to assess if the plans are aligned and feasible; determine the status of the project, technical, and process performance; and direct execution to help ensure that the performance is according to plans and schedules, within projected budgets, to satisfy project objectives.

This process evaluates, periodically and at major events, the progress and achievements against requirements, plans, and overall strategic objectives. Information is provided for management action when significant variances are detected. This process also includes redirecting the project activities and tasks, as appropriate, to correct identified deviations and variations from other technical management or technical processes. Redirection may include re-planning as appropriate.

Agile organizations have specific practices for the project planning and project assessment and control processes. Rather than establishing major control points at the transition between stages or processes, agile projects often hold checkpoints or retrospective reviews at the end of a time-boxed cycle to agree on improvements for the next cycle. Each iteration includes design, development, and test activities (test-driven development). After a sprint of approximately one to four weeks or longer, new working software elements are accepted as “done” — regarded by the agile team as complete and ready to use. Done typically means that user stories delivered in a sprint completely designed, developed, verified (tested) and validated, with necessary information for users. Lessons learned and process improvements are identified, and work begins on another sprint. Continuing learning, risk management, and process improvement can be facilitated by planning meetings that initiate each iteration and retrospective meetings held at the end of each iteration.

NOTE The SWEBOK v4 discusses project assessment and control, including monitoring and controlling projects, reporting, reviewing and evaluating performance, measurement, and closure in the Software Engineering Management knowledge area.

6.3.2.2 Outcomes

As a result of the successful performance of the project assessment and control process:

- a) Performance measures or assessment results are available.
- b) Adequacy of roles, responsibilities, accountabilities, authorities, and resources is assessed.
- c) Technical progress reviews are performed.
- d) Deviations in project performance from plans are analysed.
- e) Affected stakeholders are informed of project status.
- f) Corrective action is directed when project achievement is not meeting targets.
- g) Project replanning is initiated, as necessary.
- h) Project action to progress (or not) from one scheduled milestone, decision gate, or event to the next is authorized.

6.3.2.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the project assessment and control process.

- a) Plan for project assessment and control. This activity consists of the following task:

- 1) Define the project assessment and control strategy.

NOTE 1 The strategy identifies the expected project assessment and control activities, including planned assessment methods and timeframes, and necessary management and technical reviews.

- b) Assess the project. This activity consists of the following tasks:

- 1) Assess alignment of project objectives and plans with the project's environment, opportunities, and constraints.
- 2) Assess management and technical plans against the project objectives to determine adequacy and feasibility.
- 3) Assess project and technical status against appropriate plans to determine actual and projected cost, schedule, and performance variances.
- 4) Assess the adequacy of roles, responsibilities, accountabilities, and authorities.

NOTE 2 This assessment of the adequacy of the human resource framework includes the adequacy of personnel competencies to perform project roles and accomplish project tasks. Objective measures are used wherever possible, for example, efficiency of resource use, project achievement.

5) Assess the adequacy and availability of resources.

NOTE 3 Resources include infrastructure, personnel, funding, time, or other pertinent items. This task includes evaluating the reuse of existing processes and infrastructure resources and confirming that intra-organizational commitments are satisfied.

6) Assess progress using measured achievement and milestone completion.

NOTE 4 This assessment of progress includes collecting and evaluating data for labour, material, service costs, and technical performance, as well as other technical data about objectives, such as affordability. These are compared against measures of achievement. This includes conducting effectiveness assessments to determine the adequacy of the evolving software system against requirements. It also includes the readiness of enabling systems to deliver their services when needed.

7) Conduct required management and technical reviews, audits, and inspections.

NOTE 5 These are formal or informal, and are conducted to determine readiness to proceed to the next stage of the life cycle or project milestone, to help ensure that project and technical objectives are being met, or to obtain feedback from stakeholders.

8) Monitor critical processes and new technologies.

NOTE 6 This task includes identifying and evaluating technology maturity and feasibility of technology implementation. Technology maturity is the readiness of a technology for operational use, and is often measured on a scale from low (exists as a concept only) to high (proven in operational use).

9) Analyse measurement results and make recommendations.

NOTE 7 Measurement results are analysed to identify deviations, variations or undesirable trends from planned values that include potential concerns, and to make appropriate recommendations for corrections or preventive actions. This analysis includes, where appropriate, statistical analysis of measures that indicate trends, for example, fault density to indicate quality of outputs, distribution of measured parameters that indicate process repeatability.

10) Record and provide status and findings from assessment tasks.

NOTE 8 These are generally designated in the agreement, policies and procedures.

11) Monitor process execution within the project.

NOTE 9 Process execution includes the analysis of process measures and review of trends with respect to project objectives. Improvement actions can be handled through the quality assurance process, the quality management process, or the life cycle model management process.

c) Control the project. This activity consists of the following tasks:

1) Initiate necessary actions needed to address identified issues.

NOTE 10 Preventive, corrective, and problem resolution actions are applied when project or technical achievement is not meeting planned targets. Actions generally require replanning or reassignment of personnel, tools, and infrastructure assets when inadequacy or unavailability has been detected, or when project or technical achievement exceeds targets or plan. They often impact the cost, schedule, or technical scope. Actions sometimes involve changes to the implementation and execution of the life cycle processes.

NOTE 11 Actions are recorded and reviewed to confirm their adequacy and timeliness.

2) Initiate necessary project replanning.

NOTE 12 Project replanning is initiated when project objectives or constraints have changed, or when planning assumptions are shown to be invalid.

NOTE 13 Any change that requires a change to the agreement between acquirer and supplier invokes the acquisition and supply processes.

- 3) Initiate change actions when there is a contractual change to cost, schedule, or quality due to the impact of an acquirer or supplier request.

NOTE 14 Change actions can include modified terms and conditions for supply or initiating new supplier selection, which invokes the acquisition and supply processes.

- 4) Authorize the project to proceed toward the next milestone or event, if justified.

NOTE 15 The project assessment and control process is used to reach agreement on milestone completion.

6.3.3 Decision management process

6.3.3.1 Purpose

The purpose of the decision management process is to provide a structured, analytical framework for objectively identifying, characterizing, and evaluating a set of alternatives for a decision at any point in the life cycle and select the most beneficial course of action.

This process is used to resolve technical or project issues and respond to requests for decisions encountered during the software life cycle, to identify the alternatives that provides the preferred outcomes for the situation. The methods most frequently used for Decision Management are the trade-off study, cost-benefit analysis, and engineering analysis. Each of the alternatives is assessed against the decision criteria e.g. cost impact, schedule impact, programmatic constraints, regulatory implications, technical performance characteristics, critical quality characteristics such as safety, security, ethical values, and risk. Results of these comparisons are ranked, via a suitable selection model, and are then used to decide on an optimal solution. Key study data, e.g. assumptions and decision rationale, are typically managed to inform decision-makers and support future decision-making.

NOTE 1 When it is necessary to perform a detailed assessment of a parameter for one of the criteria, the system analysis process can be employed to perform the assessment.

NOTE 2 The SWEBOK v4 discusses decision management in several knowledge areas. The Software Engineering Economics knowledge area describes the engineering decision-making process along with various applications of that process. The Software Architecture knowledge area describes significant decisions in architecture definition. The Software Design knowledge area describes recording software designs and major design decisions.

6.3.3.2 Outcomes

As a result of the successful performance of the decision management process:

- a) Decisions requiring alternative analysis are identified.
- b) Alternative courses of action are evaluated.
- c) A preferred course of action is selected.
- d) The resolution, decision rationale, and assumptions are recorded.

6.3.3.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the decision management process.

- a) Prepare for decisions. This activity consists of the following tasks:
 - 1) Define a decision management strategy.

NOTE 1 A decision management strategy includes the identification of roles, responsibilities, accountabilities, and authorities. The strategy considers the need for obtaining information input and for returning a timely decision. It includes the identification of decision categories and a prioritization scheme. Decisions often arise as a result of an effectiveness assessment, a technical trade-off, a problem needing to be solved, an action needed as a response to risk exceeding the acceptable threshold, or a new opportunity or approval for project progression to the next life cycle stage. Organization or project guidelines determine the degree of rigor and formality to apply to the decision analysis.

- 2) Identify the circumstances and need for a decision.

NOTE 2 Problems or opportunities and the alternative courses of action that can resolve them are recorded, categorized, and reported.

- 3) Involve relevant stakeholders in the decision-making to draw on their experience and knowledge.

NOTE 3 It is good practice to identify who is accountable for the decision, as well as the subject matter expertise needed for the analysis and the decision.

- b) Analyse the decision information. This activity consists of the following tasks:

- 1) Select and declare the decision management strategy for each decision.

NOTE 4 The degree of rigor required to resolve these problems or opportunities is determined, as well as the data and system analysis needed for evaluating the alternatives, and the timeframe to reach a decision.

- 2) Determine desired outcomes and measurable selection criteria.

NOTE 5 The weighting factors for the criteria, desired value for quantifiable criteria, and the threshold values beyond which the attribute is unsatisfactory are determined.

- 3) Identify the trade space and alternatives.

NOTE 6 If numerous alternatives exist, they are qualitatively screened to reduce alternatives to a manageable number for further detailed systems analysis. This screening is often based on qualitative assessments of such factors as risks and opportunities, cost, schedule, and regulatory impacts.

- 4) Evaluate each alternative against the criteria.

NOTE 7 The system analysis process is used, as necessary, to quantify specific criteria for each alternative to be evaluated. It can include new design parameters, different architecture characteristics, SoS considerations, and the range of values for critical quality characteristics. The system analysis process assesses the range of parameter variations to obtain a sensitivity analysis for each of the alternatives evaluated. These results are used to establish the feasibility of the various alternatives.

- c) Make and manage decisions. This activity consists of the following tasks:

- 1) Determine preferred alternative for each decision.

NOTE 8 Alternatives are evaluated quantitatively, using the selection criteria. The selected alternative generally provides an optimization of, or improvement in, an identified decision.

- 2) Record the resolution, decision rationale, and assumptions.

- 3) Record, track, evaluate and report decisions.

NOTE 9 Decision records identify problems and opportunities and their disposition, as stipulated in agreements or organizational procedures and in a manner that permits auditing and learning from experience.

NOTE 10 Evaluation of decisions can include whether problems were effectively resolved, unexpected risks and consequences emerged, and opportunities were successfully pursued.

6.3.4 Risk management process

6.3.4.1 Purpose

The purpose of the risk management process is to identify, analyse, treat, and monitor the risks continually.

The risk management process systematically addresses risk throughout the life cycle of a system, product, or service towards achieving objectives.

NOTE 1 ISO/IEC/IEEE 16085 provides a more detailed set of risk management activities and tasks. This risk management process is aligned with ISO 31000.

NOTE 2 Risk management can be applied to risks related to the acquisition, development, operation, maintenance, or retirement of a system.

6.3.4.2 Outcomes

As a result of the successful performance of the risk management process:

- a) Risks are identified.
- b) Risks are analysed.
- c) Risk treatments are selected.
- d) Appropriate treatments are implemented.
- e) Risks are evaluated to assess changes in status and progress in treatment.
- f) A risk profile is maintained.

6.3.4.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the risk management process.

NOTE 1 The risk management activities can be adapted to cover opportunities.

- a) Plan risk management. This activity consists of the following tasks:

- 1) Define the risk management strategy.

NOTE 2 The strategy typically defines the scope of the risk management process, the risk management approach, and risk criteria, measures, parameters, rating scale, and treatment alternatives. The strategy includes the risk management processes of suppliers and describes how risks from suppliers are raised.

NOTE 3 To additionally cover the management of opportunities, the strategy can include opportunities in the scope and approach, along with the opportunity criteria, measures, parameters, rating scale, and treatment alternatives.

NOTE 4 ISO 9001 and ISO/IEC/IEEE 90003 describe planning for risks and opportunities and additional risk-based thinking for strategic quality management. ISO/IEC 23894 and NIST.AI.100-1 include guidance on AI risk management. IEEE 7014-2024 defines risk management concerns for AI systems that emulate humans.

- 2) Define and record the context of the risk management process.

NOTE 5 The risk context includes stakeholders' perspectives on how much risk to accept (risk appetite), risk categories, and a description (perhaps by reference) of the technical and managerial objectives, assumptions, and constraints. The risk categories include the relevant technical areas of the software system and facilitate identification of risks across the product life cycle. As noted in ISO 31000, the aim of this step is to generate a comprehensive list of risks based on those events that can create, enhance, prevent, degrade, accelerate, or delay the achievement of objectives.

NOTE 6 Opportunities, which are one type of risk, provide potential benefits for the software system or project. Each of the opportunities pursued has associated risks that detract from the expected benefit: the risks associated with not pursuing an opportunity, as well as the risk of inadequate performance to achieve the opportunity.

b) Manage the risk profile. This activity consists of the following tasks:

- 1) Define and record the risk thresholds and conditions under which a level of risk may be accepted or subject to a treatment strategy.
- 2) Establish and maintain a risk profile.

NOTE 7 The risk profile records: the risk management context; a record of each risk's state including its source or category, likelihood of occurrence, detectability, consequences, and risk thresholds; the priority of each risk based on risk criteria supplied by the stakeholders; and the risk avoidance or risk mitigation actions along with the status of their treatment. The risk profile is updated when a new risk is identified, or there are changes in the risk context or probability of occurrence. The risk priority (according to the impact, probability, and feasibility of treatment) in the risk profile is used to determine the application of resources for treatment.

NOTE 8 The same risk profile can be applied to address opportunities when they have the same context.

NOTE 9 IEC 31010 includes several methods for identifying risks.

- 3) Periodically provide the relevant risk profile to stakeholders based upon their needs.

c) Analyse risks. This activity consists of the following tasks:

- 1) Identify risks in the categories described in the risk management context.

NOTE 10 Risks are commonly identified through various analyses, such as safety, reliability, security, and performance analyses; technology, architecture, and readiness assessments; and trade-off studies. These risks are often identified early in the life cycle and continue into the utilization, support, and retirement of the software system. Additionally, risks are often identified through the analysis of measurements of the evolving software system.

- 2) Estimate the likelihood of occurrence and consequences of each identified risk.

NOTE 11 Consequences of a risk typically involve technical, schedule, cost, or quality impacts.

- 3) Evaluate each risk against its risk thresholds.
- 4) For each risk that exceeds its risk threshold, define and record recommended treatment strategies and measures.

NOTE 12 Risk treatment strategies include, but are not limited to, eliminating the risk, reducing its likelihood of occurrence or severity of consequence, or accepting the risk. Opportunity treatment strategies include pursuing or exploiting the opportunity, deferring, or monitoring. Treatments also include taking or increasing risk to pursue an opportunity. Measures provide information about the effectiveness of the treatment alternatives.

d) Treat risks. This activity consists of the following tasks:

- 1) Identify recommended alternatives for risk treatment.
- 2) Implement risk treatment alternatives for which the stakeholders determine that actions should be taken to make a risk acceptable.
- 3) Once a risk treatment is selected, coordinate management activities.

NOTE 13 The project assessment and control process can be applied.

e) Monitor risks. This activity consists of the following tasks:

- 1) Continually monitor risks and the risk management context for changes and evaluate the risks when their state has changed.

NOTE 14 Risks that exceed thresholds are considered as high priority and are continually monitored to determine if future risk treatment is necessary.

- 2) Implement and monitor measures to evaluate the effectiveness of risk treatments.
- 3) Continually monitor for the emergence of new risks and sources throughout the life cycle.

6.3.5 Configuration management process

6.3.5.1 Purpose

The purpose of the configuration management process is to manage system and system element configurations over the life cycle.

Managing includes establishing and maintaining consistency, integrity, traceability, and control. Configurations include products and their product configuration information.

Software configuration management (SCM) applies to the software system, its services, and its interfaces. Software composition analysis is used to determine the source of software, such as open source elements, that can introduce risk into the software system.

SCM manages complex dependencies of systems, applications, and services, utilizing an SBOM or a set of cascading SBOMs (SBOM of SBOMs). The SBOM maintains traceability through the entire lifecycle of the system and software development effort, including maintaining secure baselines from the commit of reviewed source code through the build, package, and deployment of verified baselines. Use of an SBOM with traceable source items helps improve the security of a software system. Specifications for SBOMs with standard formats can help improve interchange of software configuration information and software interoperability.

Information items are tracked through configuration management when considered as an element of the software system. Management and production of deliverable information items is handled using the information management process (6.3.6).

The incremental and iterative nature of agile development can facilitate efficient technical and management processes and practices to reduce the cost associated with change. In comparison, projects managed in a sequential or predictive life cycle model seek to reduce total rework cost by minimizing the number of changes, limiting the number of control points, and baselining detailed specifications which are reviewed and traced throughout the project. Automation should be used for repeatable and auditable CM processing to reduce human errors.

Specification of baselines differs in degree and timing for agile development projects compared to more traditional development efforts. The baseline of requirements may initially comprise high-level user stories (“epics”) and key performance measurements, including standards for usability. Agile projects take full advantage of the task to “Define baselines through the life cycle” [6.3.5.3 b) 3)]. During development, new baselines are agreed upon and built at least daily under configuration control. A software element is generally traceable to a high-level functional requirement and closely traceable to the use case it implements and the test case used to verify its functionality and performance. Rather than being traceable to a previously approved, baselined design document, a new software element may simply be traced to a design element or object that was, in fact, created during the construction of the software and only then placed under configuration control.

NOTE The SWEBOK v4 discusses various software configuration management activities in its Software Configuration Management knowledge area, including planning and managing the SCM process, software configuration identification, software configuration change control, software configuration status accounting, and software configuration auditing.

6.3.5.2 Outcomes

As a result of the successful performance of the configuration management process:

- a) System and system element configurations are managed.
- b) Configuration baselines, including approved configurations, are maintained.
- c) Changes to items under configuration management are controlled.
- d) Configuration status information is available.
- e) Required configuration audits are completed.
- f) System releases are approved.

6.3.5.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the configuration management process.

NOTE 1 The ISO/IEC 19770 series has procedures and requirements for an IT asset management system.

- a) Plan configuration management. This activity consists of the following tasks:
 - 1) Define a configuration management strategy, including approaches for the following:
 - i) governance of CM, including roles, responsibilities, accountabilities, and authorities, and use of configuration control (change control) boards;
 - ii) consideration of the level of risk and impact in approval of configuration baselines and regular and emergency change requests;

NOTE 2 Regularly scheduled changes are typically performed automatically, or reviewed and approved daily as a matter of routine, for example, to commit a software build, to apply software patches using approved procedures, to check-in and check-out unit-tested software elements under development. In comparison, significant changes to the software system design with major impact on project cost and schedule can involve extensive analyses, consultations with suppliers, stakeholder reviews, and approvals at the highest levels of the organization.

- iii) coordination of CM across the set of acquirer, supplier, and supply chain organizations for the life of the software system, or the extent of the agreement or project, as appropriate;
 - iv) control of access and changes to and disposition of configuration items;
 - v) the necessary baselines to be established, including criteria or events for commencing configuration control and maintaining baselines of evolving configurations;
 - vi) control of software licenses, data rights and other intellectual property assets;
 - vii) determining and communicating frequency, priorities and content of software versions and releases;
 - viii) the audit strategy and the responsibilities for validating continuous integrity and security of the configuration definition information;
 - ix) change management, including preparing stakeholders and especially users for changes in operational software systems and services.

NOTE 3 For complex or secured software systems, trade-off studies are performed, for example, to select appropriate automated tools to support SCM needs and scope as identified in the strategy.

NOTE 4 Additional guidance regarding configuration management activities can be found in ISO 10007, IEEE Std 828, and SAE ANSI/EIA-649-C.

NOTE 5 The SCM strategy is commonly documented in a plan, e.g. a configuration management plan, or sometimes in a project's SEMP, SDP, or project management plan (PMP). The strategy planning for configuration management is coordinated through the project planning process. In establishing points to establish baselines and conduct audits, CM planning is aligned with the software life cycle. The frequency of recurring SCM activities aligns with the iteration of technical processes and stages. SCM planning typically includes deciding when to review configuration management planning, what conditions require updating the CM plan, and who is authorized to change the CM plans and items held in configuration control.

- 2) Define the storage, archive and retrieval procedures for configuration items, CM artefacts, and records.

NOTE 6 The locations and conditions of storage for software system configuration items, such as source code and executable software for each software release, are established in accordance with designated levels of integrity, security, and safety. Archiving includes the ability to verify the integrity of the software product.

- b) Perform configuration identification. This activity consists of the following tasks:

- 1) Select the software system elements to be uniquely identified as configuration items subject to configuration control.

EXAMPLE Configuration items subject to configuration control in software systems usually include system requirements specifications, interface specifications; product and system elements (e.g. software objects, hardware, and services) while under development, baseline configurations or software versions established for transition between stages and as released for operational use; primary ("gold") copies of source code or executable software for different platforms or versions; site-specific configurations in operational use; and information items, such as agreements, architecture models, service descriptions and operational procedures; and items in enabling systems.

NOTE 7 Unique identifications can be applied to software items, versions, or to individually licensed copies. The identifiers are in accordance with relevant standards and product sector conventions, such that the items under configuration control are unambiguously traceable to their supplier and to their specifications or equivalent recorded descriptions. Information items are often identified and managed separately from other configuration items.

NOTE 8 Software configuration identifiers facilitate traceability when more than one developer or maintenance programme is working on the same software function, so that various branches of code can be successfully reassembled and tested.

NOTE 9 The ISO/IEC 19770 series provides an IT asset management system for tracking software licenses.

- 2) Identify the attributes of configuration items.

NOTE 10 Attributes refers to item status, or physical or logical features useful for managing or maintaining the software system. Appropriate attributes can differ for hardware and software configuration items.

NOTE 11 Configuration attributes and identifiers can reflect a decomposition of the software system, so that configuration items are tracked at the level at which change needs to be controlled.

EXAMPLE Software from external suppliers can be tracked by its license and maintenance agreement, which can involve tracking to the location, number, or size of systems where it is used or the number of concurrent users allowed. Software versions can be traced to the stakeholder requirements which they implement.

- 3) Define baselines through the life cycle.

NOTE 12 Baselines (SBOMs) capture the evolving configuration states of software system elements at designated times or under defined circumstances. The content for the baselines is developed through the technical processes, but is formalized at a point in time through the CM process. Baselines form the basis for subsequent changes. Selected baselines typically become formalized between acquirer and supplier, depending on the practices of the industry and the contractual involvement of the acquirer in the configuration management process. There are generally three major types of baselines at the system level: functional baseline, allocated baseline, and product baseline. These vary by domain or local strategy.

NOTE 13 Software systems development often entails establishing multiple developmental baselines to address evolving software configuration needs at key points in the life cycle, for example, to allow for simultaneous control of software versions during software design, prototype, integration, and test releases. Version control can involve distributed configuration management responsibilities and access limitations for archives, for example, software development or test libraries and a main configuration support library.

- 4) Obtain acquirer and supplier agreement to establish a baseline with consistent naming conventions.

NOTE 14 The project assessment and control process is used to reach agreement. When software is being developed for commercial or internal use, the acquirer or project sponsor can be the authority to approve a baseline.

- c) Perform configuration change management. This activity consists of the following tasks:

NOTE 15 Configuration change management establishes procedures and methods for managing change to a baseline once it is established. It is often referred to as configuration control. The term 'change management' is also used for managing change to organizational procedures and business workflow during the transition process.

- 1) Identify and record requests for change, release requests, and requests for variance.

NOTE 16 A request for variance is often referred to as a deviation, waiver, or concession. A request for change is also called a change request.

NOTE 17 Software releases are performed using the transition process.

EXAMPLE Releases for a software test, for software or system qualification or other formal tests, or for trial (beta) or operational use.

- 2) Coordinate, evaluate, and disposition requests for change, release requests, and requests for variance.

NOTE 18 Evaluation commonly includes analysis of rationale and need versus impact on the software and interoperating systems, considering risks and opportunities, quality, users, schedule, and cost. Risk-based evaluations consider the risk of not making a change compared to the risk of making a change. A decision is made on whether to implement or deny the change request.

NOTE 19 Requests for change, software releases, and requests for variance are often under the formal control of a configuration control board (CCB).

NOTE 20 Approval of a release for operational use can include acceptance of the verified and validated changes. Criteria for approval of a release often includes rollback plans or contingency plans in the event of an unsuccessful release.

- 3) Track and manage approved changes to the baseline, software releases, requests for change and requests for variance.

NOTE 21 This task involves prioritization, tracking, scheduling, and closing changes. Changes are then made through the technical processes, especially transition. These changes are verified or validated through the verification and validation processes, to help ensure that the approved changes have been correctly applied.

NOTE 22 Source versions or copies of incremental changes to released software versions can be maintained for the life of the system or project in a controlled environment. Software suppliers often track delivered copies of licensed software to the acquirer in order to provide agreed software maintenance.

NOTE 23 Changes and rationales are typically recorded when approved and when completed. A PCA can determine if required information items are included in the SBOM or configuration. System models and system elements are verified for consistency. Consistency between the content of information items and software elements is performed using the information management process.

- d) Perform configuration status accounting. This activity consists of the following tasks:

- 1) Develop and maintain the CM status information for software system elements, baselines, and releases.

NOTE 24 Configuration status accounting provides the data on the status of controlled products needed to make decisions regarding system elements throughout the product life cycle. Information about the configuration or SBOM can include dependencies on other software components, associated risks, vulnerabilities, or test data. Software status can include known issues; past, current, and planned progression through stages in the life cycle for software functions, and completion of verification and validation activities for software elements. Configuration status information permits forward and backward traceability to other configuration states. Configuration status records are maintained through the software or project life cycle and then archived according to agreements, relevant legislation, or organizational practice.

NOTE 25 The recording, retrieval and consolidation of the current configuration status and the status of previous configurations to confirm information correctness, timeliness, integrity, and security is managed. Audits are performed to verify conformance of a baseline to an architecture view, interface control documents, software license agreements, and other agreement requirements.

- 2) Capture, store, and report configuration management data.
- e) Perform configuration evaluation. This activity consists of the following tasks:
- 1) Verify the product configuration meets the configuration requirements by comparing requirements, constraints, and waivers (variances) with results of formal verification activities, which can involve sampling methods.
 - 2) Monitor the incorporation of approved configuration changes.
 - 3) Assess whether the software system meets functional and performance capabilities identified for the baseline.

NOTE 26 This assessment is sometimes called a functional configuration audit (FCA), which assures that the product configuration meets specified requirements.

- 4) Assess whether the operational software system elements conform to the approved configuration information.

NOTE 27 This assessment is sometimes called a physical configuration audit (PCA). For software items, criteria for the PCA can include whether specified configuration items are installed on designated systems or available only to designated users, according to software license or agreement. Baselines can be verified and stored using cryptographic hashes.

NOTE 28 PCA can determine if required information items are included in the SBOM or configuration. System models and system elements are verified for consistency. Consistency between the content of information items and software elements is the responsibility of Information Management.

NOTE 29 PCA can include monitoring to detect any unplanned unauthorized changes to production baselines.

- 5) Record the CM audit results and disposition action items.

6.3.6 Information management process

6.3.6.1 Purpose

The purpose of the information management process is to generate, obtain, confirm, transform, retain, retrieve, disseminate, and dispose of information for designated stakeholders.

Information management plans, executes, and controls the provision of information to designated stakeholders that is unambiguous, complete, verifiable, consistent, modifiable, traceable, and presentable. Information includes technical, project, organizational, agreement, and user information. Information is often derived from data records of the organization, system, process, or project.

The preparation of specifications, design artefacts, and information items or documentation during agile projects is often scoped to what is needed to begin work on a scenario or narrative of a function ("user story"). Meanwhile, software developers apply their time and skills to produce a working, testable software element. Rather than preparing elaborate review packages for briefing at infrequent major milestone

reviews, the team meets with stakeholders frequently to present evidence of completing a set of functions and to agree on the content of the next iteration. Documented information items focus on what is needed for transition, operation, and maintenance, such as information for users and baselines of tested and released versions of software with test plans and test cases. Projects reuse organizational procedures for configuration and release management, verification, and incident and problem management. Where possible, bidirectional traceability is enabled and enforced by integrated automated systems and procedures for requirements management, architecture and design, configuration management, measurement, and information management.

NOTE Managed information has these quality characteristics: unambiguous, complete, verifiable, consistent, modifiable, traceable, and presentable. Information modification capabilities include protection from tampering and unauthorized access.

6.3.6.2 Outcomes

As a result of the successful performance of the information management process:

- a) Information to be managed is identified.
- b) Information representations are defined.
- c) Information is managed.
- d) The status of information is identified.
- e) Information is available to designated stakeholders.

6.3.6.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the information management process.

NOTE 1 ISO/IEC/IEEE 15289 summarizes requirements for the content of life cycle process information items (documentation) and provides guidance on their development.

- a) Prepare for information management. This activity consists of the following tasks:

- 1) Define the strategy for information management.

NOTE 1 Information about the same topic can be developed in different ways at different points in the life cycle and for different audiences.

NOTE 2 If artificial intelligence (AI) is designated to collect or retrieve information, the risks of biased, unauthorized, or non-factual data are evaluated. For example, IEEE 7005 states that in situations where AI is used, human agency governs the final decision.

- 2) Define the items of information to be managed.

NOTE 3 This baseline includes the information that is managed during the software life cycle and possibly maintained for an extended period, according to organizational policy, agreements, or legislation.

- 3) Designate authorities and responsibilities for information management.

NOTE 4 Due regard is paid to information and data legislation, security and privacy, for example, ownership, agreement restrictions, rights of access to data and ownership of data, intellectual property, and patents. Where restrictions or constraints apply, information is identified accordingly. Staff members with knowledge of such items of information are informed of their obligations and responsibilities.

- 4) Define the content, formats, and structure of information items.

NOTE 5 The information originates and terminates in many forms (e.g. audiovisual, textual, graphical, numerical) and mediums (e.g. electronic, printed, magnetic, and optical). Organization constraints, for example, infrastructure, inter-organizational communications, and distributed project workings, are taken into account. Relevant information item standards and conventions are used according to policy, agreements, and legislative constraints.

5) Define information maintenance activities.

NOTE 6 Information maintenance includes status reviews of stored information for integrity, validity, and availability. It also includes any needs for replication or transformation to an alternative medium, as necessary, either to retain infrastructure as technology changes so that archived media can be read or to migrate archived media to newer technology.

b) Perform information management. This activity consists of the following tasks:

1) Obtain, develop, or transform the identified items of information.

NOTE 7 Information development includes collecting the data, information, or information items from appropriate sources and resulting from any life cycle process and writing, illustrating, or transforming it into usable information for stakeholders. It includes reviewing, validating, and editing information per information standards.

2) Maintain information items and their storage records, and record the status of information.

NOTE 8 Information items are maintained according to their integrity, security, and privacy requirements. The status of information items is maintained (e.g. version description, date of issue or validity date, record of distribution, security classification). Legible information is stored and retained in such a way that it is readily retrievable.

NOTE 9 The source data and tools used to transform information, along with the resulting documentation is placed under configuration control in accordance with the configuration management process. ISO/IEC/IEEE 26531 provides requirements for content management systems useful for life cycle information and documentation.

3) Publish, distribute, or provide access to information and information items to designated stakeholders.

NOTE 10 Information is provided to designated stakeholders in an appropriate form, as required by agreed schedules or defined circumstances. Information items include documentation used for certification, accreditation, licensing, or assessment ratings.

4) Archive designated information.

NOTE 11 Archiving is done in accordance with the audit, knowledge retention, and project closure purposes. The media, location and protection of the information are selected in accordance with the specified storage and retrieval periods, and with organization policy, agreements, and legislation. Arrangements are put in place to retain necessary information items after project closure.

5) Dispose of unwanted, invalid or unvalidated information.

NOTE 12 Information disposal is done according to organization policy, and security and privacy requirements. The disposal process can be applied.

6.3.7 Measurement process

6.3.7.1 Purpose

The purpose of the measurement process is to collect, analyse, and report objective data and information to support effective management and address information needs of the products, services, and processes.

Measurements can relate to projects, processes, products, services, or decisions.

NOTE The SWEBOK v4 discusses measurement commitments, planning and performing the measurement process, and evaluating measurement in its Software Engineering Management knowledge area.

6.3.7.2 Outcomes

As a result of the successful performance of the measurement process:

- a) Information needs are identified.
- b) An appropriate set of measures, based on the information needs, is identified or developed.
- c) Required data is managed.
- d) The data is analysed and the results interpreted.
- e) Measurement results provide objective information that supports decisions.

6.3.7.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the measurement process.

NOTE 1 ISO/IEC/IEEE 15939 provides a more detailed set of measurement activities and tasks that are aligned with the activities and tasks in this document.

NOTE 2 ISO 9001:2015, 7.1.5 specifies quality management system requirements for measurement and monitoring.

- a) Prepare for measurement. This activity consists of the following tasks:
 - 1) Define the measurement strategy.
 - 2) Describe the characteristics of the organization that are relevant to measurement, such as business and technical objectives.
 - 3) Identify and prioritize the information needs.

NOTE 3 The information needs are based on the organization's business objectives, the project objectives, identified risks, and other items related to project decisions.
 - 4) Select and specify measures that satisfy the information needs.

NOTE 4 Measures have these quality characteristics: verifiable, meaningful, actionable, timely and cost-effective.
 - 5) Define data collection, analysis, access and reporting procedures.
 - 6) Define criteria for evaluating the information items and the measurement process.
 - 7) Identify and plan for the necessary enabling systems or services to be used.
- b) Perform measurement. This activity consists of the following tasks:
 - 1) Integrate manual or automated procedures for data generation, collection, analysis, and reporting into the relevant processes.

NOTE 5 Measurement can be hampered by several problems, for example, inaccurate or imprecise measurements, high levels of deviation in measurements, measures that are not aligned with information needs, and misreporting or misinterpretation of measurements. Remedies for measurement problems can include selection of a different measure, refining the definition of the measure, changing the timing or procedures for measurement, or training for measurement staff.

NOTE 6 This task can involve change impacts to other life cycle processes to accomplish procedural integration.
 - 2) Collect, store, and verify data.
 - 3) Analyse data and develop information items.

- 4) Record results and inform the measurement users.

NOTE 7 The measurement analyses results are reported to relevant stakeholders in a timely, usable fashion to support decision making and assist in corrective, preventive, adaptive, or perfective actions, risk management, and improvements. Results are reported to decision process participants, technical and management review participants, and product and process improvement process owners.

6.3.8 Quality assurance process

6.3.8.1 Purpose

The purpose of the quality assurance (QA) process is to help ensure the effective application of the organization's quality management (QM) process to the project.

QA focuses on providing confidence that quality requirements are fulfilled. Proactive analysis of the project life cycle processes and outputs is performed to help ensure that the product being produced or the service being developed is of the desired quality and that organization and project policies and procedures are followed.

NOTE [Annex E](#) presents the use of assurance claims in system assurance. Establishing an assurance case can guide QA activities and help ensure critical quality characteristics are considered.

6.3.8.2 Outcomes

As a result of the successful performance of the quality assurance process:

- a) QA procedures, including criteria and methods for QA evaluations, are implemented.
- b) Evaluations of products, services, and processes are performed, consistent with quality management policies, procedures, and requirements.
- c) Results of evaluations are provided to relevant stakeholders.
- d) Incidents are resolved.
- e) Prioritized problems are treated.

NOTE Outcomes a) through d) align with the outcomes of the quality management process activities and tasks.

6.3.8.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the QA process.

NOTE 1 IEEE Std 730 provides additional detail. Numerous ISO/IEC standards and guidelines on systems and software quality requirements and evaluation (SQuaRE) also define software quality characteristics.

- a) Prepare for quality assurance. This activity consists of the following tasks:
 - 1) Define a QA strategy. The strategy is consistent with the organizational QM policies and objectives and includes the following:
 - i) priorities for applying QA resources to processes and tasks that have the most significant impact on the quality of the delivered products and services;
 - ii) defined roles, responsibilities, accountabilities and authorities;
 - iii) evaluation criteria and methods for processes, products, and services, including criteria for product or service acceptance;
 - iv) activities appropriate to each supplier (including subcontractors);

- v) required verification, validation, monitoring, measurement, review, inspection, audit, and test activities specific to the products or services;
- vi) problem resolution and process and product improvement activities.

NOTE 2 In software projects, activities and tasks that have significant impact on product quality include obtaining agreement on new and changed requirements, performance of peer reviews and unit testing, analysis of problem reports and feedback from users; validating completion of corrective actions assigned at project milestone reviews, and root cause analysis of defects. QA tasks can consider the quality of code, data, interfaces, user documentation, and training material.

2) Establish independence of QA from other life cycle processes.

NOTE 3 Resources for QA are often assigned from distinct organizations for independence from project management.

b) Perform product or service evaluations. This activity consists of the following tasks:

- 1) Evaluate products and services for conformance to established criteria, contracts, standards, and regulations.

NOTE 4 This task is often called quality control (QC) and associated with the verification and validation processes.

NOTE 5 Derived quality requirements are usually associated with quality characteristics during requirements definition processes. ISO/IEC 25010 and ISO/IEC 25030 provide additional information on system and software quality characteristics.

- 2) Monitor that verification and validation of the outputs of the life cycle processes are performed to determine conformance to specified requirements.

c) Perform process evaluations. This activity consists of the following tasks:

- 1) Evaluate project life cycle processes for conformance.
- 2) Identify and monitor implementation of software and QA process improvements.
- 3) Evaluate tools and environments that support or automate the process for conformance.
- 4) Evaluate supplier processes for conformance to process requirements.

NOTE 6 Consider items such as a collaborative software development environment, agile and iterative processes, process measures that suppliers provide, or a risk process that suppliers use. This task includes surveillance reviews of process implementation through the supply chain.

d) Manage QA records and reports. This activity consists of the following tasks:

- 1) Create records and reports related to software quality and QA activities.

NOTE 7 Create records and reports in accordance with organizational, regulatory, and project requirements, using the information management process.

- 2) Maintain, store, and distribute records and reports.
- 3) Identify incidents and problems associated with product, service, and process evaluations.

NOTE 8 Problem resolution includes the capture of lessons learned. Responsibilities for resolution are identified.

e) Treat incidents and problems. This activity consists of the following tasks:

NOTE 9 In the terminology of quality management, problems are often described as “non-conformities” when, if left untreated, they can cause the project to fail to meet its requirements. Problems and nonconformities are often caused by defects. More precisely, nonconformities are shown by errors and represent an unfulfilled requirement: a discrepancy between required and actual performance.

NOTE 10 For additional information and examples of problem categories and priority classifications, see ISO/IEC/IEEE 24748-1:2024, Annex G.

- 1) Record, analyse and classify incidents.
- 2) Identify selected incidents to associate with known errors or problems.
- 3) Record, analyse and classify problems.

NOTE 11 Analysis results include potential treatment options.

- 4) Identify root causes and corrective and preventive treatment of problems where feasible.
- 5) Prioritize treatment of problems (problem resolution) and track corrective actions.

NOTE 12 Implementation is done in the technical processes after initiation by the project assessment and control process. Organizational procedures for problem escalation can help focus resources on lagging problem resolutions.

- 6) Analyse trends in incidents and problems.
- 7) Identify improvements in processes and products that may prevent future incidents and problems.

NOTE 13 The risk management process is used to treat risks and opportunities. The life cycle model management process is used to improve the organization's processes.

- 8) Inform designated stakeholders of the status of incidents and problems.
- 9) Track incidents and problems to closure.
- 10) Evaluate adequacy of incident and problem resolutions,

6.4 Technical processes

The technical processes are used to define the requirements for a software system, to transform the requirements into an effective product, to permit consistent reproduction of the product where necessary, to use the product to provide the required services, to sustain the provision of those services, and to dispose of the product when it is retired from service.

The technical processes define the activities that enable organization and project functions to optimize the benefits and reduce the risks that arise from technical decisions and actions. These activities enable software systems and services to possess the timeliness and availability, cost effectiveness, functionality, reliability, maintainability, producibility, usability, and other qualities required by acquiring and supplying organizations. They also enable products and services to conform to the expectations or legislated requirements of society, including health, safety, security, and environmental factors.

The technical processes consist of the following:

- a) business or mission analysis process;
- b) stakeholder needs and requirements definition process;
- c) system requirements definition process;
- d) system architecture definition process;
- e) design definition process;
- f) system analysis process;
- g) implementation process;
- h) integration process;
- i) verification process;

- j) transition process;
- k) validation process;
- l) operation process;
- m) maintenance process;
- n) disposal process.

For software systems, these processes can be recursively applied at more inclusive or more detailed levels for software system definition and realization. For software systems, these processes are often performed concurrently, iterating between one another to establish a solution that has satisfactory trade-offs with respect to requirements, critical performance measures, and critical quality characteristics. At any level of abstraction, requirements and models are made consistent via iterations of applicable technical processes. When requirements and models are not directly capable of being implemented, the technical processes are applied recursively at a more detailed level or through different system views.

NOTE 1 The concept of life cycle stages and the application of these processes in any stage are described in detail in ISO/IEC/IEEE 24748-1. It has a complete set of example stages and stage outcomes for the enactment of technical processes within a software life cycle.

NOTE 2 Interface management is a set of activities that cut across software engineering processes. These cross-cutting activities of the technical and technical management processes apply and track as a specific view of the processes and software system. See ISO/IEC/IEEE 24774 for an example of an interface management process view.

NOTE 3 ISO/IEC 27002 and ISO/IEC 27034-3 provide guidance for applying security concerns in the technical processes for software systems.

6.4.1 Business or mission analysis process

6.4.1.1 Purpose

The purpose of the business or mission analysis process is to define the overall strategic problem or opportunity, characterize the solution space, and determine potential solution class(es) that can address a problem or take advantage of an opportunity.

The business or mission analysis process analyses changes in the organizational concept of operations, environment, and other strategic inputs to identify and define key problems or opportunities that should be addressed to achieve the organization's mission, vision, goals, or objectives. The business or mission analysis process also identifies, characterizes, and prioritizes alternative solution classes (or general approaches) that are candidates to address the problem or opportunity.

Business or mission analysis is related to the organization encompassing stakeholders concerned by the activities of the software life cycle. This process interacts with the organization's strategy, which is generally outside the scope of this document. The results of the organization's strategic analysis include the organizational concept of operations, strategic goals and plans, new market or mission concerns, and identified problems and opportunities. The organization's strategy establishes the context within which the business or mission analysis is performed. The organizational concept of operations relates to the leadership's intended way of operating the organization. It describes the organization's assumptions and how it intends to use, acquire, or supply the system to be developed, existing systems, and possible future systems in support of an overall operation or series of operations of the business. In the case that the organization is the SoI, the organization's strategy is part of the system definition.

NOTE Mission engineering is an interdisciplinary process encompassing the entire technical effort to analyse, design, and integrate current and emerging operational needs and capabilities to achieve desired mission outcomes.

This process has application through the life of the software system solution and can be revisited if there are changes in the environment, needs, or other drivers. Projects using agile approaches benefit from an initial and iterated business or mission analysis to reduce risks and focus on priorities.

In some domains, business or mission analysis relates to the concept of identifying and analysing capabilities that are needed or desired by the organization. This process focuses on the necessary capabilities and interacts with the portfolio management process for identifying the trade space that can address the capability. The identified problems or opportunities are often translated into target capabilities. As applicable within a given domain, the problem or opportunity space includes the target capabilities.

Business or mission analysis can be performed at various scales, and is not directly aligned with the establishment of one or more projects or systems,

6.4.1.2 Outcomes

As a result of the successful performance of the business or mission analysis process:

- a) The problem or opportunity space is defined.
- b) The solution space is characterized.
- c) Preliminary operational concepts and other concepts in the life cycle stages are defined.
- d) Alternative solution classes are analysed.
- e) The preferred alternative solution class(es) are selected.
- f) Enabling systems or services needed for business or mission analysis are available.
- g) Traceability of strategic problems and opportunities and the preferred alternative solution classes is established.

6.4.1.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the business or mission analysis process.

- a) Prepare for business or mission analysis. This activity consists of the following tasks:
 - 1) Review identified problems and opportunities in the organization strategy and concept of operations to identify potential problems and opportunities with respect to desired organization missions, vision, goals, and objectives.

NOTE 1 Problems or opportunities can arise in the organization or business or mission, vision, concept of operations, and other strategic goals and objectives. Problems can include deficiencies or gaps in existing capabilities, systems, products, or services.
 - 2) Define the business or mission analysis strategy.

NOTE 2 The strategy includes the approach to be used to identify and define the problem space, characterize the solution space, and select a solution class.
 - 3) Identify and plan for the necessary enabling systems or services needed to support business or mission analysis.

NOTE 3 This task includes identification of requirements and interfaces for enabling systems. Enabling systems for business or mission analysis include the business systems and repositories of the organization or other accessible entities.
 - 4) Obtain or acquire access to the enabling systems or services to be used.

NOTE 4 The Validation process is used to objectively confirm that the enabling system achieves its intended use for its enabling functions.
- b) Define the problem or opportunity space. This activity consists of the following tasks:
 - 1) Analyse customer complaints, problems, and opportunities in the context of relevant trade-space factors.

NOTE 5 This analysis is focused on understanding the scope, basis, or drivers of the problems or opportunities, as opposed to the synthesis that is the focus of system analysis and decision management needed for trade-off studies. The focus here includes changes in mission requirements, business opportunities, capabilities, performance improvement, or lack of existing systems, security and safety improvement, factors such as cost and effectiveness, regulation changes, user dissatisfaction, and PESTEL factors (political, economic, social, technological, environmental, and legal). Relevant factors can be identified through external, internal, or SWOT (strengths, weaknesses, opportunities, and threats) analysis.

NOTE 6 The outputs of the analysis are considered as part of the portfolio management decisions.

2) Define the mission, business, or operational problem or opportunity.

NOTE 7 This definition includes the context and any key parameters, without regard to a specific solution, since the solution can be an operational change, a change to an existing product or service, or a new system.

3) Prioritize the potential problem or opportunity against other business needs.

c) Characterize the solution space. This activity consists of the following tasks:

1) Define preliminary operational concepts and other life cycle concepts.

NOTE 8 Concepts are related to major stakeholder groups, such as customers, users, administrations, regulators, and system owners, that are further defined in the stakeholder needs and requirements definition process.

NOTE 9 Preliminary life cycle concepts include preliminary acquisition concepts, preliminary deployment concepts, preliminary operational concepts, preliminary support concepts, and preliminary retirement concepts. Operational concepts include high level operational modes or states, operational scenarios, potential use cases or usage within a proposed business strategy. These concepts enable feasibility analysis and evaluation of alternatives. These concepts are further refined within the stakeholder needs and requirements definition process.

NOTE 10 The operating environment can have vulnerabilities associated with specific security threats and safety hazards. While many such vulnerabilities may be known, mechanisms can be employed to anticipate other vulnerabilities that are not currently recognized. These vulnerabilities need to be understood in association with the product under development. The system and human interfaces are key concerns for system assurance, and related vulnerabilities are examined in the context of mission-critical threats.

2) Identify alternative solution classes that span the potential solution space.

NOTE 11 These classes can range from simple operational changes to various software system developments or modifications. The solution space can include the identification of existing assets, systems, and software products suitable for reuse, and changed or additional services that can address the need for operational or functional modifications. The solution space characterization often invokes the system architecture definition process for a user architecture viewpoint, resulting in architecture views, e.g. capability views, program views and operational views, as proposed by ISO/IEC/IEEE 42010.

NOTE 12 Assumptions about solutions made during this process are generally based on limited data and are subject to change when more complete and better-understood information about stakeholder needs and requirements is obtained.

d) Evaluate alternative solution classes. This activity consists of the following tasks:

1) Assess each alternative solution class.

NOTE 13 Each alternative solution class is assessed against defined criteria that are established based on the organization's strategy. Feasibility of the solution class is one key decision criteria. The Portfolio Management process provides some criteria to be considered.

NOTE 14 The System Analysis process is used to assess the value of each criterion for each alternative solution class. Structured affordability trade-offs are recommended. Including cost as a criterion aids affordability decisions. The assessment of alternatives can include modelling, simulation, analytical techniques, or expert judgment to understand the risks, feasibility, and value of the alternative candidate solution classes.

- 2) Select the preferred alternative solution class(es).

NOTE 15 The decision management process is used to evaluate alternatives and to guide selection. Selected alternatives are validated in the context of the organization's strategy. Feedback on risks, feasibility, market factors, and alternatives is provided for use in updating the organization's strategy.

- 3) Provide feedback to strategic level life cycle concepts to reflect the selected solution classes.

- e) Manage the business or mission analysis. This activity consists of the following tasks:

- 1) Record key business or mission analysis decisions and the rationale.
- 2) Maintain traceability of business or mission analysis and the alternative solution classes

NOTE 16 Through the life cycle, bidirectional traceability is maintained between the business or mission problems and opportunities and the preferred alternative solution classes with the organizational strategy, stakeholder needs and requirements, and system analysis results supporting decisions.

- 3) Provide key artefacts and information items that have been selected for baselines.

NOTE 17 The configuration management process is used to establish and maintain configuration items and baselines. This process identifies candidates for the baseline, and the information management process controls the information items.

6.4.2 Stakeholder needs and requirements definition process

6.4.2.1 Purpose

The purpose of the stakeholder needs and requirements definition process is to define the stakeholder requirements for a system that can provide the capabilities needed by users and other stakeholders in a defined environment.

It identifies stakeholders or stakeholder classes involved with the system throughout its life cycle, and their needs. It analyses and transforms these needs into a common set of stakeholder requirements that express the intended interaction the system will have with its operational environment and that are the reference against which each resulting operational capability is validated. The stakeholder requirements are defined considering the context of the SoI, which includes interoperating systems and enabling systems. This also includes consideration of laws and regulations, environmental restrictions, and ethical values.

Using the stakeholder needs and requirements definition process, concepts, needs, and requirements are defined in the context of the defined problem or opportunity, the associated capabilities required, and the preferred solution class(es). This process includes defining the operational context of the solution. The operational concept addresses what the system should do and why.

Agile methods emphasize the stakeholder needs and requirements definition process, facilitating change through a high degree of ongoing stakeholder involvement. In agile projects, key stakeholders, such as the acquirer or user representatives, are not just approvers of information, measurements, and evaluation reports. Continuous stakeholder involvement is consistent with this document, which identifies the involvement of stakeholders in every technical process as well as in the Tailoring process ([Annex A](#)). They are closely involved in requirements management [6.4.3.3 d)] at each iteration, by bringing new requirements and changes in priorities and participating when prioritized requirements are selected from a backlog of undeveloped stories or functions and further refined for development. The iterative approach encourages flexibility to add, reprioritize, or defer requirements which are recognized as being within the general scope of the project. Also, stakeholder involvement in the approval of tested software in each iteration means that validation is continual throughout the project.

With the incremental definition of evolving requirements, the concept of project scope differs in an agile project from projects where scope is defined by a predetermined baseline of specified requirements. When an agile project requires a defined product, its scope is initially tied to high-level or fundamental requirements. More detailed levels of product definition are expected to emerge as additional knowledge is gained throughout construction.

The stakeholder requirements are the basis of the validation criteria for the software system and its elements.

NOTE 1 Stakeholder requirements express the organizational or business deliverables that provide value by fulfilling stakeholder needs. Depending on the viewpoint or source of the requirements, they can include business, customer, or user requirements. Business requirements apply to needs of commercial or non-commercial, not-for-profit entities.

NOTE 2 Further information on requirements engineering can be found in ISO/IEC/IEEE 29148. SWEBOK v4 discusses several topics related to stakeholders, requirement types and properties, requirements elicitation, requirements analysis, requirements specification, requirements validation, requirements management, requirement prioritization, requirements traceability, and tools to support software requirements.

6.4.2.2 Outcomes

As a result of the successful performance of the stakeholder needs and requirements definition process:

- a) Stakeholders of the system are identified.
- b) Required characteristics, context of use of capabilities, operational concepts, and other life cycle concepts, are defined.
- c) Constraints on a system are identified.
- d) Stakeholder needs are defined.
- e) Prioritized stakeholder needs are transformed into stakeholder requirements.
- f) Critical performance measures are defined.
- g) Stakeholder agreement that their needs and expectations are reflected adequately in the requirements is achieved.
- h) Any enabling systems or services needed for stakeholder needs and requirements are available.
- i) Traceability of stakeholder requirements to stakeholders and their needs is established.

6.4.2.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the stakeholder needs and requirements definition process.

- a) Prepare for stakeholder needs and requirements definition. This activity consists of the following tasks:

- 1) Identify the stakeholders who have an interest in the solution throughout its life cycle.

NOTE 1 Stakeholders can include individuals and classes of people who are users, operators, supporters, developers, producers, trainers, maintainers, disposers, acquirer and supplier organizations, parties responsible for external interfacing entities, regulatory bodies, and others who have a legitimate interest in the system. Where direct communication is not practicable e.g. for consumer products and services, representatives or designated proxy stakeholders are selected.

NOTE 2 Initially, a set of key stakeholders or stakeholder groups and their likely areas of interest can be identified. As analysis proceeds, additional information often leads to changing some such characterizations and identifying additional stakeholders. Most stakeholders can benefit from successful operation and use of the system. However, negative, adverse, or hostile stakeholders, whose interests and needs conflict with the system's success, e.g. competitors, hackers, terrorists, can be identified.

- 2) Define the stakeholder needs and requirements definition strategy.

NOTE 3 Even among the key stakeholder groups, there can be differences in priorities to reconcile. When the stakeholder interests oppose each other, but do not oppose the software system, the requirements definition strategy is intended to gain consensus among the stakeholder classes to establish a common set of acceptable requirements.

NOTE 4 The intents or desires of detractors, those that oppose the acquirers of the system, are addressed through the risk management process, threat analyses of the system analysis process, or the system requirements for security, adaptability, or resilience. Based on analysis of the impact of adverse or hostile stakeholders opposed to the system, constraints can be derived that can mitigate risk. In this case, those stakeholder needs are not satisfied, but rather addressed in a manner to help ensure system assurance and integrity.

- 3) Identify and plan for the necessary enabling systems or services needed to support stakeholder needs and requirements definition.

NOTE 5 This task includes identification of requirements and interfaces for the enabling systems. Enabling systems for stakeholder needs and requirements definition include tools for facilitation and requirements management.

- 4) Obtain or acquire access to the enabling systems or services to be used.

NOTE 6 The validation process is used to objectively confirm that the enabling system achieves its intended use for its enabling functions.

- b) Define stakeholder needs. This activity consists of the following tasks:

- 1) Identify stakeholder needs.

NOTE 7 Identification of stakeholder needs includes elicitation of needs directly from the stakeholders, identification of implicit stakeholder needs based on domain knowledge and context understanding, and documented gaps from previous activities. Needs often include identification of critical operational, data integrity, and usability issues and measures of effectiveness. Functional analysis is often used to aid the elicitation of needs.

NOTE 8 Quality characteristics of the quality model in ISO/IEC 25010 and quality model application to requirements analysis in ISO/IEC 25030 are useful to elicit and identify quality requirements of non-functional requirements, which are often implicit stakeholder needs. Guidance for data security controls in ISO/IEC 27002 can assist in defining requirements for data protection.

NOTE 9 Stakeholder needs describe the needs, wants, desires, expectations, and perceived constraints of identified stakeholders. Understanding stakeholder needs for security and privacy of the operational environment can reduce the potential for disruption in plans, schedules, and performance. If significant issues are likely to arise relating to users and other stakeholders and their involvement in or interaction with a software system, recommendations for identifying and treating human-system issues can be found in ISO TS 18152.

- 2) Prioritize needs and select which needs can be met.

NOTE 10 The decision management process is typically used to support prioritization and selection of needs. The system analysis process is used to analyse needs for feasibility or other factors.

- 3) Define and describe the stakeholder needs and rationale.

NOTE 11 Needs concentrate on system purpose and behaviour, and are described in the context of the operational environment and conditions. It is useful to trace needs to their sources and rationale.

- c) Develop the operational concept and other life cycle concepts. This activity consists of the following tasks:

NOTE 12 Other life cycle concepts can include acquisition concepts, deployment concepts, support concepts, security concepts, and retirement concepts. In this activity, the preliminary life cycle concepts defined within the business or mission analysis process are further developed in the context of specific stakeholder needs, as associated scenarios and interactions are defined. See ISO/IEC/IEEE 29148:2018, Clauses 5 and 6 for more information on operational concepts, and ISO/IEC/IEEE 29148:2018, Annex A for an annotated outline of a system operational concept.

- 1) Define the context of use (the operational environment) within the concept of operations, the preliminary life cycle concepts, and the preferred solution classes.

NOTE 13 Context of use is often captured using a context of use description (ISO/IEC 25063). Preliminary life cycle concepts are developed by the business or mission analysis process.

- 2) Define scenarios or use cases identifying required capabilities that correspond to operational concepts and the context of use.

NOTE 14 Scenarios are used to analyse the operation of the system in its intended environment in order to identify additional needs or requirements that have not been explicitly identified by any of the stakeholders, e.g. legal, regulatory, and social obligations. The context of use of the system is identified and analysed, including the activities that users perform to achieve system objectives; the relevant characteristics of the users, e.g. expected training and knowledge, frequency of system use, responsibilities, and accessibility concerns; the physical environment, e.g. available light and temperature; and any equipment to be used, e.g. protective or communication equipment. The social and organizational influences on users that affect system use or constrain its design are analysed when applicable. Scenarios centred on attackers, their environments, tools, techniques, and capabilities are key considerations for operational concept development. Scenarios are prioritized to reflect the weighted importance of the various operational needs.

NOTE 15 These scenarios often motivate updates to the operational or other life cycle concepts. Abuse and failure scenarios highlight the need for additional functional requirements (or more specific derived requirements) to mitigate risks that are identified in the abuse or failure scenarios.

- 3) Identify the factors affecting interactions between users and the system.

NOTE 16 Usability requirements account for human capabilities and skills limitations. ISO 9241-210 can be used to define:

- i) physical, mental, and learned capabilities;
- ii) workplace, environment, and facilities, including other equipment in the context of use;
- iii) normal, unusual, and emergency conditions;
- iv) operator and user recruitment, training, and culture.

NOTE 17 If usability is important, usability requirements are specified and implemented through the life cycle processes. Further information on human-system issues can be found in ISO TS 18152 and information on usability in ISO/IEC TR 25060.

- 4) Identify the interface boundaries across which the solution interacts with external systems.

NOTE 18 Identifying the interactions between the SoI and the interfacing, enabling, and interoperating systems is helpful for identifying interface boundaries (also see [5.2.3](#)).

- d) Transform stakeholder needs into stakeholder requirements. This activity consists of the following tasks:

- 1) Identify the constraints on a system solution.

NOTE 19 These constraints can result from 1) instances or areas of the stakeholder-defined solution; 2) implementation decisions made for other elements in the system structure; 3) required use of defined enabling, legacy, or interfacing systems, system elements, resources, and staff; or 4) stakeholder-defined affordability objectives, as well as unavoidable consequences of existing agreements, management decisions and technical decisions.

- 2) Identify the stakeholder requirements and functions that relate to critical quality characteristics, such as assurance, safety and health, security, environment, and ethics.

NOTE 20 See the ISO/IEC/IEEE 15026 series for additional information on system and software assurance. ISO/IEC/IEEE 24748-7000 details requirements elicitation for ethical values. IEEE 7007 extends its approach to the ethical design of automated systems and robots. See ISO/IEC 25030 for further information regarding quality characteristics from a quality in use perspective.

NOTE 21 The risk management process can be used to identify potential hazards and safety risks. Safety risks include those associated with methods of operations and support. For example, the IEC 61508 series provides detailed requirements.

NOTE 22 Identifying security risks facilitates the identification of additional security requirements and functions for system security, such as physical, procedural, communications, computers, programs, data, and emissions; access and damage to protected personnel, properties and data and information, compromise of sensitive information, and denial of approved access to property and information. Security functions include mitigation and containment.

- 3) Define stakeholder requirements that can provide value by accomplishing business needs.

NOTE 23 More information on stakeholder requirements can be found in ISO/IEC/IEEE 29148:2018, Clauses 8 and 9, which provide a description of and an annotated outline for a stakeholder requirements specification.

- e) Analyse stakeholder requirements. This activity consists of the following tasks:

- 1) Analyse the complete set of stakeholder requirements for consistency with business needs, life cycle concepts, scenarios, interactions, constraints, and critical quality characteristics.

NOTE 24 Analysing stakeholder requirements is to give confidence that stakeholder needs and requirements are understood accurately and that satisfying stakeholder requirements can reasonably satisfy the needs. Stakeholder requirements are analysed for characteristics of individual requirements, as well as characteristics of the set of requirements. Potential analysis characteristics include that the requirements are necessary, implementation-free, unambiguous, consistent, complete, singular, feasible, traceable, verifiable, affordable, and bounded. ISO/IEC/IEEE 29148 provides additional information on characteristics of requirements.

NOTE 25 The system analysis process is used to assess feasibility and affordability. Assessing feasibility and affordability based on stakeholder requirements involves preliminary estimates of a likely system solution, its risks, and its probable costs. The verification and validation processes are used in the review of stakeholder requirements.

- 2) Define critical performance measures and quality characteristics that enable the assessment of technical achievement.

NOTE 26 Technical and quality measures and critical performance parameters are associated with each effectiveness measure identified in the stakeholder requirements. The critical performance measures, e.g. measures of effectiveness and measures of suitability, are defined, analysed, and reviewed to help ensure stakeholder requirements are met and to help ensure identification of project cost, schedule or performance risk associated with any non-compliance. ISO/IEC/IEEE 15939 provides a process to identify, define and use appropriate measures. INCOSE TP-2003-020-01 provides information on the selection, definition, and implementation of technical performance measures. IEEE Std 982 and ISO/IEC 25023 provide relevant quality measures.

- 3) Review the analysed requirements with applicable stakeholders to confirm that their needs and expectations have been adequately captured and expressed.

NOTE 27 The stakeholder requirements are reviewed at key decision times in the life cycle to help ensure that account is taken of changes in needs.

- 4) Resolve stakeholder requirements issues.

NOTE 28 Problematic requirements violate the characteristics for individual requirements or the set of requirements as defined in ISO/IEC/IEEE 29148.

- f) Manage the stakeholder needs and requirements definition. This activity consists of the following tasks:

- 1) Obtain explicit agreement with designated stakeholders on the stakeholder requirements.

NOTE 29 This task includes confirming that stakeholder requirements meet stakeholder needs within identified constraints, are expressed correctly, are comprehensible to originators, and have not corrupted or compromised stakeholder intentions in the resolution of conflicts.

- 2) Record key stakeholder requirements decisions and the rationale.

NOTE 30 Rationale includes information about major alternatives and enablers.

3) Maintain traceability of stakeholder needs and requirements.

NOTE 31 The stakeholder requirements are recorded in a form suitable for requirements management through the life cycle. These records establish the stakeholder requirements baseline, and retain changes of need and their origin throughout the software life cycle. These records are the basis for traceability to.

NOTE 32 Through the life cycle, bidirectional traceability is maintained between the stakeholder needs and requirements and the decisions made by the Business or Mission Analysis process as well as the system requirements, and subsequent software system elements. Traceability is often maintained using a data repository.

4) Provide key artefacts and information items that have been selected for baselines.

NOTE 33 The configuration management process is used to establish and maintain configuration items and baselines. This process identifies candidates for the baseline, and the information management process controls the information items. For this process, the stakeholder needs, stakeholder requirements, and operational concept are typical information items that are baselined.

6.4.3 System requirements definition process

6.4.3.1 Purpose

The purpose of the system requirements definition process is to transform the stakeholder, user-oriented view of desired capabilities into a technical view of a solution that meets the operational needs of the user.

This process creates a set of measurable system requirements that specify, from the supplier's perspective, what characteristics, attributes, and functional and performance requirements the system is to possess, to satisfy stakeholder requirements. As far as constraints permit, the requirements should not imply any specific implementation.

Business or stakeholder requirements usually cannot be directly decomposed into system requirements; because they are qualitatively different. Using the system requirements definition process, the engineering team tasked with providing a solution transforms the stakeholder requirements into system requirements. In the application of the requirements processes, a range of analysis techniques and trade-offs are applied iteratively and recursively to transform concepts into needs and needs into requirements (e.g. mission analysis, business analysis, operational analysis, requirements analysis).

System requirements are implementable features which are ways how the system reasonably can satisfy the stakeholder requirements.

As the software system is decomposed into elements, each element, in turn, is treated as a system, function, or set of functions and this process can be used to further specify requirements. Requirements analyses and tools support traceability of requirements between the software system and its elements.

NOTE 1 ISO/IEC/IEEE 29148 provides further details on the development of concepts, needs, and requirements. It includes lower-level elaboration for the processes discussed in this document, as well as annotated outlines for documenting operational concepts, stakeholder needs and requirements, and system requirements.

NOTE 2 The SWEBOK v4 Software Requirements knowledge area discusses stakeholders, requirement types and properties, requirements elicitation, requirements analysis, requirements specification, requirements validation, requirements management, requirement prioritization, requirements traceability, and tools to support software requirements.

6.4.3.2 Outcomes

As a result of the successful performance of the system requirements definition process:

- a) The system description, including interfaces, functions, and boundaries, for a system solution is defined.
- b) System requirements (functional, performance, process, non-functional, and interface) and design constraints are defined.
- c) Critical performance measures are defined.

- d) The system requirements are analysed.
- e) Enabling systems or services needed for system requirements definition are available.
- f) Traceability of system requirements to stakeholder requirements is developed.

6.4.3.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the system requirements definition process.

- a) Prepare for system requirements definition. This activity consists of the following tasks:
 - 1) Define the functional boundary of the software system or element in terms of the behaviour and properties provided.

NOTE 1 The functional boundary definition is partly based on the context of use and operational scenarios defined in the frame of the stakeholder needs and requirements definition process. The boundary relates to the software system's stimuli (input) and its responses to users and external systems. The interactions between the software system and its operational environment in terms of interface properties and constraints, such as procedural flows, calling orders, data formats and flows, throughput, and timing, are specified or described. The expected software system behaviour is expressed in quantitative terms, at its boundary. For software, boundaries are commonly expressed in application program interfaces (API) and a graphical user interface (GUI) or interface files or services, including data formats.

- 2) Define the system requirements definition strategy.

NOTE 2 The strategy includes the approach to be used to identify, define and manage the system requirements with the selected life cycle model, e.g. evolutionary, incremental, or iterative. Many factors can influence the strategy, for example, complexity of the software system and information and functions to be managed; need for ready access and common understanding by multiple team members; degree of collaborative involvement by the acquirer or user representatives throughout the development stage; whether the project involves a new development, a modification, re-use, or integration of existing systems; and process documentation requirements including period of retention. The life cycle model influences when and how often the system requirements definition is done.

- 3) Identify and plan for the necessary enabling systems or services needed to support system requirements definition.

NOTE 3 This task includes identification of requirements and interfaces for the enabling systems. AI systems that can generate typical requirements by function and check for completeness of requirements sets are gaining popularity. Enabling systems for requirements definition include tools for facilitation and requirements management. Tools for software requirements management which are integrated with software development, test, and CM can simplify tracking and expedite software construction. Plans for enabling systems and description of modelling techniques used in support of the system requirements definition process can be incorporated into an SDP.

- 4) Obtain or acquire access to the enabling systems or services to be used.

NOTE 4 The validation process is used to objectively confirm that the enabling system achieves its intended use for its enabling functions.

- b) Define system requirements. This activity consists of the following tasks:

- 1) Define each function that the software system or element is required to perform.

NOTE 5 Software functions can be described in use cases, user stories, or scenarios, and involve the transformation of data and information to achieve user needs (stakeholder requirements). In some cases, functions are derived from analysis of critical quality characteristics, such as performance, security, or availability, e.g. system diagnosing function or highly frequent data backup function for reliability.

NOTE 6 Enabling functions that are required to support the SoI in achieving its functionality are also identified and defined concurrently with the function of the SoI.

- 2) Identify required states or modes of operation of the software system.

NOTE 7 States or modes of operation can be modelled and represented in multiple modelling techniques and perspectives to give a sufficiently complete description of the desired system or element requirements.

NOTE 8 Conditions for performance of functions often involve interoperability across functions or elements. For example, some software requirements (e.g. a performance timing limit) can be allocated across multiple software system elements, affecting treatment of the requirement in a test case or regression test.

3) Define necessary implementation constraints.

NOTE 9 For software elements, this includes the implementation decisions that are allocated from system architecture definition at higher levels in the software system, introduced by stakeholder requirements, or solution limitations. Implementation constraints include the conditions under which the system is capable of performing the function, the conditions under which the system is to commence performing that function (input) and the conditions under which the system is to cease performing that function (output).

4) Identify requirements that relate to risks, criticality of the software system, or critical quality characteristics.

NOTE 10 Non-functional requirements and critical quality characteristics in software systems commonly include those related to health, safety, security and software assurance, reliability, availability, and supportability (maintainability), and time constraints for throughput and performance. The system analysis process can be used to determine appropriate values for performance requirements, considering the anticipated cost of achieving them and their impact on operation and use of the system.

NOTE 11 Analysis and definition of safety considerations include those requirements relating to methods of operation and maintenance, environmental influences, and risk of personnel injury. It also includes expressing each safety-related function and its associated integrity, in terms of the necessary risk reduction and allocation to designated safety-related systems. Applicable standards are used concerning functional safety, e.g. the IEC 61508 series, and environmental protection, e.g. ISO 14001. Analysis includes security considerations, e.g. those related to compromise and protection of sensitive information, data, and material. The security-related risks are defined, including administrative, personnel, physical, computer, communication, network, emission, and environment factors. The ISO/IEC 27036 series provides guidance for information security requirements for the outsourcing of products and services. ISO/IEC 25030 provides guidance for external system quality factors and characteristics. IEEE Std 7009 establishes a baseline of methods and tools for the design, implementation, and use of effective fail-safe mechanisms in autonomous and semi-autonomous systems that incorporate AI features. [Annex E](#) provides a software assurance view of the life cycle processes. Refer to ISO/IEC/IEEE 15026-4 for additional system and software assurance guidance.

NOTE 12 For software systems intended for human interaction, human-factors engineering (ergonomics) specifications are considered. For systems that have usability requirements, recommendations for obtaining a desired level of usability can be found in ISO 9241-220.

5) Define system requirements and requirements attributes, including the following:

- i) data elements, data structures and formats, and database or data retention requirements;
- ii) user interfaces and user documentation (information for users) and user training;
- iii) interfaces with other systems and services;
- iv) functions and non-functional characteristics, including critical quality characteristics and cost targets;
- v) transition of operational processes and data from existing automated and manual systems, migration approach and schedule, software installation and acceptance of the product;
- vi) requirement attributes, such as rationale; priority; traceability to software system elements, test cases, and information items; methods of verification; inclusion in approved baselines; and evaluated risk.

NOTE 13 Requirements definition involves iterative and recursive steps in parallel with other life cycle processes. Depending on the life cycle model that is being employed, it is useful to compare the resources to be spent in assuring initial correctness of requirements versus the resource needed to evolve requirements based on verification and validation results.

NOTE 14 The system requirements and attributes are recorded with a level of detail and in a form suitable for requirements management through the life cycle. See ISO/IEC/IEEE 29148:2018, Clauses 5 and 6, for more information on requirements, and ISO/IEC/IEEE 29148:2018, Clauses 8 and 9, for a description of and an annotated outline for a system requirements specification and a software requirements specification.

c) Analyse system requirements. This activity consists of the following tasks:

1) Analyse the complete set of system requirements for quality characteristics.

NOTE 15 Requirements are analysed for characteristics of individual requirements, as well as characteristics of the set of requirements. Potential analysis characteristics include that the requirements are necessary, implementation-free, unambiguous, consistent, complete, singular, feasible, traceable, verifiable, affordable, and bounded. The verification process is used to determine if requirements meet the attributes and characteristics of good requirements. In some cases, the technical and economic feasibility of validating and verifying alternative formulations of requirements is evaluated. ISO/IEC/IEEE 29148 provides additional information on characteristics of requirements.

NOTE 16 The system analysis process can be used to assess feasibility, affordability, balance, and other requirements characteristics. The system analysis process is used to determine appropriate values for requirement parameters, considering the estimated cost, schedule, and technical performance of the software system.

NOTE 17 Anticipating that some requirements can be achieved incrementally or even deferred or waived, requirements can be prioritized.

2) Define critical performance measures that enable the assessment of technical achievement.

NOTE 18 Technical and quality measures and critical performance parameters associated with each effectiveness measure identified in the software system element requirements are identified. The critical performance measures (e.g. measures of performance and technical performance measures) are analysed and reviewed to help ensure system requirements are met and to help ensure identification of project cost, schedule or performance risk associated with any non-compliance. ISO/IEC/IEEE 15939 provides a process to identify, define and use appropriate measures. INCOSE TP-2003-020-01 provides information on the selection, definition, and implementation of critical performance measures. ISO/IEC 25023 provides relevant quality measures.

3) Review the analysed requirements with applicable stakeholders.

NOTE 19 The review can help confirm that the specified requirements are a necessary and sufficient response to stakeholder requirements and a necessary and sufficient input to other processes, in particular software architecture, design, and verification. The validation process is used to determine if the system requirements address the users' needs.

4) Identify and resolve issues, deficiencies, conflicts, and weaknesses within the complete set of requirements.

NOTE 20 Defective requirements include those that are not verifiable, ambiguous, violate the characteristics for individual requirements, or are inconsistent with others in the set of requirements. Resolution of issues with requirements can be iterative within certain life cycle models.

d) Manage system requirements. This activity consists of the following tasks:

NOTE 21 Maintaining system requirements includes defining, recording, and controlling the baseline, typically under formal configuration management, along with managing changes resulting from the application of other life cycle processes such as architecture or design.

1) Obtain explicit agreement on the system requirements.

NOTE 22 Reaching agreement involves confirming that system requirements are expressed correctly, comprehensible to originators and implementers, and that the resolution of conflict in the requirements is consistent with stakeholder decisions.

- 2) Record key system requirements decisions and the rationale.

NOTE 23 Rationale includes information about major alternatives and enablers.

- 3) Maintain traceability of the system requirements.

NOTE 24 Through the life cycle, bidirectional traceability is maintained between the system requirements and the stakeholder requirements, architectural entities, interface definitions, analysis results, verification methods or techniques, and allocated, decomposed, and derived requirements. Traceability allows verification that achievable stakeholder requirements are met by one or more system or element requirements, and such requirements meet or contribute to meeting at least one stakeholder requirement. Traceability is often facilitated by an appropriate data repository or integrated development and test infrastructure.

- 4) Provide key artefacts and information items that have been selected for baselines.

NOTE 25 The configuration management process is used to establish and maintain configuration items and baselines. This process identifies candidates for the baseline, and the information management process controls the information items, such as requirements specifications. For this process, the system requirements are typical artefacts that are baselined.

6.4.4 System architecture definition process

6.4.4.1 Purpose

The purpose of the system architecture definition process is to generate system architecture alternatives, select one or more alternatives that address stakeholder concerns and system requirements, and express this in consistent views and models.

The system architecture definition activities define a solution based on principles, concepts, and properties logically related to and consistent with each other. The solution architecture has features, properties, and characteristics which satisfy, as far as possible, the problem or opportunity expressed by a set of system requirements (traceable to mission, business, and stakeholder requirements) and life cycle concepts (e.g. operational, support).

This process transforms related architectures (e.g. strategic, enterprise, reference, and SoS architectures), organizational and project policies and directives, life cycle concepts and constraints, stakeholder concerns and requirements, and system requirements and constraints into the fundamental concepts and properties of the system and the governing principles for evolution of the system and its related life cycle processes.

The architecture of a software system can be understood as a set of architectural elements with their properties and relationships, chosen to achieve characteristics such as interoperability, scalability, environmental resilience, encapsulation, availability, affordability, security, robustness, execution efficiency, or mission effectiveness (fitness for use).

The system architecture definition process is applied iteratively and concurrently with the business or mission analysis, stakeholder needs and requirements definition, and system requirements definition processes to determine the preferred solution to address stakeholder concerns.

Architecture definition does not necessarily focus on each of the requirements. The design definition process, on the other hand, is driven by all the requirements that have been vetted through evaluation with the architecture and more detailed analyses of feasibility. Architecture focuses on suitability, viability, and desirability, whereas design focuses on compatibility with technologies and other design elements and on feasibility of construction and integration. An effective architecture is as design-agnostic as possible to allow for maximum flexibility in the design trade space. Even for a single-product software system, the design of the product can change over time while the architecture remains constant. An effective architecture also highlights and supports trade-offs for the design definition process and possibly other processes, such as portfolio management, project planning, system requirements definition, and verification.

Sometimes, through the system architecture definition process, some requirements are deemed to be inappropriate, unaffordable, or unsuitable. These requirements issues are resolved through iteration of the

system requirements definition process. The architecture also addresses key stakeholder concerns, since not all of these can be captured in requirements.

As the system architecture definition process unfolds, insights are gained into the relation between the requirements specified for the software system and the emergent properties and behaviours of the system that arise from the interactions and relations between the system elements.

NOTE 1 When an enterprise or SoS is the SoI, then the enterprise architecture or SoS architecture is the system architecture.

NOTE 2 System architecture definition can apply to a product line rather than a single software system. A product line architecture describes the shared properties of a group of related systems with common components and interrelationships. In product line architectures, the architecture necessarily spans several designs. The architecture serves to make the product line cohesive and to help ensure compatibility and interoperability across the product line.

NOTE 3 Refer to ISO/IEC/IEEE 42010 for more information about architecture description and the uses and nature of architecture, The SWEBOK v4 Software Architecture knowledge area discusses software architecture descriptions, the architecture process, and architecture evaluation, the Software Engineering Models and Methods knowledge area contains information on various types of models and ways to analyse those models.

6.4.4.2 Outcomes

As a result of the successful performance of the system architecture definition process:

- a) Problem space is refined with respect to key stakeholder concerns, context, and perspectives.
- b) Alignment of the architecture with applicable policies, directives, objectives, and constraints is achieved.
- c) Concepts, properties, characteristics, behaviours, functions, or constraints that are significant to architecture decisions of the system are allocated to architectural entities.
- d) Identified stakeholder concerns are addressed by the system architecture.
- e) Traceability of system architecture elements to key architecturally-relevant stakeholder and system requirements is established.
- f) Architecture views and models of the system are developed.
- g) System elements, including their interfaces with each other, are identified.
- h) Enabling systems or services needed for system architecture definition are available.

6.4.4.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the system architecture definition process.

NOTE 1 ISO/IEC/IEEE 42020 presents a detailed elaboration of architecture processes including architecture governance, architecture management, architecture conceptualization, architecture evaluation, architecture elaboration, and architecture enablement. ISO/IEC/IEEE 42030 presents a detailed architecture evaluation framework. The architecture definition process in this document focuses on architecture conceptualization, elaboration, and evaluation in activities b) through d), and architecture enablement, handled through the infrastructure management process, in activity e). The architecture governance and management processes of ISO/IEC/IEEE 42020 are handled in this document as part of the portfolio management, project planning, and project assessment and control, processes.

- a) Prepare for system architecture definition. This activity consists of the following tasks:
 - 1) Define the roadmap, approach, and strategy for system architecture definition.

NOTE 2 The roadmap includes the identification of key milestones and decisions to be informed by the system architecture effort. The roadmap includes opportunities to communicate with designated stakeholders, and the cadence of architecture evaluation activities. The roadmap shows how the architecture can evolve to an envisioned end state and often has a longer timeframe than for the current project or SoI.

NOTE 3 The approach is the way the work is accomplished, such as the selection of an architecture framework, how to engage with stakeholders, required or desirable process outputs, the measurement approach for architecture evaluation (refer to the measurement process), and how to apply existing and relevant architecture assets and models (the architecture collection). The strategy deals with the systematic plan of action for implementing the approach consistent with the roadmap.

- 2) Identify and categorize available architecture assets into an architecture collection based on their relevance, respective purpose and scope, time validity, and alignment with each other.
- 3) Formalize the architecture governance and management approach and specify governance-related roles and responsibilities, accountabilities, and authorities related to design, quality, security, and safety.
- 4) Identify and plan for the necessary enabling systems or services needed to support the System architecture definition process.

NOTE 4 Enabling systems include an architecture description framework and requirements and interfaces for its enabling systems and services. Enabling systems for system architecture definition can include tools for collaboration, modelling, analysis, and AI assistance for architecture development and evaluation. Architecture reuse repositories can include artefacts such as architecture patterns, viewpoints and model kind specifications, reference models, and domain and reference architectures.

- 5) Obtain or acquire access to the enabling systems or services to be used.

NOTE 5 The validation process is used to objectively confirm that the enabling system achieves its intended use for its enabling functions. The infrastructure management process supports reuse of enabling systems.

b) Conceptualize the system architecture. This activity consists of the following tasks:

- 1) Characterize the problem space, including stakeholder concerns and perspectives.

NOTE 6 Defining the problem space is done in conjunction with the business or mission analysis, stakeholder requirements definition, and system requirements definition processes. The problem space usually includes opportunities.

NOTE 7 Key drivers in the problem space can be identified by reviewing: a) market studies, industry projections, competitor product plans, and scientific findings; b) organizational strategies, organizational-level concept of operations, organizational policies and directives, regulatory and legal constraints, and stakeholder requirements; c) mission or business concept of operations, SoI and related system operational concept, operational environment, technology roadmaps, and system requirements; and d) other factors that impact the suitability of the software system through its life cycle.

NOTE 8 Stakeholder concerns related to architecture conceptualization can include system integrity concerns, e.g. that the software system can be compromised intentionally or unintentionally via a threat agent or cause accidents as a safety hazard. Stakeholder concerns and constraints are often associated with the system's life cycle stages, such as utilization, e.g. security, effectiveness, accuracy, usability, interoperability with existing systems, availability, and risks to data in the system; support, e.g. the supportability of the system over its projected life-span and obsolescence management; evolution of the software system and its environment, e.g. adaptability, scalability, resilience, and survivability; production and distribution; testability; and retirement, e.g. sensitive data eradication or retention. Perspectives can be used to group related concerns for framing by one or more viewpoints, e.g. operational, deployment, acquisition, and consumer perspectives.

NOTE 9 Concerns affecting software system architecture typically include data sources and performance implications for data-intensive systems, interoperability with and constraints on the use of outsourced, existing, newly developed, proprietary, commercially available, or open-source software elements, including software licensing. While software architecture is ideally design-agnostic, the feasibility of implementing the architecture in an affordable software system is a significant constraint for most systems.

- 2) Establish architecture objectives and critical success criteria.

NOTE 10 Architecture objectives can involve use of existing architecture styles and patterns, elements, principles such as replaceable elements, feasibility of implementation and integration; availability of COTS and open-source components; data sources for data-intensive systems; and performance and other quality implications.

3) Synthesize potential solutions in the solution space (trade space).

NOTE 11 Solutions account for boundary conditions, root causes, drivers, and relevant scenarios for each identified problem and opportunity.

4) Formulate candidate architectures and their properties.

NOTE 12 A candidate architecture can include concepts for the components and processes with rules for their interaction and interdependence. Candidate software architectures can include information flows, control flows, state transitions, data structures, operational rules, and timelines. This task includes relating the candidate architecture solutions to other architectures and to affected entities in the environment to help ensure consistency.

c) Elaborate the system architecture. This activity consists of the following tasks:

1) For one or more selected conceptual architectures. identify, select, develop, or modify architecture artefacts, such as viewpoints, model kinds, modelling templates, view generation methods, and metamodels to be used for generating the necessary models and views.

NOTE 13 Frameworks, viewpoints, modelling templates, view generation methods and metamodels can specify modelling formalisms, languages and notations that enable the modelling effort to be more effective in providing key insights and communicating the rationale and traceability to requirements.

NOTE 14 Architecture viewpoints that include the human as part of a system capture the human requirements to inform how the human impacts system definition (i.e. user or human views). The following are typical considerations in the identification or development of architecture viewpoints:

- i) selection, adaptation, or development of viewpoints and model kinds based on stakeholder concerns;
- ii) identification of expected users of architecture elaboration information, including relevant architecture descriptions, models, and data;
- iii) identification potential architecture framework(s) or reference architectures to be used in developing models and views.

2) Develop models and views of the architecture.

NOTE 15 Models allocate concepts, properties, characteristics, behaviours, functions, or constraints to architectural entities. The items being allocated can be physical, logical, or conceptual. Models include such items as system elements, interfaces, and interactions with external entities; activities, roles, personnel, techniques, processes, nodes, data elements, layers, protocols, and preliminary identification of functional hardware and software items.

EXAMPLE A logical view of software can represent business processes in its functions; a process view can represent the events and transformations occurring within different states of the software and can include concurrency and timing concerns; a structural view represents the different system components, which can be associated with physical or virtual system elements, an information view represents the relationships between data elements contained in and transformed by the software.

3) Define the software system context and boundaries in terms of interfaces and interactions with external entities, to help ensure consistency of the elaborated system architecture.

NOTE 16 The system architecture definition process defines interfaces to the extent needed to support essential architectural decisions and understanding. These interface definitions are then refined by the design definition process.

4) Harmonize the architecture models and views with each other.

d) Evaluate the system architecture. This activity consists of the following tasks:

NOTE 17 This activity can also be applied to the enabling systems. The architecture and its elements can be evaluated at any stage of conceptualization or elaboration. Often an initial screening of candidate architectures is conducted before a detailed evaluation, to avoid unnecessary effort in evaluating candidates that are duplicative, too costly or risky, unsuitable, or infeasible,

NOTE 18 The system analysis, decision management, risk management, verification and validation processes can be used to support this task.

- 1) Define architecture evaluation criteria based on stakeholder concerns and key requirements.

NOTE 19 The results of problem space analysis can be used as a basis for the evaluation criteria.

- 2) Analyse and assess architecture entities, their properties and architecture models against stakeholder concerns, constraints, and requirements.
- 3) Select a preferred system architecture solution.
- 4) Obtain explicit acceptance of the preferred architecture by designated stakeholders.

NOTE 20 The validation process is used to confirm that the architecture models and views reflect stakeholder requirements, that stakeholder concerns are addressed, and to help ensure that future iterations of software system architecture better address stakeholder concerns.

- e) Manage the system architecture definition results. This activity consists of the following tasks:

- 1) Allocate requirements to architectural entities.
- 2) Capture key decisions and rationale for the conceptualized and elaborated architecture.
- 3) Maintain concordance and completeness of the architectural entities and their architectural characteristics.

NOTE 21 This task includes updating the architecture based upon technology changes to respond to obsolescence, or to implementation or operational experience. The entities to be checked are not only technical. These are also, for example, legal, economical, organizational, security and operational entities that are normally part of stakeholder requirements and concerns.

NOTE 22 Throughout the life cycle, traceability is typically maintained among the architectural entities or elements (viewpoints, model kinds, models, and views), the requirements, stakeholder concerns, software system design, interface definitions, analysis results and verification procedures.

- 4) Establish the architecture baseline of the selected architecture.

NOTE 23 The configuration management process is used to establish and maintain configuration items and baselines. This process identifies candidates for the baseline. The information management process controls the information items, such as architecture descriptions, that are produced for human use.

- 5) Coordinate use of the conceptualized and elaborated architecture by intended users.

NOTE 24 Users of candidate conceptual architectures typically include those doing evaluation or elaboration of the architecture, managers of an architecture collection, architects of related systems, and other collaborators and reviewers.

6.4.5 Design definition process

6.4.5.1 Purpose

The purpose of the design definition process is to provide sufficient detailed data and information about the system and its elements to realise the solution in accordance with the system requirements and architecture.

This process transforms architecture and requirements into a design of the system that can be realised. This process results in sufficiently detailed data and information about the system and its elements to enable implementation consistent with architectural entities defined in models and views of the system

architecture, in conformance with applicable system requirements, and in alignment with design guidelines and standards adopted by the organization or project.

The design definition process provides feedback to the system architecture definition process to consolidate or confirm the allocation, partitioning, and alignment of architectural entities (e.g. strategic goals, capabilities and effects, operational activities, resource functions) to system elements that comprise the system. The effect of choosing various design elements can be lessened if the software system is properly architected.

NOTE 1 An architecture entity (an entity being architected or one subject to the system architecture definition process) and a system element (a discrete part of a system that fulfils specified requirements) represent two different notions.

For software systems, design activities typically iterate with activities in system requirements definition and system architecture definition. Design definition is typically applied iteratively and incrementally to develop a detailed design, including software elements, interfaces, databases, and user documentation. Software design is usually concurrent with software implementation, integration, verification, and validation. During design and implementation, further process application refines allocation of evolving requirements among software elements.

The design definition process is driven by requirements that have been vetted through the architecture and more detailed analyses of feasibility. Architecture focuses on suitability, viability, and desirability, whereas design focuses on compatibility with technologies and other design elements and feasibility of implementation and integration. An effective architecture is as design-agnostic as possible to allow for maximum flexibility in the design trade space.

NOTE 2 The SWEBOK v4 Software Design knowledge area addresses high-level and detailed design, design qualities, recording designs and design rationales, design strategies and methods, and the analysis and evaluation of software designs.

6.4.5.2 Outcomes

As a result of the successful performance of the design definition process:

- a) Design alternatives for system elements are assessed.
- b) System requirements are allocated to the system design or its elements.
- c) Interfaces between system design elements comprising the system are defined.
- d) Design characteristics of each system elements are defined.
- e) Enabling systems or services needed for design definition are available.
- f) Design enablers necessary for design definition efforts are defined.
- g) System design is evaluated.
- h) Traceability of the design is established.

NOTE Design definition considers applicable technologies and their contribution to the system solution. Software design is often not fully specified in a formal description, but it is sufficiently detailed to permit software implementation (construction) and test specification in test-based development. Design provides the 'implement-to' level of the definition, such as scenarios, stories, state diagrams, flowcharts, and high-level design descriptions. For software elements, this process can result in a detailed design description that can be verified against requirements and the software architecture.

6.4.5.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the design definition process.

a) Prepare for design definition. This activity consists of the following tasks:

- 1) Define the design definition strategy, consistent with the selected life cycle model and anticipated design artefacts.

NOTE 1 The software design strategy can include initial or incremental decomposition into system elements; creation of various views of automated procedures, data structures and control systems; selection of design patterns, or progressively more detailed definition of objects and their relationships.

- 2) Select and prioritize principles and characteristics for the software system design and its evolution,

NOTE 2 Design principles include controlling ideas such as abstraction, modularization and encapsulation, separation of interface and implementation, concurrency persistence of data, interoperability, use of selected design patterns, ease of replacing and upgrading system elements, or security levels. Security considerations include the principle of least privilege, layered defences, restricted access to system services, and other considerations to minimize and defend the system attack surface. Design characteristics include, for example, availability, fault tolerance and resilience, scalability, usability, capacity and performance, testability, portability, and affordability.

NOTE 3 ISO 9241-210 provides human centred design/ergonomic design guidelines. ISO/IEC/IEEE 24748-7000 includes processes for prioritizing ethical values and developing an ethically aligned design.

- 3) Define and prepare or obtain the necessary design enablers.

NOTE 4 Design enablers include models, equations, algorithms, calculations, formal expressions and values of parameters, patterns, and heuristics, which are associated with design characteristics using adequate representation such as drawings, logical diagrams, flowcharts, coding conventions, logic patterns, information models, business rules, user profiles, scenarios, use cases or user stories, and tables of metrics and their values, e.g. function points or user story points.

- 4) Identify and plan for the necessary enabling systems or services needed to support design definition.

NOTE 5 This task includes identification of requirements and interfaces for the enabling systems. Enabling systems for design definition include selection of software and system platforms, programming languages, design notations and tools for collaboration and design development, design reuse repositories (for product lines, design patterns, and design artefacts), and design standards. ISO/IEC 26550 describes establishing a domain design for a product line.

- 5) Obtain or acquire access to the enabling systems or services.

NOTE 6 The validation process is used to objectively confirm that the enabling system achieves its intended use for its enabling functions.

b) Create the system design. This activity consists of the following tasks:

- 1) Transform architectural and design characteristics into the design of software system elements.

NOTE 7 Characteristics apply to physical and logical system elements, such as database structures, provisions for memory and storage, software processes and controls, external interfaces such as user interfaces, or services.

- 2) Identify software system elements that relate to architectural entities and design characteristics.

NOTE 8 Sometimes the software system elements are initially notional until design definition has occurred, since this depends on the actual design(s) to be done. Sometimes a “reference architecture” is created using these notional system elements to convey architectural intent and to check for design feasibility.

- 3) Refine or define the interfaces among the software system elements and with external entities.

NOTE 9 Interfaces are identified and defined in the system architecture definition process (see 6.4.4) to the level or extent needed for the architecture intent and understanding. These are refined in the Design Definition process based on the design characteristics, interfaces, and interactions of software elements with other elements composing the software system and with external entities. Additional interfaces are sometimes identified and defined that were not addressed in the system architecture definition.

- c) Assess alternatives for obtaining software system elements. This activity consists of the following tasks:

- 1) Determine technologies required for each element composing the software system.

NOTE 10 Several technologies are sometimes used for a given software system element, e.g. internet presence, embedded systems, adaptation of open-source software, human operator roles.

- 2) Identify candidate alternative for the software system elements.

NOTE 11 Alternatives include newly designed and constructed items; reuse adaptations of existing product lines, components, objects, or services; low-code or no-code tools, or acquisition or reuse of non-developed items (NDI). NDI include COTS (commercial-off-the-shelf), FOSS (free and open source software) packages or elements, and tools for no-code and low-code work.

NOTE 12 A make-or-buy decision and resulting implementation and integration approach typically involve trade-offs of the design criteria, including cost and feasibility of implementation. Design choices commonly consider enabling systems required to test the candidate alternative (test-driven design and development) and sustainability over the system life, including maintenance costs. The maintenance process can be used to determine the suitability of the design for long-term maintenance and sustainability.

- d) Evaluate the system design. This activity consists of the following tasks.

NOTE 13 The system analysis process can be used for analyses and assessments to support the decision management process in performing the selection. Design reviews are conducted using the Validation process.

NOTE 14 Assess the feasibility of realizing design characteristics. If warranted by assessment results, examine other alternative design options, or perform trade-offs in the architecture or requirements when design characteristics are impractical to implement.

NOTE 15 This task facilitates providing feedback to the system architecture definition process for potential modifications, for example, to modify the allocation of software system elements, possibly to modify the expected architectural characteristic due to factors discovered during the design process, or to make stakeholders aware of the potential impacts.

NOTE 16 Design suitability includes any associated risks with respect to its suitability for the intended application, consideration of ease of implementation and integration, verification and validation, usability in operation, supportability and ease of maintenance, and eventual system disposal. The existence of technical debt in the system negatively impacts design suitability.

- 1) Assess each candidate alternative against criteria developed from expected design characteristics and element requirements to determine suitability for the intended application.
- 2) Assess each system and element design alternative for how well it meets the stakeholder requirements and system requirements.
- 3) Choose the preferred alternatives among candidate design solutions for the software system elements.

- e) Manage the results of design definition. This activity consists of the following tasks:

- 1) Establish the design artefacts and rationale.

NOTE 17 Commonly captured information includes the software system elements and affiliated requirements and design data, e.g. for software elements, internal and external interfaces, data structures, implementation and test requirements, unit aggregation data for integration, and test cases. Rationale typically includes information about major implementation options and enablers.

EXAMPLES prototypes, data models, pseudocode, entity-relationship diagrams, use cases, user role and privilege matrixes, interface specifications, service descriptions, and procedures. Design artefacts are developed, obtained, or modified for selected alternatives. The data can be associated with detailed acceptable margins for implementation.

- 2) Establish traceability between the detailed design elements, the system requirements, and the architectural entities of the software system architecture.

NOTE 18 Through the life cycle, bidirectional traceability is maintained between the design and the verification methods or techniques, and software system element requirements. Allocations and design properties are assigned to software elements, software units and affiliated artefacts, at a detailed enough level to permit software testing and implementation, including construction.

- 3) Determine the status of the software system and element design.

NOTE 19 The measurement process is used to establish measures for the completeness and quality of the design as it progresses. The verification and validation processes are invoked to verify and validate the detailed design and implementation.

NOTE 20 Periodic assessment of the design characteristics includes the evolution of the software system and of its architecture, as well as forecasting potential obsolescence of elements and technologies, their replacement by others over time in the life cycle of the software system, and the consequences for the design definition. Deferred issues, commonly called technical debt, can signal needed design changes. The risk management process is typically applied to evaluate risks in the design strategy, initial design, and the evolving design.

- 4) Provide key artefacts and information items that have been selected for baselines.

NOTE 21 The configuration management process is used to establish and maintain configuration items and baselines for artefacts such as design models. This process identifies candidates for the baseline, and the information management process controls the information items, such as design descriptions and specifications.

6.4.6 System analysis process

6.4.6.1 Purpose

The purpose of the system analysis process is to provide a rigorous basis of data and information for technical understanding to aid decision-making across the life cycle.

System analysis covers a wide range of differing analytic functions, levels of complexity, and levels of rigor. It is used to provide input for diverse technical assessments and analytical needs concerning operational concepts, determination of requirement values, resolution of requirements conflicts, assessment of alternative architectures or system elements, performance and risk analyses, and evaluation of engineering strategies (integration, verification, validation, and maintenance). Formality and rigor of the analysis depends on the criticality of the information needed or artefact supported, the amount of information or data available, the size of the project, and the schedule for the results.

The system analysis process can be employed for the entire software system or any element. This process is often used in conjunction with the decision management, project assessment and control, and risk management processes. Typical approaches include mathematical analysis, modelling, simulation, experimentation, and other techniques to analyse technical performance, system behaviour, feasibility, affordability, critical quality characteristics, SoS considerations, technical risks, life cycle costs, and to perform sensitivity analysis of the potential range of values for parameters across all life cycle stages.

6.4.6.2 Outcomes

As a result of the successful performance of the system analysis process:

- a) System analyses needed are identified.
- b) System analysis assumptions and results are validated.

- c) System analysis results are provided for decisions or technical assessment needs.
- d) Enabling systems or services needed for system analysis are available.
- e) Traceability of the system analysis results is established.

6.4.6.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the system analysis process.

- a) Prepare for system analysis. This activity consists of the following tasks:

- 1) Define the system analysis strategy.
- 2) Identify the problem or question that requires analysis.

NOTE 1 Problem identification includes technical, functional, and non-functional objectives of the analysis. Non-functional objectives include critical quality characteristics, various properties, technology maturity, and technical risks. The problem statement or question to be answered by the analysis is essential to establish the objectives of the analysis and the expectations and utility of the results.

EXAMPLE For software systems, the high-level question that requires analysis is whether the software system as used in its intended or foreseeable environments can be a potential source of harm, irrespective of any fault, failure, or malfunction. Safety analyses can be iterated with respect to the outputs of each technical process and used to adjust the processes' activities.

NOTE 2 For guidance on the topic of safety of the intended functionality, see ISO 26262-6.

- 3) Identify the stakeholders of the analysis.
- 4) Identify and validate contexts and assumptions.

NOTE 3 Validation of assumptions is an ongoing concern. If assumptions change or are determined to be incorrect over time, the analysis is revised.

- 5) Define the scope, objectives, and level of fidelity of the analysis.

NOTE 4 The necessary level of fidelity (accuracy or precision) is a factor in determining the appropriate level of rigor.

- 6) Select the methods to support the analysis.

NOTE 5 The methods are chosen based on time, cost, fidelity, technical drivers, and criticality of analysis. Analysis methods have a wide range of levels of rigor and include expert judgment, worksheet computations, parametric estimates and calculations, historical data and trend analysis, engineering models, simulation, visualization, and prototyping. Due to cost and schedule constraints, most projects typically perform system analysis only for critical characteristics.

- 7) Identify and plan for the necessary enabling systems or services needed to support the analysis.

NOTE 6 This task includes identification of requirements and interfaces for the enabling systems. The system analysis enabling systems include the tools, relevant reusable models, and available and new data and data repositories needed to support the analysis. The methods chosen are a major factor in determining what tools are appropriate to support the analysis.

- 8) Obtain or acquire access to the enabling systems or services to be used.

NOTE 7 The infrastructure management process enables the provision of systems analysis services. The validation process is used to objectively confirm that the enabling system achieves its intended use for its enabling functions.

- 9) Collect the data and inputs needed for the analysis.

NOTE 8 The origin, quality, and validity of data is important to the formulation and execution of the analysis. Criteria for trustworthiness of data needed for the analysis are established. Data and inputs are reviewed for quality and validity (i.e. trustworthy data). Use authoritative sources.

b) Perform system analysis. This activity consists of the following tasks:

- 1) Apply the selected analysis methods to perform the required analysis.
- 2) Review the analysis results for quality and validity.

NOTE 9 The results are coordinated with associated analyses that have been previously completed. Trustworthiness of the results can be determined in a review.

- 3) Establish conclusions and recommendations.

NOTE 10 The appropriate subject matter experts and stakeholders are identified and engaged in this task.

- 4) Record the results of the system analysis.

c) Manage the system analysis. This activity consists of the following tasks:

- 1) Maintain traceability of the system analysis results.

NOTE 11 Through the life cycle, bidirectional traceability is maintained between the analysis results and any software system item for which the analysis is supporting a decision or providing rationale, e.g. system requirement values, architecture alternatives. Traceability is often facilitated by an appropriate data repository.

- 2) Provide key artefacts and information items that have been selected for baselines.

NOTE 12 The configuration management process is used to establish and maintain configuration items and baselines. This process identifies candidates for the baseline, and the information management process controls the information items. For this process, analysis records or reports are typical information items that are managed.

6.4.7 Implementation process

6.4.7.1 Purpose

The purpose of the implementation process is to realize a specified system element.

This process transforms requirements, architecture, and design, including interfaces, into actions that create a system element according to the practices of the selected implementation technology, using appropriate technical specialties or disciplines. This process results in a system element that satisfies specified system requirements (including allocated and derived requirements), architecture, and design.

Software system elements can include hardware, software, and services. For software implementation, this process transforms specified designs, behaviour, interfaces, and implementation constraints into a software system element implemented as a software product or service, also known as a software item. Software implementation results in a software element that satisfies specified requirements through verification and stakeholder requirements through validation. Software implementation includes various combinations of construction (coding of newly built software elements), acquisition of new software packages (e.g. from open source or a commercial or organizational source), or re-use of existing elements, with or without modification. Software code generation can utilize AI-based tools, low-code or no-code methods, framework-based techniques, and model-based software engineering ([Annex D](#)).

Software implementation commonly involves use of the agreement processes to obtain non-developmental items (NDI), such as hardware and operating systems (the platform) or enabling systems and services. The configuration management process is used to account for implemented and baselined software items.

The strategy for software implementation considers that this process is typically performed along with the technical management processes and many of the technical processes, especially:

- a) The integration process to continuously incorporate software elements into the software build.
- b) The verification process, which provides objective evidence that the software implementation fulfils its specified requirements and identifies anomalies, defects, and errors in implementation-related information items and artefacts, e.g. system requirements, architecture, design or other descriptions; processes; and software elements and items.
- c) The validation process, which confirms that the implementation fulfils requirements for a specific intended use of a software work product.

NOTE The SWEBOK v4 discusses implementation in the Software Construction knowledge area. This knowledge area addresses management; measurement; practical considerations, e.g. construction design, languages, testing, reuse, and integration; construction technologies, e.g. object-oriented, error and exception handling, executable models, distributed software, and tools.

6.4.7.2 Outcomes

As a result of the successful performance of the implementation process:

- a) Implementation constraints that influence the requirements, architecture, or design are identified.
- b) A system element is realized.
- c) Enabling systems or services needed for implementation are available.
- d) Implementation results and anomalies are identified.
- e) Traceability is established.

During implementation of a software system, software elements are acquired, identified for reuse from organizational assets, or developed (constructed). Software elements that are acquired can range from a simple product purchase in accordance with organizational or project acquisition procedures to a complex systems integration effort involving acquisition of a software system using the Acquisition and Supply processes. Data and Information can be embedded in the software or implemented as a separate element, which can be delivered as part of the software product or as a separately delivered information item. Adaptation includes configuration of software elements and libraries, such as open-source repositories, that are reused or modified. Construction can involve software coding, adaptive reuse and integration of existing units, refactoring, database development, and construction of manual or automated test procedures for each unit. Construction can include associated data structures, application programming interfaces (API), service descriptions, user documentation, test cases, or other elements.

6.4.7.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the implementation process.

- a) Prepare for implementation. This activity consists of the following tasks:
 - 1) Define an implementation strategy, with considerations such as the following:
 - i) development policies and standards, including standards that govern applicable safety, security, privacy, and environmental practices; programming or coding standards; unit test policies; and language-specific standards for implementing security features;
 - ii) for reused or adapted software, methods to determine the level, source, and suitability of the reused system elements and security of the supply chain;
 - iii) procedures and methods for software development (construction) and development of unit tests; and the use of peer reviews, unit tests, and walkthroughs during implementation;

- iv) change management considerations for manual processes;
- v) implementation priorities to support data and software migration and transition, along with retirement of legacy systems;
- vi) planning, design and creation of manual or automated test procedures to verify that a software unit meets its requirements; when done prior to coding can aid design and implementation of code as well as demonstrating adequacy of the code after the code has been implemented;
- vii) comprehensive or specialized life cycle development and support environments for realizing and managing requirements, models and prototypes, deliverable system or software elements, and test specifications and test cases.

NOTE 1 The implementation strategy is commonly recorded in a project's SDP or SEMP, or sometimes in a PMP.

- 2) Identify constraints from the implementation strategy and implementation technology on the system requirements, architecture characteristics, design characteristics, or implementation techniques.

NOTE 2 Constraints include current or anticipated limitations of the chosen implementation technology, e.g. for software, the operating system, database management system, web services; acquirer furnished materials or system elements for adaptation; and limitations resulting from the use of required implementation-enabling systems.

- 3) Identify and plan for the necessary and distinct software environments, including enabling systems or services needed to support development and testing.

NOTE 3 Implementation of software commonly uses distinct environments that are separated under configuration control from the operational (production) environment. Common implementation process, enabling systems, and services include comprehensive or specialized life cycle development and support environments for realizing and managing requirements, models and prototypes, deliverable items, test environments, specifications, and test cases; simulators for external systems, training systems; and content management systems for user documentation.

- 4) Obtain or acquire access to the software environments and other enabling systems or services.

NOTE 4 The validation process is used to objectively confirm that the integration enabling system achieves its intended use for its enabling functions.

- b) Perform implementation. This activity consists of the following tasks:

NOTE 5 Throughout the implementation process, the verification process is used to objectively confirm the system elements conform to requirements. The validation process is used to objectively confirm the element is suitable to be used in its intended operational environment according to stakeholder requirements.

- 1) Realize or adapt software elements, according to the strategy, constraints, and defined implementation procedures.

NOTE 6 For software elements that are developed, at the lowest level of implementation, executable software units are constructed, following established security practices for the development languages, controlled, made available to authorized roles, and stored according to the CM procedures for development artefacts.

- 2) Realize or adapt hardware elements of software systems.

NOTE 7 Hardware elements are acquired or fabricated using applicable techniques relevant to the physical implementation technology and materials selected. As appropriate, hardware elements are verified for conformance to specified system requirements and critical quality characteristics. In the case of repeated system element implementation (e.g. mass production and replacement system elements), the implementation procedures and fabrication processes are defined and can be automated to achieve consistent and repeatable producibility. Some common hardware elements in software systems include integrations of acquired COTS systems, special modifications, for example, for test or operational environments, and hardware controls with embedded software.

3) Realize or adapt service elements of software systems.

NOTE 8 Service elements include a set of services to be provided. The ISO/IEC 20000 series applies to management of system elements realized in services, including strategy, design, and transition. As appropriate, services are verified for conformance to the system requirements and service criteria. For example, operational resource elements are verified for conformance to the system requirements and operational concept. Service elements can include network communications, training, software packaging and distribution services, software customization services for customer-specific needs, operational and security monitoring, and user assistance.

4) Evaluate software unit and affiliated data or other information according to the implementation strategy and criteria.

NOTE 9 Criteria for evaluation commonly include satisfaction of unit requirements and test criteria, unit test coverage, traceability requirements, consistency with software element requirements or design, internal unit requirement consistency, and feasibility for further process activity, e.g. integration, verification, validation, operations, and maintenance.

NOTE 10 The "manage results of implementation" activity is used to record the results of construction and to address anomalies.

5) Combine the software unit into the software element being implemented.

NOTE 11 For software, the implementation process is applied to merge an implemented software branch into the software element or object being constructed (electronic or on physical media, which is stored in a controlled location and made available to authorized roles, e.g. for use in the integration and transition processes. Configuration and product information is captured by the configuration management and information management processes when the element is stored.

6) Record objective evidence that the software system element meets requirements.

NOTE 12 Evidence is provided in accordance with supply agreements, legislation, and organization policy. Evidence includes element modifications made due to processing changes or non-conformances found during the verification and validation processes. The objective evidence is part of the element's as-implemented configuration baseline established through the configuration management process and includes the results of verification tasks, such as unit testing, analysis, inspections, walk-through events, demonstrations, product, or technical reviews.

c) Manage results of implementation. This activity consists of the following tasks:

1) Record implementation results and anomalies encountered.

NOTE 13 Anomalies can be due to the implementation strategy, the implementation enabling systems, or incorrect software system definition. The system analysis, project assessment and control, and QA processes are used to analyse the data to identify the root cause, enable corrective or improvement actions, and to record lessons learned.

2) Maintain traceability of the implemented software system elements.

NOTE 14 To support traceability throughout the life cycle during operations and maintenance, sources of software licenses and other system assets in the supply chain are recorded. The information management and configuration management processes are used to maintain license and maintenance support terms for a software application and its required infrastructure (host system). The ISO/IEC 19770 series provides requirements for an IT asset management system.

NOTE 15 Bidirectional traceability is maintained between the implemented elements and the software system architecture; design, and related requirements, including interface requirements and definitions that are necessary for implementation; and validation and verification plans, procedures, and results.

3) Provide key artefacts and information items that have been selected for baselines.

NOTE 16 The configuration management process is used to establish and maintain configuration items and baselines. This process identifies candidates for the baseline, and the information management process controls the information items. For this process, the software system elements (source code), software packages, and unit test results are typical artefacts that are baselined.

6.4.8 Integration process

6.4.8.1 Purpose

The purpose of the integration process is to synthesize a set of system elements into a realized system (product or service) that satisfies the system requirements.

This process encompasses planning for, preparing for, and aggregating a progressively more complete set of system elements or artefacts. Interfaces are identified and activated to enable interoperability and subsequent verification and possibly validation of the requirements (including characteristics) of the system elements as intended. This process also connects and checks out interfaces of the SoI with enabling systems for which there is direct interaction.

Software system integration iteratively combines implemented software system elements to form complete or partial system configurations in order to build a product or service. Software integration is typically performed daily or continuously during development and maintenance stages, using automated tools. Continuous integration involves frequent inclusion or replacement and archiving of items in software libraries under CM control.

NOTE A detailed description of the integration process can be found in ISO/IEC/IEEE 24748-6. The SWEBOK v4 discusses integration in the Software Construction knowledge area. The Software Maintenance knowledge area discusses continuous integration, continuous delivery, continuous testing, and continuous deployment.

6.4.8.2 Outcomes

As a result of the successful performance of the integration process:

- a) Integration constraints that influence system requirements, architecture, or design, including interfaces, are identified.
- b) Approach and checkpoints for the correct operation of the assembled interfaces and system functions are defined.
- c) Enabling systems or services needed for integration are available.
- d) A system composed of implemented system elements or artefacts is integrated.
- e) The system external interfaces (system to external environment) and system internal interfaces (between implemented system elements) are checked.
- f) Integration results and anomalies are identified.
- g) Traceability of the integrated system elements is established.

6.4.8.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the integration process.

- a) Prepare for integration. This activity consists of the following tasks:
 - 1) Define the integration strategy.

NOTE 1 Integration builds sequences of progressively more complete software system element or software item configurations. It is dependent on applicable software system element availability and is consistent with a fault isolation and diagnosis strategy. Successive applications of the Integration process and the Verification process, and when appropriate the Validation process, are repeated for elements in the system structure until the SoI has been realized. Simulators or prototypes are typically utilized for system elements that are not yet implemented, e.g. receiving data from interfacing systems. Integrating the implemented software system elements is based on the priorities of the related requirements and system architecture definition, typically focusing on the interfaces, while minimizing integration time, cost, and risks. Software system integration commonly maintains version control through the Configuration Management process for selection of configuration items to be integrated.

NOTE 2 For software integration, the integration strategy typically is consistent with a regression strategy which is applied for re-verifying software elements when related software units (and potentially associated requirements, design, and user documentation) are changed.

NOTE 3 Defining a strategy for software unit and element integration commonly accompanies defining the strategy for other processes that occur concurrently, such as:

- i) The Implementation process to help ensure timely coordination of implementation and integration process tasks and enabling systems, for example, combined software development and test environments to support automated or continuous implementation and integration of software units and elements.
- ii) The verification process to provide objective evidence that the integrated software fulfils its specified requirements and to identify anomalies (errors, defects, faults) in integration-related information items, e.g. system requirements, architecture, design, test, or other descriptions; processes; and software elements, items, and units.
- iii) The validation process to confirm that a work product fulfils requirements for a specific intended use of an integrated software function.
- iv) The QA process to support integration process and work product audits and inspections and to address problem, non-conformance, or incident reporting and handling.

NOTE 4 The integration strategy is commonly recorded in a plan, e.g. a project's SDP or SEMP.

- 2) Identify and define criteria for integration and points at which the correct operation and integrity of the interfaces and the selected software system functions are verified.

NOTE 5 Detailed verification of the interfaces is performed using the verification process. Software integration typically involves combining software elements, resulting in a set of integrated software elements, that is consistent with the software design, and that satisfies the functional and non-functional system requirements on an equivalent of the operational environment.

NOTE 6 For projects involving multiple suppliers or development teams, the availability of software system elements for integration is typically part of the project schedule with milestones under the project assessment and control process. Integration proceeds as the software is verified in its functionality, performance, and suitability for site-specific or platform-specific environments. At major integration points, for example, completion of a stage, element, or version, check points for reviews and validation with stakeholders are typically held. The frequency of these reviews is related to the selected life cycle model and development method.

- 3) Identify and plan for the necessary enabling systems or services needed to support integration.

NOTE 7 This task includes identification of requirements and interfaces for the enabling systems. Enabling systems for integration commonly include integration facilities, specialized equipment, training systems, discrepancy reporting systems, simulators, measurement devices, and environmental security. For software, this can involve regression test suites and CM systems for the integrated testing of software systems, incident and problem reporting systems, simulators representing external systems or undeveloped elements, and software library management systems for development operations. Changes or specializations needed for the enabling systems to support the integration tasks need to be identified and defined. Typically, the enabling systems or services used for integration during development stages can also help support system element integration as the software system and enabling environments evolve to operational status. This DevOps approach supports iterative software system implementation, integration, verification, transition, validation, operation, and maintenance processes.

- 4) Obtain or acquire access to the enabling systems or services to be used to support integration.

NOTE 8 The validation process is used to objectively confirm that an integration enabling system achieves its intended use for its enabling functions.

- 5) Identify constraints for integration to be incorporated in the system requirements, architecture, or design.

NOTE 9 Constraints typically affect requirements for accessibility, supply chain security, safety for integrators, required interconnections for sets of implemented software system elements and for enablers, and interfaces.

- b) Perform integration. Successively integrate software system elements and configurations until the complete system is synthesized. This activity consists of the following tasks:

- 1) Obtain implemented software system elements in accordance with agreed schedules.

NOTE 10 The implemented software system elements are provided from the developers or received from suppliers, the acquirer, or other resources and typically placed under CM control. The elements are handled in accordance with relevant health, safety, security, and privacy considerations.

- 2) Integrate the implemented elements.

NOTE 11 This task is performed to achieve software system configuration (complete or partial) connecting the implemented elements as described in the integration strategy, using the defined procedures, interface control descriptions, and the related integration enabling systems.

NOTE 12 In terms of software, integrating the implemented elements can involve linking together pieces of object code or simply bringing together the implemented elements that are part of the software configuration in a methodical piece by piece approach. Software elements are typically compiled into a “build” so that branched units are properly linked or merged in the assembled element. Firmware elements are fabricated, often as prototypes, and integrated in hardware elements. If software functions are not yet available for integration, emulated functionality (stubs or scaffolding) can be used to temporarily support integration of software elements or represent input from external interfaces. Successful aggregations result in an integrated software element, that is stored and available for further processing, i.e. additional software element integration, verification, or validation.

NOTE 13 Anti-counterfeit, anti-tamper, system and software assurance and interoperability concerns can arise when performing integration and identifying and defining checkpoints. Integration and Verification processes often use fictitious data for security or privacy considerations. The ISO/IEC/IEEE 15026 series and ISO/IEC 27000 include information on assurance, integrity, and security considerations affecting integration.

- 3) Check that the integrated software interfaces or functions run from initiation to an expected termination within an expected range of data values.

NOTE 14 As part of the acceptance of the implemented software system elements, selected elements are checked to help ensure they meet acceptance criteria as specified in the integration strategy and applicable agreements. Checking can include conformance to the agreed configuration, compatibility of interfaces, and the presence of mandatory information items. The project assessment and control process can be used in accordance with the integration strategy to plan and conduct technical reviews of the integrated software system elements, for example, a test readiness review to help ensure the integrated element or system with its affiliated data and information items is ready for qualification testing.

c) Manage results of integration. This activity consists of the following tasks:

1) Record integration results and anomalies encountered.

NOTE 15 Anomalies are typically due to the integration strategy, the integration enabling systems, execution of the integration or incorrect system or element definition. Where inconsistencies exist at the interface between the system, its specified operational environment and systems that enable the utilization stage, the deviations lead to corrective actions. Anomaly resolution typically involves the technical processes, often repetitive application of the Implementation process. The QA and project assessment and control process are used to analyse the data to identify the root cause, enable corrective or improvement actions, and to record lessons learned.

2) Maintain traceability of the integrated software system elements.

NOTE 16 Bidirectional traceability is maintained between the integrated system elements and the software system architecture, design, and system or element requirements, such as use cases, and including interface requirements and definitions that are necessary for integration. Integrated software elements and their components are identified by version. Versions of integrated software elements are commonly traceable to implemented units, test procedures, and test cases.

3) Provide key artefacts and information items that have been selected for baselines.

NOTE 17 The configuration management process is used to establish and maintain configuration items and baselines. The integration process identifies candidates for the baseline, and the information management process controls the information items. For this process, the test cases, regression tests, and automated test scripts are typical artefacts that are baselined. The integration strategy is a typical information item that is baselined.

6.4.9 Verification process

6.4.9.1 Purpose

The purpose of the verification process is to provide objective evidence that a system, system element, or artefact fulfils its specified requirements and characteristics.

The verification process identifies the anomalies in any artefact (e.g. system requirements, architecture description, or design description), implemented system elements, or life cycle processes using appropriate methods, techniques, standards, or rules. This process provides the necessary information to determine resolution of identified anomalies.

Verification can be performed across all technical processes and reiterated throughout the life cycle as technical processes are repeated until system retirement and disposal. The Verification process is typically applied continually throughout software development and particularly at key decision points in a software system's transitions. It demonstrates that the requirements (including functional and non-functional requirements) have been met, or that process outcomes have been achieved or process activities have been performed. Different domains and engineering or development communities can identify the milestones, verification strategies and criteria differently.

For software systems, the verification process is typically instantiated for the following purposes:

- a) to confirm that a software work product or service properly reflects the specified requirements (often called software verification);
- b) to confirm that the integrated software product meets its defined requirements (often called software qualification testing), and that an evolving software work product functions consistently with related software;
- c) to confirm that the implementation of each software system requirement is tested for compliance and that the software system is ready for delivery (often called system qualification testing).

NOTE 1 The verification process determines that the "product is built right". The validation process determines that the "right product is built".

NOTE 2 Construction of an assurance case (as described in [Annex E](#) and in the ISO/IEC/IEEE 15026 series) can be helpful to provide insight for verification activities and to present verification results.

NOTE 3 The ISO/IEC/IEEE 29119 series provides detailed processes and techniques for verification performed through testing. IEEE Std 1012-2012 provides additional details about these processes for systems, software, hardware, and interfaces being developed, maintained, or reused.

NOTE 4 The SWEBOK v4 discusses verification in the Software Testing and Software Quality knowledge areas. The Software Testing knowledge area addresses test levels, testing objectives, test techniques, test measurement, and the test process. Testing is also discussed in the context of Implementation in the Software Construction knowledge area, operations in the Software Operations knowledge area, ongoing maintenance in the Software Maintenance knowledge area, and security testing in the Software Security knowledge area. The Software Quality Assurance knowledge area discusses static analysis, dynamic analysis, formal analysis, software quality control, and technical reviews and audits in the context of verification, validation, and testing.

6.4.9.2 Outcomes

As a result of the successful performance of the verification process:

- a) Constraints of verification that influence the requirements, architecture, or design are identified.
- b) Enabling systems or services needed for verification are available.
- c) The system or system element, or artefact is verified.
- d) Data providing information for corrective actions is reported.
- e) Objective evidence that the realized system fulfils the requirements, architecture and design is provided.
- f) Verification results and anomalies are identified.
- g) Traceability of the verified system elements is established.

6.4.9.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the verification process.

- a) Prepare for verification. This activity consists of the following tasks:

- 1) Define the verification strategy.

NOTE 1 A verification strategy generally focuses on minimizing cost, schedule, or risk, providing a balanced approach for confirming that the software system or element has been “built right”. The verification strategy is aligned with the integrity level and identified risks of the software system, considering factors such as software complexity, criticality, safety level, security level, accuracy, likelihood of meeting performance and reliability requirements, as well as testability. ISO/IEC/IEEE 15026-3 provides details of integrity levels.

NOTE 2 The verification strategy and schedule account for dynamic changes when anomalous results (events, incidents, or problems) occur. According to the progress of the project, planned verification activities are redefined or rescheduled when unexpected events or system evolutions occur.

NOTE 3 The verification strategy can be documented in a plan, for example, a verification plan, or a project’s SDP or SEMP.

- 2) Identify the verification scope, including the software system, element, or artefact, the properties to be verified, and the expected results.

NOTE 4 Overall verification scope includes the software *SoI* or system elements, including interfaces. The verification scope identifies the software system, element, or artefact to be verified, e.g. the actual system, or a model, a mock-up, a prototype, code, a procedure, a plan, or other document, and the expected results, such as conformance, or performance, fault tolerance, and recovery after service interruption. The properties to be verified can include requirements, architecture and design characteristics, integration, and accuracy of documentation. Design characteristics can include security implications of the design in the context of the planned operational environment and the achievement of critical quality characteristics as stated in the requirements.

3) Identify the constraints that potentially limit the feasibility of verification activities

NOTE 5 Constraints include system type, life cycle process model, technical feasibility, cost, time, availability of verification enablers or qualified personnel, contractual constraints, and characteristics such as criticality of the mission. Such constraints often factor into verification strategy determination, for example, whether an organizationally independent verification effort is necessary or justified.

4) Identify verification priorities.

NOTE 6 In software systems, exhaustive verification (sometimes called 100 % code coverage) is typically infeasible. The verification strategy typically includes trading off what is to be verified (scope) against the constraints or limits, and deducing what verification tasks to perform and how many iterations of verification and rework are needed to reduce risk. Coverage is often measured by requirements coverage and structural (line, branch) coverage. A model-based testing approach can enable the generation and management of multiple scenarios. Potential verification tasks that are candidates for deletion are evaluated for the risks their withdrawal imposes.

5) Identify constraints from the verification strategy to be incorporated in the system requirements, architecture, or design.

NOTE 7 Constraints typically result from practical limitations of accuracy, uncertainty, repeatability that are imposed by the verification enablers, the associated measurement methods, the need for software system integration, and the availability, accessibility, and interconnection with enablers.

6) Define the purpose, conditions, and conformance criteria for each verification task.

7) Select appropriate verification methods or techniques and associated criteria for verification tasks, such as inspection, analysis, demonstration, or testing.

NOTE 8 The selection of verification methods or techniques is made according to the type of system, the purpose of the verification, the objectives of the project, and the acceptable risks. Verification methods or techniques include review, inspection (including code inspection, walkthroughs, technical and peer review), analysis (including modelling and simulation, and analogy/similarity), demonstration, and dynamic and static testing.

NOTE 9 The selected verification approach, methods, and techniques can be coordinated with relevant stakeholders to help ensure the verification approach is acceptable.

8) Identify and plan for the necessary enabling systems or services needed to support verification.

NOTE 10 Verification enabling systems include verification facilities, qualified personnel, equipment, simulators, test automation tools, and incident and problem management systems. Software system verification is typically performed in distinct controlled environments that do not interfere with operational software or ongoing development. If the enabling systems for verification differ in capability from the planned operational environment, the Measurement process can be used to calibrate the performance of the verification enabling systems and suitability for the verification activity.

9) Obtain or acquire access to the enabling systems or services to be used to support verification.

NOTE 11 The acquisition of the enabling systems can be done through various ways such as rental, procurement, development, reuse of organizational assets or subcontracting. Usually, the acquisition of the complete set of enablers is a mix of these ways. The validation process is used to objectively confirm that the verification enabling system achieves its intended use for its enabling functions.

b) Perform verification. This activity consists of the following tasks:

- 1) Define the verification procedures, each supporting one or a set of verification activities.

NOTE 12 Verification procedures, which can be performed by automated scripts, include the requirements to be verified, the type of software system element or artefact to be verified and the expected results (success criteria), such as conformance, or performance of a function or capacity in terms of response time or throughput. The procedures identify the purpose of the verification with success criteria (expected results), the verification technique to be applied, the necessary enabling systems (facilities, equipment), and the environmental conditions to perform each verification procedure (resources, qualified personnel, specialized procedural set-up or work instructions). Verification procedures include how the verification procedure results are recorded, analysed, stored, and reported.

- 2) Perform the verification procedures.

NOTE 13 Verification occurs, in accordance with the verification strategy, at the appropriate time in the schedule, in the defined environment, with defined enabling systems and resources. The performance of a verification activity consists of capturing a result from the execution of the verification procedure; comparing the obtained and recorded result with the expected result; and deducing a degree of correctness (or success/failure) of the submitted element.

c) Manage results of verification. This activity consists of the following tasks:

- 1) Review verification results and anomalies encountered and identify follow-up activities.

NOTE 14 Anomalies can originate in the verification strategy, the verification enabling systems, execution of the verification, or incorrect system definition. The project assessment and control and the quality assurance processes are used to analyse the data to identify the root cause, enable corrective or improvement actions, and to record lessons learned.

NOTE 15 The evaluation of verification results and follow-up corrective action can vary greatly depending on the purpose of the verification. For elements of software, examples include a modification or waiver of requirements, a simple defect fix for a failed software element, followed by re-verification, or major project re-direction based on a failure to attain a key milestone, e.g. failed software system qualification testing. Often simple or recommended solutions to anomalies discovered during verification, are recorded with the verification result to facilitate analysis and potential corrective action.

- 2) Record incidents and problems during verification and track their resolution.

NOTE 16 Performing problem resolution is handled through the QA and project assessment and control processes. During software verification, the conditions under which the problem occurred are documented so that, if possible, the problem can be duplicated and the root cause of the software defect identified. Changes to the requirements, architecture, design, or system elements are done using other technical processes.

- 3) Obtain stakeholder agreement that the software system or element meets the specified requirements.
- 4) Maintain traceability of the verified software system elements.

NOTE 17 Bidirectional traceability is maintained between the verified system elements and the record of verification activity, system architecture, design, or system requirements.

- 5) Provide key artefacts and information items that have been selected for baselines.

NOTE 18 The configuration management process is used to establish and maintain configuration items and baselines. This process identifies candidates for the baseline, and the information management process controls the information items. For this process, the verification strategy and verification procedures are typical information items.

6.4.10 Transition process

6.4.10.1 Purpose

The purpose of the transition process is to establish a capability for a system to provide services specified by stakeholder requirements in the operational environment.

This process moves the system in an orderly, planned manner to be operable in the intended environment, which may be a new or changed environment, e.g. operations or validation. As a result of the transition, the system is functional and compatible with enabling, interfacing, and interoperating systems in the environment. It installs a verified system, together with relevant enabling systems, (e.g. planning system, support system, operator training system, user training system), as defined in agreements. The transition process can be used every time the system or system elements are transitioned from one entity or environment to another.

For software systems, the purpose of the transition process is to establish a capability for a system to provide services in a different environment.

Software configurations are changed through the controlled release of a new version. The purpose of a release is to authorize and effect the availability of a software feature, function, or system for a specific purpose, with or without restrictions to a subset of users.

The transition process is often used for recurring deployments of software to different environments, e.g. from a development environment to a test or maintenance environment, or between various test environments, or from one operational environment to another (e.g. rehosting or use of cloud services). Transition encompasses release management of selective releases of software features to different groups of users. Transition can involve changes to the data sources, data structure, or updates or upgrades of functional software. Transition includes recurring scheduled or emergency patches and fixes for security and other concerns. Transition can involve transfer between organizations and also encompasses the addition of a large group of new users to an existing software system or service. Transitions to backup or contingent sites are typically planned and rehearsed for business continuity and disaster recovery. Transition for software systems can involve the use of cloud services, physical relocation of hardware, the installation and activation or deactivation of physical or virtual infrastructure or enabling systems in different locations, or no change to the physical infrastructure. Transition to a new system often is performed concurrently with retirement and disposal of an existing system, entailing data migration from the old system to its replacement.

NOTE 1 Transition can involve knowledge transfer using the knowledge management process.

NOTE 2 In the case of system upgrades, a typical goal is that transition activities are accomplished with minimal disruption to ongoing operations.

NOTE 3 Software releases are performed using the transition process, while accounting for released software is performed using the configuration management process.

NOTE 4 The SWEBOKv4 discusses software release and delivery in the Software Engineering Operations knowledge area.

6.4.10.2 Outcomes

As a result of the successful performance of the transition process:

- a) Transition constraints that influence system requirements, architecture, or design are identified.
- b) Any enabling systems or services needed for transition are available.
- c) The site is prepared.
- d) The system, as installed in its operational location, is capable of delivering its specified functions.
- e) Operators, users, and other stakeholders necessary to the system utilization and support are trained.
- f) Transition results and anomalies are identified.

- g) The installed system is activated and ready for operation.
- h) Traceability of the transitioned elements is established.

6.4.10.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the transition process.

- a) Prepare for the software system transition. This activity consists of the following tasks:
 - 1) Define a strategy for managing software releases and other software system transitions, including the following considerations:
 - i) establishing the type of transition and transition success criteria;
 - ii) determining the frequency of recurring transitions, such as updates and upgrades to development, test, and operational software systems;
 - iii) minimizing security risks, disruption, and downtime during transition;
 - iv) archiving, destroying, or converting and validating data from previous systems, versions, or environments to the target system; including data received through external interfaces;
 - v) contingency planning for problem resolution, backup and return to the last working system version;
 - vi) scheduling transitions consistent with ongoing business processing, with phased or synchronized transition of systems;
 - vii) planning for change management for stakeholders, including interface partners, human operators, system administrators, and software system or service users.

NOTE 1 Change management activities are often conducted to design changes in business processes associated with the new system, plan the transition in business processes, and gain user commitment to productive use of the new system:

- coordinating with associated strategies for validation of the transitioning system or element;
- initiating user support and maintenance activities with the transfer and update of system design documentation, user documentation, and test procedures;
- planning for concurrent execution of the transition, operations, and disposal processes, when a new system is commissioned and an old system is decommissioned.

NOTE 2 The strategy includes roles and responsibilities, approval authority, use of readiness reviews and training.

- 2) Identify and define facility, site, communications network, or target environment changes needed for software system installation or transition.

NOTE 3 For each transition, identify and define any needed changes in infrastructure or enabling systems. A site survey can be performed to identify needed changes in the physical environment to install or use the software system, such as changes to maintain the physical and information security of the system.

- 3) Identify information needs and arrange for user documentation and training of operators, users, and other stakeholders necessary for system utilization and support.

NOTE 4 Transition includes migration or activation of user access to the software system. User roles are established and user accounts and access controls are implemented.

- 4) Prepare detailed transition information, such as plans, schedules, and procedures.

NOTE 5 The transition strategy is commonly recorded in a plan, for example, a transition plan, or a project's SDP or SEMP. Transition schedules help validate that sufficient resources and infrastructure are available to support the transition, so that activities can be executed within a reasonable timeframe to minimize disruption. Schedules can include rehearsals for complex transitions, in which procedures, such as database and system backup and restore and software installation, are tested to verify durations and correct results.

NOTE 6 During a specified period of changeover or concurrent operation, the transfer of services is managed so that continuing conformance to persistent stakeholder needs or an agreed level of service is achieved. If a period of parallel operations for both the old and new systems is needed, special procedures are identified and developed for receiving and utilizing data from interface partners.

- 5) Identify system constraints from transition to be incorporated in the software system requirements, architecture, or design.
- 6) Identify and plan for the necessary enabling systems or services needed to support transition.

NOTE 7 This task includes identification of requirements and interfaces for the enabling systems. Transition often involves the use of highly automated infrastructure to deliver, install, and activate or inactivate software. For electronic software distribution, temporary or continuing changes in connectivity are often needed for software and data migration and continuing sustainment. Enabling systems can include backup or alternate systems for use during a transitional period.

- 7) Obtain or acquire access to the enabling systems or services to be used.

NOTE 8 The Validation process is used to objectively confirm that the transition enabling system achieves its intended use for its enabling functions.

- b) Perform the transition, including software releases. This activity consists of the following tasks:

- 1) Prepare the site or configure the virtual environment in accordance with installation requirements.

NOTE 9 Site preparation is conducted in accordance with applicable health, safety, security, and environmental regulations. Virtual environments and new communication resources are initialized and verified. Shipping and receiving of physical system elements and enabling systems are arranged.

- 2) Demonstrate that the enabling systems are ready to support functions provided by the transitioning system.

NOTE 10 This task examines readiness of enabling systems for operation in the target environment. For example, activation of monitoring, problem reporting, access control, backup and recovery, and user assistance (customer support) can be demonstrated.

- 3) Deliver or move the transitioning system to its physical or virtual location and interface to interoperating systems.

NOTE 11 The software system installation includes configuring it with required data, changes to the environment, or business process changes. Databases are instantiated and data migration is performed as applicable. Licenses and maintenance agreements for system elements and other intellectual property are transferred.

NOTE 12 Typically software is delivered electronically. For physical media, hardware, and embedded software systems, it is sometimes necessary to account for temporary storage prior to delivery or installation.

- 4) Perform software releases.

NOTE 13 The life cycle model helps determine the frequency of iterative or incremental software releases. The Integration process is used to select and configure a release package, software version, update, or patch. These changes are verified or validated through the verification or validation processes. The configuration management process is used to obtain approval of the release and to maintain configuration records.

NOTE 14 For software systems, automated version control tools can help ensure that only the correct source code versions are accessed, updated, tested, and documented for approved changes by appropriate personnel, and released.

NOTE 15 The software system release is distributed in accordance with agreements and with the policies of the organizations involved.

- 5) Demonstrate proper activation of the software system to deliver its required functions.

NOTE 16 This task can involve the steps needed to activate the product to a usable state, including start-up and readiness evaluations, in accordance with operational procedures, organizational policies, and regulations. Where the exact location or environment of use is not available or when software is accessed from multiple or mobile locations, a representative example is selected.

- 6) Provide user documentation and training for the operators, users, and other stakeholders necessary for system utilization and support.

NOTE 17 Agreed information items are delivered as part of the software release in electronic or physical form, such as training material, logistics support packages, or user documentation.

- 7) Review the software system for readiness for use in the new environment.

NOTE 18 This task can include integrity checks of data and operations, e.g. that the software code and data representations properly initialize, execute, and terminate as specified. Specific attention is given to the data interfaces and security concerns. Information assurance and interoperability functions are exercised.

NOTE 19 For a transition to operations, this includes the results of functional demonstrations, validation activities, and sustainment demonstrations. A readiness review can be conducted. Deficiencies, risks, and problems that impact the success of the transition are resolved, accepted for waiver, or closed.

- 8) Commission the software release or system for operations.

NOTE 20 This task can include providing support to the users, administrators, and operators during the operations commencement (commissioning) of the system.

- c) Manage results of transition. This activity consists of the following tasks:

- 1) Track and record distribution of software system releases to specified environments or acquirers.

NOTE 21 Tracking can include an FCA or PCA using the CM process.

- 2) Record transition results and anomalies encountered.

NOTE 22 Anomalies can result from the transition strategy, the transition enabling systems, execution of the transition or incorrect software system or database system definition. Where inconsistencies exist between the system, its operational environment, and enabling systems, the deviations are resolved through corrective actions, including requirement changes. The project assessment and control and QA processes are used to analyse the data to identify the root cause, enable corrective or improvement actions, and to record lessons learned.

- 3) Record transition incidents and problems and track their resolution.

NOTE 23 During transition, the conditions under which the problem occurred are documented so that, if possible, the problem can be duplicated and the root cause of the defect identified. Changes to the requirements, architecture, design, or software system elements are done using other technical processes.

- 4) Maintain traceability of the transitioned software system elements.

NOTE 24 Bidirectional traceability is maintained between the transitioned and deployed system and elements and the approved and controlled versions of the software system and enabling systems.

- 5) Provide key artefacts and information items that have been selected for baselines.

NOTE 25 The CM process is used to establish and maintain configuration items and baselines, including transitioned software system elements. This process identifies candidates for the baseline, and the information management process controls the information items. For this process, the transition strategy, training material, and installation, transition and data migration procedures, and user documentation are typical information items that are baselined.

6.4.11 Validation process

6.4.11.1 Purpose

The purpose of the validation process is to provide objective evidence that the system, when in use, fulfils its business or mission objectives and stakeholder needs and requirements, achieving its intended use in its intended operational environment.

The objective of validating a system or system element, or artefact is to acquire confidence in its ability to meet validation criteria. Validation is confirmed by stakeholders. This process provides the necessary information so that identified anomalies can be resolved by the appropriate technical process where the anomaly was created.

For software systems, the following are also purposes of the validation process:

- a) to confirm that a need for a specific intended use of the software work product is fulfilled (often called software validation);
- b) to achieve confidence (especially with an acquirer or customer) that the delivered product meets stakeholder needs and requirements and is fit for use (often called software acceptance testing).

NOTE 1 Acceptance criteria, as used for acceptance testing, include criteria to determine whether the delivered product is fit to use or not. Acceptance criteria can be specified and agreed between two parties, i.e. an acquirer and a supplier, and included in the stakeholder requirements.

The validation process is typically used at key points in a product's life cycle to demonstrate that the product's requirements for stakeholder intended operational use have been met. Validation is also applicable to the software engineering artefacts (viewed as software system elements, such as requirements, architecture, design, design characteristics, or system elements) produced in the definition and realization of the system.). Different domains and engineering or development communities can identify the milestones, validation strategies and criteria differently.

For software systems, highly iterative life cycle models often feature frequent involvement by the acquirer, user representative, or other stakeholders to validate, e.g. the priority of requirements for inclusion in an iteration, the usability of the software interface through prototypes, and the suitability of the software for performing business tasks and fulfilling the operational concept. In agile, customer or user representatives are frequently part of the team to validate the work.

NOTE 2 The validation process determines that the "right product is built". The verification process determines that the "product is built right."

NOTE 3 Construction of an assurance case (see [Annex E](#)) can be helpful to provide insight for validation activities and to present validation results.

NOTE 4 The ISO/IEC/IEEE 29119 series provides detailed processes and techniques for validation performed through testing. IEEE Std 1012-2012 provides detailed requirements.

NOTE 5 The SWEBOK v4 discusses validation in the Software Testing and Software Quality knowledge areas. The Software Testing knowledge area addresses test levels, testing objectives, test techniques, test measurement, and the test process. Testing is also discussed in the context of Implementation in the Software Construction knowledge area, operations in the Software Operations knowledge area, ongoing maintenance in the Software Maintenance knowledge area, and security testing in the Software Security knowledge area. The Software Quality Assurance knowledge area discusses static analysis, dynamic analysis, formal analysis, software quality control, and technical reviews and audits in the context of verification, validation, and testing.

6.4.11.2 Outcomes

As a result of the successful performance of the validation process:

- a) Validation criteria are defined.
- b) The availability of services required by stakeholders is confirmed.

- c) Constraints of validation that influence the requirements, architecture, or design are identified.
- d) The system or system element, or artefact is validated.
- e) Enabling systems or services needed for validation are available.
- f) Validation results and anomalies are identified.
- g) Objective evidence of successful validation is provided.
- h) Traceability of the validated system elements is established.

6.4.11.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the validation process.

- a) Prepare for validation. This activity consists of the following tasks:

- 1) Define the validation strategy.

NOTE 1 A validation strategy generally focuses on minimizing cost, schedule, or risk by progressively building confidence in the quality and suitability of the software system for the stakeholders. Stakeholder validation is related to their concerns for factors such as safety level, security level, alignment with ethical values, usability, desired performance, and reliability.

NOTE 2 The validation strategy reflects the life cycle model, and often involves repeated validation for iterative, incremental, or evolutionary life cycles.

NOTE 3 The validation strategy can be documented in a plan, for example, an acceptance plan, or a project's SDP or SEMP.

- 2) Identify the validation scope, including the characteristics of the software system, element, or artefact to be validated, and the expected results of validation.

NOTE 4 Software system validation is typically performed both in distinct controlled environments that do not interfere with operational software or ongoing development, as well as in operational environments, typically before full operational use, e.g. beta testing or acceptance testing for a specified duration with agreed criteria. Scope includes stakeholder requirements, including related views of the system, e.g. scenarios or concept of operation to be evaluated. The scope depends on what is appropriate for the systems life cycle stage: the SoI or a system element or engineering artefact, such as a concept description or document, an operational scenario, a model, a mock-up, or prototype. The scope also includes evaluating that the software product or service is usable in its intended environment for the principal or critical functions. Additional characteristics to be validated can include usability of the documentation; fault tolerance, resilience, and recovery features of the software.

- 3) Identify the constraints that potentially limit the feasibility of validation.

NOTE 5 Constraints include practical limitations of accuracy, uncertainty, repeatability that are imposed by the validation enablers, the associated measurement methods, and the availability, accessibility, and interconnection with enablers. The validation strategy is constrained by the progress of the project; in particular, planned validation tasks are redefined or rescheduled when unexpected events or system evolutions occur. Validation can be extended to include ongoing measurements of user and customer satisfaction or complaints.

- 4) Identify validation priorities.

NOTE 6 To make effective use of stakeholders' time and expertise, validation typically focuses on stakeholder priorities, while verification is used for non-functional requirements. Potential validation tasks that are candidates for deletion are evaluated for the risks their withdrawal imposes.

NOTE 7 The supplier, the acquirer, or an agent of the acquirer participates in or performs validation. The responsibility is often designated in the agreement.

- 5) Identify system constraints from the validation strategy to be incorporated in the stakeholder requirements.
- 6) Define the purpose, conditions, and conformance criteria for each validation task.
- 7) Select appropriate validation methods or techniques and associated criteria for each validation task.

NOTE 8 Software system validation methods or techniques include inspection, analysis, analogy/similarity, demonstration, simulation, peer review, and testing. Software validation techniques typically include demonstrations, inspection, reviews and code walkthroughs, usability tests, and trial use of the software, e.g. beta testing, operational testing, user testing, or an acceptance test with agreed criteria. The selection of validation methods or techniques is made according to the type of system, the purpose of the validation, the objectives of the project, legal and regulatory requirements, and the acceptable risks of a validation activity. For software systems with human interaction, usability testing is commonly used to validate that representative users can achieve specified goals with effectiveness, efficiency, and satisfaction in a specified context of use. Further details for usability testing are found in ISO/TR 25060.

NOTE 9 Where appropriate, validation steps or states are defined, e.g. in-house validation, on-site validation, operational validation that progressively build confidence in conformance of the evolving software system, and assist diagnosis of any encountered discrepancies.

NOTE 10 Criteria for stakeholder acceptance of service performance is commonly stated as a service level and recorded in a service level agreement (SLA). Service levels typically measure capacity, availability, reliability, and timely response for services, and result in performance requirements for the supporting systems.

- 8) Identify and plan for the necessary enabling systems or services needed to support validation.

NOTE 11 Enabling systems can include simulators, usability laboratory or test facility, qualified personnel, stakeholder, and user representatives, according to selected validation methods or techniques. This task includes identification of requirements and interfaces for enabling systems.

- 9) Obtain or acquire access to the enabling systems or services to be used to support validation.

NOTE 12 The infrastructure management process or acquisition process can be invoked to obtain access to enabling systems, such as through rental, procurement, development, reuse, or subcontracting. Usually access to the complete set of enablers is a mix of these ways. The validation process is also used to objectively confirm that the validation enabling system achieves its intended use for its enabling functions.

b) Perform validation. This activity consists of the following tasks:

- 1) Define the validation procedures, each supporting one or a set of validation activities.

NOTE 13 Validation procedures identify stakeholder requirements to be validated, the associated software system artefact, e.g. the actual system, or a model, a mock-up, a prototype, code, a set of instructions or other information item, and the expected results (success criteria), such as completed and timely performance of a function. The procedures identify the purpose of the validation with success criteria (expected results), the validation technique to be applied, the necessary enabling systems (facilities, equipment), and the environmental conditions to perform each validation procedure (resources, qualified personnel, participating stakeholders, and specialized procedural set-up or work instructions). Validation strategy includes how the validation procedure results is recorded, analysed, stored, and reported.

- 2) Perform the validation procedures in the defined environment.

NOTE 14 Validation occurs in accordance with the validation strategy, at the appropriate time in the schedule, in a defined environment (such as the operational environment, a similar test environment, or other representative environment), with defined enablers and resources. The performance of a validation activity typically consists of capturing execution results, comparing the obtained result with the success criteria, and deducing a degree of compliance or stakeholder satisfaction with the software system, element, service, or engineering artefact.

NOTE 15 Acceptance tests are sometimes defined in the agreement to demonstrate satisfactory installation and system activation. This task interacts with the transition process to objectively confirm that the system fulfils stakeholder requirements in the operational environment. Acceptance tests, as specified in agreements, can define the criteria that demonstrate that the software system can deliver the required functions and services when installed and sustained in its operational environment. Specific attention is given to the key functions and logical interfaces.

c) Manage results of validation. This activity consists of the following tasks:

- 1) Review validation results and anomalies encountered and identify follow-up activities.

NOTE 16 Confirm that the services of the system that are required by stakeholders are available. Anomalies can result from the validation strategy, the validation enabling systems, execution of the validation, incorrect system definition, or inefficient or ineffective system design, implementation, and integration.

NOTE 17 The evaluation of validation results and follow-up activities can include acceptance of the anomaly as a low-risk occurrence. Corrective action can vary greatly depending on the impact of the validation result. For elements of software, examples include a simple defect fix for a failed software element, additional training for users, corrections and clarifications in documentation, or major project re-direction based on a failure to attain a key milestone, e.g. failed software system acceptance testing.

- 2) Record incidents and problems during validation and track their resolution.

NOTE 18 The project assessment and control process and QA process are used to analyse the data to identify the root cause of problems, enable corrective or improvement actions, and to record lessons learned. During software validation, the gap between stakeholder expectations and system performance is documented so that, if possible, the root cause of the discrepancy can be identified. Problem resolution typically involves determining the severity and impact of the problem and whether or when a software discrepancy is to be corrected or accepted for a time as a known error. Often simple or recommended solutions to anomalies discovered during validation are recorded with the validation result to facilitate analysis and potential corrective action. Actual changes to the stakeholder and system requirements, architecture, design, or system elements are done within other technical processes.

- 3) Obtain stakeholder agreement that the software system or element meets the stakeholder needs.
- 4) Maintain traceability of the validated system elements.

NOTE 19 Bidirectional traceability is maintained between the validated system elements and the stakeholder requirements and record of validation results.

- 5) Provide key artefacts and information items that have been selected for baselines.

NOTE 20 The configuration management process is used to establish and maintain configuration items and baselines. This process identifies candidates for the baseline (such as the validated software system or element), and the information management process controls the information items. For this process, the validation strategy and validation results are typical information items that are baselined.

6.4.12 Operation process

6.4.12.1 Purpose

The purpose of the operation process is to use the system to provide its products or services.

This process establishes requirements for and assigns personnel to operate the system, and monitors the products or services and operator-system performance. To sustain products or services, it identifies and analyses operational anomalies in relation to agreements, stakeholder requirements and organizational constraints.

The operation process typically aims to control or reduce the cost of operations while sustaining an acceptable or improved level of service.

Software systems can have dedicated infrastructure, but are typically operated in distributed environments where other software systems and services, e.g. the internet, are active. The security, availability, and

operational performance of the software SoI are thus a matter of concern within a larger SoS. It can include coordination with pre-existing, concurrent or continuing services delivered by other systems that provide identical or similar services. The operation process includes interface management to agree with interface partners on the exchange of data through communications among software systems and services.

NOTE ISO/IEC 20000-1 details the operation of a service management system, including operation and improvement of managed operational services. The SWEBOK v4 discusses operations in the Software Engineering Operations knowledge area, including operations planning; incident management; operations support; and monitoring, measuring, tracking, and reviewing operations.

6.4.12.2 Outcomes

As a result of the successful performance of the operation process:

- a) Operation constraints that influence system requirements, architecture, or design are identified.
- b) Enabling systems, services, and material needed for operation are available.
- c) Trained, qualified operators are available.
- d) System products or services that meet stakeholder requirements are delivered.
- e) System product performance during operation is monitored.
- f) Support to the stakeholders is provided.

6.4.12.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the operation process.

- a) Prepare for operation. This activity consists of the following tasks:
 - 1) Define an operation strategy, including the following considerations:
 - i) the expected or agreed capacity, availability, response time, and security of services as they are introduced, routinely operated, and withdrawn from service;
 - ii) the human resources strategy, depending on the need to define training and qualification requirements, train or obtain personnel to control and monitor software system operations, administer system access, and support customer service requests and user assistance;
 - iii) the release criteria and schedules of the software system to permit modifications that sustain existing or enhanced services;
 - iv) the approach to implement the operational modes in the operational concept, including normal operations and preparations for, and testing of, envisioned types of contingency operations;
 - v) measures for operation that provide insight into performance levels;
 - vi) the operational and occupational safety strategy for operators and others using or in contact with the software system during operation, accounting for safety regulations;
 - vii) the environmental protection and sustainability strategy for operating the software system.
 - 2) Identify system constraints from operation to be incorporated in changes to the system requirements, architecture, design, implementation, or transition.
 - 3) Identify and plan for the necessary enabling systems or services needed to support operation.

NOTE 1 ISO/IEC 16350 provides guidance for operational aspects.

NOTE 2 This task includes identification of operational requirements and interfaces for the enabling systems. Special modes of the operational software system, for example, a training mode, can sometimes be active alongside or instead of a full operational mode. Enabling systems include monitoring for changes in threats to the software system.

- 4) Obtain or acquire access to the enabling systems or services to be used.

NOTE 3 The validation process is used to objectively confirm that the operation enabling system achieves its intended use for its enabling functions.

- 5) Identify or define training and qualification requirements for personnel needed for software system operation.

NOTE 4 The training and qualification includes awareness of the software system in its operational environment and a defined program of familiarization, with appropriate failure detection and isolation instruction. Operator knowledge, skill and experience requirements guide the personnel selection criteria, and where relevant, their authorization to operate is confirmed. The scope of qualification depends on the SoI and its environment. For example, in some environments regulatory requirements include certification of operators, whereas in others there is no certification requirement.

- 6) Depending on the need for human intervention and control of operations, assign trained, qualified personnel to be operators.

NOTE 5 With due regard for separation of duties, such as for administrative control of system access and investigation of security issues, many modern software products minimize the need for operators as distinct from end users. Operators commonly support enabling systems, such as cloud services, database and system software, security monitors, data storage, and help desk.

- b) Perform operation. This activity consists of the following tasks:

- 1) Use the software system in its intended operational environment.

NOTE 6 Where agreed, continuous service capacity and quality is maintained when the software system replaces an existing system or element that is being retired.

- 2) Apply materials and other resources, as required, to operate the software system and sustain its services.

NOTE 7 Resources include energy sources for hardware, connectivity for software, and human or automated operators.

- 3) Monitor software system operation, including consideration of the following:

- i) managing adherence to the operation strategy (e.g. operational procedures);
- ii) recording and reporting significant events, such as possible breaches of software and data confidentiality and integrity;
- iii) operating the software system in a safe manner and compliant with legislated guidelines e.g. those concerning data management, occupational safety, and environmental protection;
- iv) recording when software system or service performance is not within acceptable parameters.

NOTE 8 Anomalies can be due to the operation strategy, the operation enabling systems, execution of the operation, or incorrect software system definition. The system sometimes exhibits unacceptable performance when system elements implemented in hardware have degraded or exceeded their useful life or the system's operational environment affects the software operation, for example, workload above capacity thresholds, utilization by contending applications, security hacks, or software defects.

- 4) Consistent with the operational strategy, develop and, where feasible, automate operational procedures to minimize the risk of operational anomalies.

NOTE 9 Operational procedures cover handling routine (pre-approved) change requests and service requests, troubleshooting, and incident reporting, especially for security incidents.

- 5) Consistent with the operational strategy, analyse measurements to confirm that:
- i) service performance is within acceptable parameters or agreed service levels for the agreed workload;
 - ii) system and service availability and response times are acceptable;
 - iii) cost of operation is consistent with objectives and constraints;
 - iv) potential improvements are identified and prioritized.

NOTE 10 Operator feedback and suggestions are often useful input for improving software system operational performance. The quality assurance and measurement processes can be applied.

- 6) Perform contingency operations, if necessary.

NOTE 11 Contingencies can involve operating the software system in a degraded mode, performing back-out and restore operations, system shutdown, implementation of work-around procedures to restore operation, or other modes for special conditions. If needed, the operator performs steps necessary to enter contingency operations and possibly power down the system. Contingency operations are performed in accordance with pre-established procedures for such events. Often these procedures are accompanied by a continuity plan.

- c) Manage results of operation. This activity consists of the following tasks:

- 1) Record results of operation and anomalies encountered.

NOTE 12 The project assessment and control and QA processes are used to analyse the incident and problem data to identify the root cause, enable corrective or improvement actions, and to record lessons learned.

- 2) Record operational incidents and problems and track their resolution.

NOTE 13 Changes to the requirements, architecture, design, or software system elements are done using other technical processes.

NOTE 14 If an incident is experienced during operation, an incident management procedure can be applied. The operator records the incident (or is alerted to an automated notification) and performs corrective actions prescribed in validated operating procedures to restore normal operations. Some procedures allow for provision of a temporary workaround solution until root cause analysis can be performed. ISO/IEC/IEEE 23612 provides additional guidance for incident management.

NOTE 15 During software operation, problem management procedures can be applied. The conditions under which the problem occurred are typically documented, consistent with maintaining or restoring operational availability, so that, if possible, the problem can be duplicated in a test environment and the root cause identified. Problem resolution usually involves determining the severity and impact of the problem and whether or when the problem is to be corrected or accepted for a time as a known error.

- 3) Maintain traceability of the operational services and configuration items.

NOTE 16 Bidirectional traceability is maintained between the operational services and the business or mission needs, operational concept, concept of operations, and stakeholder requirements. The operational configuration items are traceable to the released versions and validated through PCA or FCA.

- 4) Provide key artefacts and information items that have been selected for baselines.

NOTE 17 This process identifies candidates for the baseline, and the information management process controls the information items, such as reports on operational service performance. Key artefacts (information items) for the operations process are listed in [Annex B](#).

- d) Support the customer. This activity consists of the following tasks:

- 1) Assist the customers and users to resolve complaints, incidents, problems, and service requests.

NOTE 18 Assistance and consultation includes providing or recommending sources for training, documentation, vulnerability resolution, anti-counterfeit activities, and other services supporting effective use of the software system.

NOTE 19 Customer support can include communication with customers of services, users, and other stakeholders to receive service requests and change requests, resolve complaints, and provide information on the resolution of incidents and problems.

- 2) Record and monitor requests and subsequent support activities.
- 3) Determine the degree to which the delivered software system or services satisfy the needs of the customers and users.

NOTE 20 The results are analysed and the required activities to restore or amend software system or services to provide continued customer satisfaction and software system usability are identified. Wherever possible the benefit of such activity is agreed with stakeholders or their representatives. The customer satisfaction data also serves as an input to the QM process.

6.4.13 Maintenance process

6.4.13.1 Purpose

The purpose of the maintenance process is to sustain the capability of the system to provide a service.

This process monitors the system's capability to deliver products or services; records incidents for analysis; takes corrective, preventive, adaptive, additive, perfective actions and confirms restored capability. The process includes packaging, handling, storage, and transportation for the required replacement system elements. This is often required to support the objectives of the integration and transition processes, including required system and software assurance.

The need for maintenance can arise from multiple causes other than failures, such as changes to interfacing systems or infrastructure, evolving security threats, and technical obsolescence of system elements and enabling systems over the system life cycle.

For software systems, the maintenance process makes corrections, changes, and improvements to deployed software systems and elements. The software systems maintenance approach differs for systems that are freely available, in wide commercial distribution, or operating in a small number of controlled environments.

Often the extension of capability, mid-life upgrade, or evolution of legacy systems becomes a new software system development project that applies the set of processes within an appropriate life cycle. If so, the portfolio management process is the starting point to initiate the work. In other cases, software system maintenance is performed as a continuing series of prioritized work items, possibly on a level of effort basis. A given maintenance item, e.g. problem report or enhanced feature, can lead to a series of lifecycle activities. Multiple software engineering processes may be invoked, resulting in analyses, new or changed requirements, and changes in architecture, design, construction, testing, and integration. Maintenance of software system elements can include hardware, software, and services, such as communication or web services. Maintenance is closely connected with the configuration management process and software asset management and is performed concurrently with the other technical processes.

In level of effort maintenance work, scope may be controlled through time-boxed schedules or resource-limited teams. This approach is particularly appropriate for software maintenance efforts, where the extent or content of corrective or adaptive work is not fully pre-specified.

NOTE ISO/IEC/IEEE 14764 and ISO/IEC 16350 provide additional details. The SWEBOK v4 Software Maintenance knowledge area discusses software maintenance, measurement, techniques, maintenance process and support activities, and tools. The guide also discusses models, techniques and measures that support software reliability.

6.4.13.2 Outcomes

As a result of the successful performance of the maintenance process:

- a) Maintenance and logistics constraints that influence system requirements, architecture, or design are identified.
- b) Enabling systems or services needed for maintenance and logistics are available.
- c) Replacement, repaired, or revised system elements are made available.
- d) The need for required maintenance and logistics actions is reported.
- e) Failure and lifetime data, including associated costs, is determined.

6.4.13.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the maintenance process.

- a) Prepare for maintenance. This activity consists of the following tasks:
 - 1) Define a maintenance strategy, including consideration of the following:
 - i) establishing priorities, typical schedules, and procedures for performing, verifying, distributing, and installing software maintenance changes in conformance with operational availability requirements;
 - ii) establishing techniques and methods for becoming aware of the need for corrective, adaptive, and perfective maintenance;
 - iii) periodic assessment of the design characteristics in case of evolution of the software system and of its architecture;
 - iv) forecasting potential obsolescence of elements and technologies using information on technical changes in related systems;
 - v) establishing priorities and resources to obtain access to the correct versions of the product and product information needed for performing maintenance (e.g. scheduled or phased installation, maintenance patches or software upgrades);
 - vi) measures for maintenance that provide insight into performance levels, effectiveness, and efficiency, including access to historical fault and failure;
 - vii) agreed rights to data and the impact on data in the system during problem resolution and maintenance activity;
 - viii) approach to assure that counterfeit or unauthorized system elements are not introduced into the system;
 - ix) impact of the maintenance change on other software systems elements versus the risk of leaving a reported software anomaly in place;
 - x) the skill and personnel levels required to effect system or software repairs or replacements, fixes, patches, updates, and upgrades, considering legal and regulatory requirements regarding health and safety, security, and the environment.
 - 2) For non-software elements, define a logistics strategy throughout the life cycle, including acquisition and operational considerations: the number and type of replacement elements to be stored, their storage locations and conditions, their anticipated replacement rate, and their storage life and renewal frequency.

NOTE 1 Supportability implications are considered early during concept exploration or development stages. Logistics helps to ensure that the necessary material and resources, in the right quantity and quality, are available at the right place and time throughout deployment and sustainment stages.

- 3) Identify constraints from maintenance to be incorporated in the system requirements, architecture, or design.

NOTE 2 These often result from the need to a) re-use existing maintenance and verification enabling systems; b) re-use existing holdings of replaceable system element and accommodate re-supply limitations; c) conduct maintenance in specific locations or environments. For example, software architectures and designs that emphasize encapsulation, modularity, and scalability can be simpler to maintain. Requirements to document the system design and construction can reduce the effort needed to reverse engineer systems and elements when maintenance is needed. The system architecture and design reflect the need to roll back, back up, and recover data during problem resolution. Functions to make the system available for remote diagnostics and maintenance can be incorporated in the architecture and design.

- 4) Identify trade-offs such that the system and associated maintenance and logistics activities result in a solution that is affordable, operable, supportable, and sustainable.

NOTE 3 The system analysis and decision management processes are used to perform the assessments and trade-off decisions.

- 5) Identify and plan for the necessary enabling systems or services needed to support maintenance.

NOTE 4 This task includes identification of requirements and interfaces for the enabling systems. The selection of enabling systems for maintenance often reflects the need to re-use existing or equivalent design, development, and configuration management infrastructure as during the initial system implementation.

- 6) Obtain or acquire access to the enabling systems or services to be used.

NOTE 5 The validation process is used to objectively confirm that the maintenance enabling system achieves its intended use for its enabling functions.

- b) Perform maintenance. This activity consists of the following tasks:

- 1) Review stakeholder requirements, complaints, events, incident reports, and problem reports to identify corrective, adaptive, perfective, and preventive maintenance needs.

NOTE 6 For software systems with iterative life cycles, changing requirements can be considered as the source for adaptive and perfective maintenance activities. For software maintenance, this process makes corrections, changes, and improvements to deployed software, as well as patching and updates to maintain system security.

- 2) Analyse the impact of maintenance changes on data structures, data, and related software functions, user documentation, and interfaces.

NOTE 7 Reviews and analyses often include factors such as the category of maintenance action; size of modification; cost involved; time to modify; and impacts on performance, safety, or security.

- 3) Upon encountering unexpected faults that cause a software system failure, restore the system to operational status.

NOTE 8 Restoration to full or degraded operational status can often be accomplished through a rollback, workaround or by identifying and correcting the cause of the fault. If full restoration is delayed or not possible, the system is restored to a degraded mode, consistent with the contingency planning. If possible, the fault is replicated using a distinct environment similar to the operational environment and the root causes of the fault are identified. The configuration management process, especially release management activities, is invoked to control scheduled and emergency changes to the system.

- 4) Implement the procedures for correction of defects and errors, or for replacement or upgrade of system elements.

NOTE 9 Correction of defects and errors uses problem resolution, handled through the QA and project assessment and control processes.

NOTE 10 Typically, regression testing is performed to verify that the maintenance change has not introduced other issues, i.e. complete and correct implementation of the new and modified requirements without effect on the performance of the original, unmodified requirements. The transition process can be applied for deployment of major maintenance changes; minor fixes are typically handled as part of the maintenance process. Actions are recorded to facilitate future maintenance and problem resolution, and for logistics analyses of degradable system elements.

NOTE 11 System and data recovery procedures and maintenance information are often made available on media that is usable at the point of performing maintenance.

- 5) Perform preventive maintenance by replacing, patching, augmenting, or upgrading software system elements, to improve the performance of a software system that is projected to reach unacceptable service levels, for example, lack of capacity due to increases in demand or stored data, or to avoid unacceptable operating conditions, for example, running with outdated security software.
- 6) Identify when adaptive or perfective maintenance is required.

NOTE 12 Adaptive and perfective maintenance actions usually involve change to the system requirements, architecture and design. A new project can be started to modify the existing software system.

c) Perform logistics support. This activity consists of the following tasks:

NOTE 13 The logistics activities enable the software system to sustain operational readiness. The activities include provisions for staffing, supply support, support equipment, technical data needs (information for users) and agreed data rights, training support, communications, equipment/computing resource support, and facilities.

- 1) Obtain resources to support the software system through its life cycle or the project's life (acquisition logistics).

NOTE 14 Acquisition logistics considerations are included in the agreement resulting from the agreement processes. This includes performing analysis to identify cost-effective changes to the initial design of the system for supportability and ease of maintenance, as well as arrangements for distributing software fixes and upgrades during utilization/deployment. These decisions are often constrained by availability requirements and impact the supply chain management.

- 2) Monitor the quality and availability of replacement elements and enabling systems, their delivery mechanisms, and their continued integrity during storage.

NOTE 15 Operational logistics involves the concurrent adjustment of both the SoI and enabling systems throughout the operational life to help ensure effective and efficient delivery of software functions. It also includes availability of skilled resources. For example, reliable enabling systems are available with the capacity to read software stored on previous media formats, or to migrate backup files to current media and currently maintained enabling systems.

- 3) Implement mechanisms for software system or element electronic or physical distribution, including packaging, handling, storage and communications or transportation needed for items during the life cycle.

NOTE 16 Software distribution and installation is typically automated. Software packages commonly include software license terms, including data rights, and elements for software asset management. Logistics planning for other systems elements is often required to support the objectives of the integration and transition processes.

NOTE 17 Consider the need to store spare elements or backup copies of software onsite or in additional locations, to maintain software system capabilities, as required (perhaps at a reduced level for contingency operations).

- 4) Confirm that logistics actions to fulfil software system or element supportability requirements or achieve operational readiness are planned and implemented.

NOTE 18 These logistic actions can include staffing, supply support, support equipment, technical data needs (information for users), training support, communications, equipment/computing resource support, and facilities.

d) Manage results of maintenance and logistics. This activity consists of the following tasks:

- 1) Record incidents and problems, including their resolutions, and significant maintenance and logistics results.

NOTE 19 Anomalies can be due to the maintenance strategy, the maintenance enabling systems, execution of the maintenance and logistics, or incorrect system definition, as well as misidentification of root causes. Incident and problem reporting, including resulting actions taken, is tracked through the QA process. The project assessment and control and QA processes are used to enable corrective or improvement actions, and record lessons learned. This activity can include changes to logistics or software distribution procedures. Changes to the software system requirements, architecture, or design are done within other technical processes.

- 2) Identify and report trends of incidents, problems, and maintenance and logistics actions.

NOTE 20 Trend data and problem resolution reports are used to inform operations and maintenance personnel, customers, and other stakeholders and projects that are creating or utilizing similar system entities.

NOTE 21 Bidirectional traceability is maintained between the recorded maintenance actions and the software system elements and life cycle artefacts. Changes in software asset management, such as assignment of software licenses to replacement systems, are recorded.

- 3) Provide key artefacts and information items that have been selected for baselines.

NOTE 22 The configuration management process is used to establish and maintain configuration items and baselines and to track licenses and data rights. This process identifies candidates for the baseline, and the information management process controls the information items, such as maintenance procedures.

- 4) Monitor and measure customer satisfaction with system and maintenance support.

NOTE 23 ISO 10004 contains guidelines for monitoring and measuring customer satisfaction. Customer satisfaction data is used in the QM process.

6.4.14 Disposal process

6.4.14.1 Purpose

The purpose of the disposal process is to end the existence of a system element or system for a specified intended use, appropriately handle replaced or retired elements, appropriately handle waste products, and properly attend to identified critical disposal needs (e.g. per an agreement, per organizational policy, or for environmental, legal, safety, or security aspects).

This process deactivates, disassembles, and removes the system or any of its elements from the specific use. It addresses any waste products, consigning them to a final condition and returning the environment to its original or an acceptable condition. The waste products can be in-process resulting during any life cycle stage, e.g. waste materials during fabrication. This process destroys, stores, or reclaims system elements and waste products in an environmentally sound manner, in accordance with legislation, agreements, organizational constraints and stakeholder requirements. Disposal includes preventing expired, non-reusable, or inadequate elements from getting back into the supply chain. Where required, it maintains records in order that the health of operators and users, and the safety of the environment, can be monitored. When part of the system will continue to be in use in a modified form, the Disposal process helps ensure the proper handling of the portion being retired.

Disposal of software systems encompasses the termination of services and disposal of software elements, stored data, media and firmware, information items, and associated hardware elements that should be reused or transitioned to another system. The disposal process is intended to be applicable in any stage of a software systems life cycle. For software, the disposal process applies throughout the life cycle to source code or executable copies of the software, personally identifiable or controlled data used in the software system, and associated information items, retained under centralized configuration control or distributed for use, for example, disposing of prototypes in early life cycle stages, and decommissioning elements

replaced from modifications during utilization/deployment and support stages. When the SoI is being modified for technology or capability upgrades, only the impacted elements are deactivated and removed.

NOTE 1 The business or mission analysis process and the decision management process are typically applied to address the impact on stakeholders of system disposal and potential new system capabilities.

NOTE 2 The disposal process is intended to be applicable throughout the life cycle of the system, including disposing prototypes during the concept and development stages, dealing with waste during the production stage, decommissioning elements from modifications during the utilization and support stages, and ultimately removing the system from service during the retirement stage.

NOTE 3 During the retirement stage, the disposal process can be used to dispose of the system at the end of its life or, if the system is still useful, to manage the transition from its current life cycle to a new life cycle, whether in a different part of the organization or to a different organization.

6.4.14.2 Outcomes

As a result of the successful performance of the disposal process:

- a) Disposal constraints are provided as inputs to requirements, architecture, design, and implementation.
- b) Enabling systems or services needed for disposal are available.
- c) The system elements or waste products are destroyed, stored, reclaimed, or recycled in accordance with safety and security requirements.
- d) The environment is returned to its original or an agreed state.
- e) Records of disposal actions and analysis are available.

6.4.14.3 Activities and tasks

The project shall implement the following activities and tasks in accordance with applicable organization policies and procedures with respect to the disposal process.

- a) Prepare for disposal. This activity consists of the following tasks:
 - 1) Define a disposal strategy for the software system, to include each system element and to identify and address critical disposal needs, including the following considerations:
 - i) permanent termination of the system's functions and delivery of services, for example, physical destruction of data storage devices, or transition of the software system elements for future reuse in modified or adapted form;
 - ii) identification of ownership and responsibility for retention or destruction of data and intellectual property in the software system;
 - iii) transformation of the product into, or retention in a socially and physically acceptable state, thereby avoiding subsequent adverse effects on stakeholders, society and the environment;
 - iv) the health, safety, security and privacy concerns applicable to disposal actions and to the long-term condition of resulting physical material and information;
 - v) notification to relevant stakeholders of significant disposal activities, for example, retirement or replacement of a system, software products or services, retirement schedule, or replacement options;
 - vi) identification of schedules, activities, responsibilities, and resources for disposal activities.
 - 2) Identify constraints on disposal for the system requirements, architecture and design characteristics, or implementation techniques.

NOTE 1 Constraints can include access to and availability of archives or long-term storage locations and available skilled resources for system deactivation and communication with stakeholders and interface partners.

- 3) Identify and plan for the necessary enabling systems or services needed to support disposal.

NOTE 2 This task includes identification of requirements and interfaces for the enabling systems.

- 4) Obtain or acquire access to the enabling systems or services to be used.

NOTE 3 The validation process is used to objectively confirm that the disposal enabling system achieves its intended use for its enabling functions.

- 5) Specify containment facilities, storage locations, inspection criteria and storage periods, if the software system or data is to be stored, consistent with security and environmental considerations.
- 6) Define preventive methods to preclude disposed elements and materials that should not be repurposed, reclaimed, or reused from re-entering the supply chain.

- b) Perform disposal. This activity consists of the following tasks:

- 1) Deactivate the software system or element to prepare it for removal.

NOTE 4 Interfaces to other systems are considered, in accordance with special procedures or instructions, and relevant health, safety, security and privacy constraints.

- 2) Remove the software system, its elements, its data, and non-reusable material from use or production for appropriate disposition.

NOTE 5 Disposition can include reuse, archiving, recycling, reconditioning, overhaul, or destruction. The disposition is conducted in accordance with relevant safety, security, privacy, and environmental standards and with regard for laws and regulations. Elements of the software system that have useful life remaining, either in their current condition or following modification, are transferred to other systems-of-interest or organizations. Where appropriate, system elements can be refurbished to extend their useful life.

- 3) Withdraw impacted operating staff from the software system or system element and record relevant operating knowledge.

NOTE 6 With the cessation of operations, operators can be reallocated, redeployed, or retired. The knowledge management process can be applied to preserve operators' knowledge and skills.

- 4) Reuse, recycle, recondition, overhaul, archive, or destroy designated software system elements.

NOTE 7 System elements and their parts that are not intended for reuse are disposed of, so that they do not get back into the supply chain.

- 5) Conduct destruction of the system elements, as necessary, to reduce the amount of waste treatment or to make the waste easier to handle.

NOTE 8 When the element is non-maintainable or non-recyclable, it is necessary to prevent the element from getting back into the supply chain, for example, complete erasure of all software from all system storage media and removal of license keys, data, and interfaces. This activity includes obtaining the destruction services to melt, crush, incinerate, demolish, or eradicate the system or its elements as necessary.

- c) Finalize the disposal. This activity consists of the following tasks:

- 1) Confirm that detrimental health, safety, security, and environmental conditions following disposal have been identified and treated.
- 2) Return the environment to its original state or to a state that is specified by agreement.
- 3) Archive information gathered through the lifetime of the product to permit audits and reviews in the event of long-term hazards to health, safety, security, and the environment, and to permit future software system creators and users to build a knowledge base from experience.

Annex A **(normative)**

Tailoring process

A.1 General

This annex provides requirements for the tailoring of this document.

Tailoring is not a requirement for full conformance to this document. In fact, tailoring is not permitted if a claim of “full conformance” is to be made. If a claim of “tailored conformance” is made, then this process shall be applied to perform the tailoring.

NOTE 1 Additional guidance for tailoring can be found in ISO/IEC/IEEE 24748-1 and ISO/IEC/IEEE 24748-2, which provide guidelines on the application of life cycle processes.

NOTE 2 Irrespective of tailoring, organizations and projects are always permitted to implement processes that achieve additional outcomes or implement additional activities and tasks beyond those required for conformance to this document.

A.2 Tailoring process

A.2.1 Purpose

The purpose of the tailoring process is to adapt the processes of this document to satisfy particular circumstances or factors that:

- a) Surround an organization that is employing this document in an agreement.
- b) Influence a project that is required to meet an agreement in which this document is referenced.
- c) Reflect the needs of an organization to supply products or services.

A.2.2 Outcomes

As a result of the successful performance of the tailoring process, modified or new life cycle processes are defined to achieve the purposes and outcomes of a life cycle model.

A.2.3 Activities and tasks

If the life cycle processes defined in this document are tailored, then the organization or project shall implement the following tasks in accordance with applicable policies and procedures with respect to the tailoring process, as required.

- a) Identify and record the circumstances that influence tailoring. These influences may include the following:
 - 1) stability of, and variety in, operational environments;
 - 2) risks, commercial or performance, to the concern of stakeholders;
 - 3) novelty, size, and complexity;
 - 4) starting date and duration of utilization;
 - 5) integrity issues such as safety, security, privacy, accuracy, usability, and availability;

- 6) emerging technology opportunities;
 - 7) profile of budget and organizational resources available;
 - 8) availability of the services of enabling systems;
 - 9) roles, responsibilities, accountabilities, and authorities in the overall life cycle of the system;
 - 10) the need to conform to other standards.
- b) In the case of properties critical to the system, take due account of the life cycle structures recommended or mandated by standards relevant to the dimension of the criticality.
- c) Obtain input from parties affected by the tailoring decisions:
- 1) the system stakeholders;
 - 2) the interested parties to an agreement made by the organization;
 - 3) the contributing organizational functions.
- d) Make tailoring decisions in accordance with the decision management process to achieve the purposes and outcomes of the selected life cycle model.

NOTE 1 Organizations establish their life cycle models as a part of the life cycle model management process. sometimes an organization tailors processes of this document to achieve the purposes and outcomes of the stages of their life cycle model.

NOTE 2 Projects that are adapting organizational life cycle models or directly applying this document, it is can tailor processes of this document to achieve the purposes and outcomes of the stages of a suitable life cycle model.

- e) Select the life cycle processes to be tailored and indicate that selected outcomes, activities, or tasks are not applicable.

NOTE 3 An organization or project sometimes encounter a situation where there is the desire to modify a provision of this document. Because of unanticipated consequences on other processes, outcomes, activities or tasks, such modifications are generally avoided. If necessary, modification is performed by indicating that the provision is not applicable (making the appropriate claim of tailored conformance) and, with careful consideration of consequences, implementing a process that achieves additional outcomes or performs additional activities and tasks beyond those of the tailored standard.

Annex B (informative)

Examples of process information items

[Table B.1](#) provides a possible set of work products, including artefacts, records, information items, and data stores associated with each process. This list is not all-inclusive: for each process, an organization may decide to develop a policy, plan, procedures, reports, and records, to demonstrate the outcomes or perform the activities and tasks. Where less intensive documentation is considered sufficient, information items can be combined. Also, the organizational policies and procedures can be applied or tailored for each process and project. Typical item titles are shown, including common examples of alternate titles in parentheses.

An artefact is any kind of work product and includes information items. Information items represent an identifiable body of information for human use to communicate to stakeholders. Entries without an item type represent data that can be stored or presented in multiple information items or models.

NOTE 1 See ISO/IEC/IEEE 15289 for guidance on content and management of information items.

Artefacts, records, record stores, and information items are usually initiated in one process and revised, enhanced, or completed in other processes. For convenience, they are listed once in this table, in a process where they are commonly initiated. When the software system artefacts and information are transformed or elaborated by another process, traceability is maintained and a traceability mapping can be produced. For example, traceability can be maintained between organizational and project processes, and between requirements, architecture and design elements, and verification cases.

NOTE 2 Every process has a strategy, whether or not it is documented. When it is documented, then it is part of plan, policy, or other information item.

Table B.1 — Sample information items by process

Process group	Process	Typical item title	Item type
6.1 Agreement processes			
6.1.1	Acquisition process		
		Acquisition plan	info
		Request for supply (e.g. request for proposal, request for tender)	info
		Agreement change request	info
		Agreement (e.g. contract)	info
		Agreement change management procedure	info
		Delivery acceptance report	info
		Acquisition records and reports (acquisition approach, agreement changes, supplier selection report, supply acceptance, delivery acceptance)	info, record
6.1.2	Supply process		
		Supply response (e.g. proposal, tender)	info
		Agreement change request	info
		Agreement change management procedure	info
		Supply delivery records (for system, software, product, or service)	record
		Agreement (contract)	Info
		Supply records and reports (supply approach, system and element delivery records, change requests)	record, info
6.2 Organizational project-enabling processes			

Table B.1 (continued)

Process group	Process	Typical item title	Item type
6.2.1	Life cycle model management process		
		Organizational policies	info
		Life cycle processes	artefact
		Life cycle process description	info
		Life cycle model	artefact
		Organizational procedures (process management procedures)	info
		Process assessment report	info
		Process improvement plan	info
		Process improvement report	info
6.2.2	Infrastructure Management process		
		Infrastructure requirements	artefact
		Infrastructure element	artefact
		Infrastructure plan	info
		Infrastructure description (element specification)	info
		Infrastructure change request (improvement request)	info
		Infrastructure records and reports	
6.2.3	Portfolio management process		
		Project portfolio	store
		Project budget (resource allocation)	artefact
		Project authorization (charter)	info
		Portfolio management records and reports (portfolio analysis, evaluations, project direction)	record, info
6.2.4	Human resource management process		
		Skill needs report	info
		Training plan (skills development plan)	info
		Skill development assets (training materials)	info
		Skill development records (skill inventory, training record)	record
		Staff assignment records	record
6.2.5	Quality management process		
		Quality management plan (policies, objectives)	info
		Quality management procedures	info
		Quality assurance evaluation results	record
		Corrective and preventive action report (problem management report)	info
		Quality management records and reports (criteria and methods, assessment results, corrective and preventive action report)	info
6.2.6	Knowledge management process		
		Knowledge management taxonomy	artefact
		Knowledge management procedures (change control, access control)	
		Knowledge asset records	record
		Knowledge assets	artefact
		Knowledge asset reports	info
6.3 Technical management processes			
6.3.1	Project planning process		

Table B.1 (continued)

Process group	Process	Typical item title	Item type
		Project plan (e.g. project technical management plan, systems or software engineering management plan, software development plan) (management, technical, and process specific)	info
		Work breakdown structure	artefact
		Resource request	info
		Project schedule	artefact
		Project infrastructure and services requirements	artefact
		Project life cycle model (tailored)	artefact
		Project planning records and reports (objectives, constraints, schedules, budgets)	record, info
6.3.2	Project assessment and control process		
		Project status report (progress report, assessment report)	info
		Review minutes	info
		Authorization to proceed to next milestone	info
		Change requests	info
		Project assessment and control records and reports (project performance data, project control requests, meeting minutes, reviews, status, variances, corrective actions)	record, info
6.3.3	Decision management process		
		Decision request	info
		Decision records	record
		Decision reports	info
		Decision register (assumptions, rationale, resolution)	store
6.3.4	Risk Management process		
		Risk management plan	info
		Risk profile	record
		Risk action request	info
		Risk register	store
		Risk records and reports (profiles)	record
6.3.5	Configuration management process		
		Configuration management plan	info
		Configuration management procedures	info
		Configuration management records (release approvals)	record
		Configuration baseline (SBOM)	artefact
		CM change/variance request (release request, variance request)	info
		Configuration status report	info
		Configuration evaluation report	info
6.3.6	Information management process		
		Information item archive (content management system, document management system)	store
		Information management procedures	info
		Information management report	info
		Information management records	record
6.3.7	Measurement process		
		Measurement analysis results and recommendations	info

Table B.1 (continued)

Process group	Process	Typical item title	Item type
		Measurement data	record
		Measurement procedures	info
		Measurement information needs	record
		Measurement report	info
6.3.8	Quality assurance process		
		Quality assurance procedures	info
		Quality assurance evaluation report	info
		Quality assurance records (criteria and methods, evaluations, corrective actions)	record
		Incident and problem records	record
		Incident and problem reports	info
6.4 Technical processes			
		Requirements for enabling systems (records)	record
		Traceability mappings	artefact
6.4.1	Business or mission analysis process		
		Preliminary life cycle concept, System life cycle concepts (e.g. operational, acquisition, support, deployment, security)	artefact
		Solution alternative classes	artefact
		Problem or opportunity statement	info
		Solution alternative analysis report (with preliminary operational concepts)	info
6.4.2	Stakeholder needs and requirements definition process		
		Operational concept	artefact
		Stakeholder needs assessment (critical performance needs, product need assessment)	info
		Stakeholder requirements	artefact
		Stakeholder requirements specification	info
		Stakeholder requirements report	info
		Critical performance measures	artefact
6.4.3	System requirements definition process		
		System or element description	info
		System requirements	artefact
		Critical performance measures	artefact
		System requirements specification	info
		Requirements change request	info
6.4.4	System architecture definition process		
		Architecture viewpoints	artefact
		Architecture views and models (including system elements)	artefact
		Interface definition (initial)	artefact
		Architecture reports (description, assessments, rationales)	Info
6.4.5	Design definition process		
		Design alternatives and characteristics (design baseline)	artefact
		Design artefacts report (Design description, evaluation)	info
		Interface specification	info
		Interface definition (API)	artefact

Table B.1 (continued)

Process group	Process	Typical item title	Item type
6.4.6	System analysis process		
		System analysis report (assumptions, evaluations, results, recommendations)	info
		System analysis models	artefact
6.4.7	Implementation process		
		Software system element	artefact
		Implementation procedures	info
		Implementation records (constraints, unit test results)	record
6.4.8	Integration process		
		Interface control description	info
		Integration, operation, and test procedures	info
		Integrated system and system elements (software library, API)	artefact
		Integration records and reports (approach, criteria, results, discrepancies)	record
6.4.9	Verification process		
		Verified system	artefact
		Verification plan (test plan)	info
		Verification procedures (test procedures)	info
		Verification records	record
		Verification report (approach, criteria, results, discrepancies)	info
6.4.10	Transition process		
		Prepared site for operations target environment (operational site)	artefact
		Transition plan (release approach, contingency/backout approach, change management plan)	info
		Transition procedures	info
		Trained operators and users	artefact
		System/software release report	info
		Installed, transitioned system or element	artefact
		Transition records and reports (approach, constraints, discrepancies, installation,)	record
6.4.11	Validation process		
		Validated system	artefact
		Validation plan (acceptance plan)	
		Validation procedures	info
		Validation records	record
		Validation report (approach, constraints, discrepancies)	info
6.4.12	Operation process		
		Operational system	artefact
		Continuity plan	info
		Operational procedures (User documentation) (information for users, system monitoring, continuity procedures, security procedures)	info
		Operation records	record
		Incident, event, or problem report	info
		Customer support request (service request)	info
		Customer support records	record

Table B.1 (continued)

Process group	Process	Typical item title	Item type
		Operation report (performance, discrepancies, monitoring, evaluation, customer satisfaction)	info
6.4.13	Maintenance process		
		Replacement or repaired system or system element	artefact
		Maintenance plan (logistics plan)	info
		Maintenance procedures (logistics procedures, sustainability procedures)	info
		Maintenance (logistics) records	record
		Maintenance requests (supply requests)	record
		Maintenance (logistics) report (approach, constraints, maintenance/supply requests, discrepancies)	info
6.4.14	Disposal process		
		Disposal records	record
		Archive report	info
		Disposed/archived system or system elements (and data)	artefact
		Restored system environment	artefact

Annex C

(informative)

Process reference model for assessment purposes

C.1 General

Some users of this document may choose to assess the implemented processes in accordance with ISO/IEC 33004. This annex provides a process reference model (PRM) suitable for use in conjunction with this standard.

The PRM is composed of the processes in the body of this document, including the name, statement of purpose, and statement of outcomes for each process. [Clause C.3](#) identifies the processes in the process reference model and the clauses in which they are defined.

C.2 Conformance with ISO/IEC 33004

ISO/IEC 33004:2015, 5.3 places requirements on process reference models suitable for assessment by that document. [Table C.1](#) quotes the requirements for process reference models and describes how these are met by this document.

Table C.1 — Implementation of requirements for process reference models in ISO/IEC 33004

Process reference model requirements (see ISO/IEC 33004:2015, 5.3.1)	ISO/IEC/IEEE 12207 implementation
a) Declaration of domain	Clause 1
b) Description of relationship between the process reference model and context of use.	Clause 5
c) Description of processes	Clause 6 in the description of each process.
d) Description of relationship between processes	5.6
Process reference model community of interest and consensus (see ISO/IEC 33004:2015, 5.3.2)	
a) Characterization of community of interest	The relevant community of interest is the users of this document and ISO/IEC/IEEE 15288.
b) Achievement of consensus	Both this document and ISO/IEC/IEEE 15288 are International Standards satisfying the consensus requirements of ISO, IEC, and IEEE.
c) If no action taken to achieve consensus, action documented	Not applicable
Unique process descriptions (see ISO/IEC 33004:2015, 5.3.3)	The process descriptions are unique. The process descriptions are per the requirements of ISO/IEC/IEEE 24774.
Process descriptions (see ISO/IEC 33004:2015, 5.4)	The process descriptions are per the requirements of ISO/IEC/IEEE 24774.

Table C.1 (continued)

Process reference model requirements (see ISO/IEC 33004:2015, 5.3.1)	ISO/IEC/IEEE 12207 implementation
a) processes described with its purpose and outcomes	Clause 6
b) outcomes are necessary and sufficient to achieve the process purpose	Clause 6 and output artefacts in Annex B Process outcomes include: — production of an artefact; — a significant change of state; — meeting of specified constraints, e.g. requirements, goals, and objectives.
c) process quality characteristics are at the basic level of process measurement frameworks conforming to ISO/IEC 33003	Some outcomes can be interpreted as contributing to levels of capability above level 1. However, conforming implementation of the relevant processes does not require achievement of these higher levels of capability.

C.3 The process reference model

The PRM is composed of the statement of purpose and outcomes of each of the processes included in [Clause 6](#). The PRM for the software life cycle is composed of the set of processes shown in [Figures 4](#) and [5](#).

Annex D (informative)

Model-based systems and software engineering (MBSSE)

D.1 MBSSE and description

MBSSE (model-based systems and software engineering) is the formalised application of modelling to support systems and software engineering throughout the whole life cycle of an SoI. MBSE (model-based systems engineering) is the formalised application of modelling to support systems engineering throughout the whole lifecycle of an SoI.

Typically, there is a need to address engineering of software systems in a digital environment. Consequently, the scope and content of this annex covers both MBSE and MBSSE, with a primary focus on approaches to the use of MBSSE.

NOTE ISO/IEC/IEEE 24641 specifies a reference framework for systems and software engineering with a model-based approach.

D.2 Implementation of life cycle processes in an MBSSE approach

This document provides a common process reference model for the engineering of software systems. This document provides a comprehensive set of processes from which an organization can develop life cycle models appropriate to its products; services; and the framework for development, assessment, support, and improvement of the life cycle processes.

Execution of these life cycle processes can be greatly facilitated through the adoption of MBSSE. In this approach, the software engineering activities rely on evolving models that serve as the “main or major source of knowledge” about the SoI and its life cycle stages. The set of life cycle processes included in this document and their relationships to the supporting reference framework are included in ISO/IEC/IEEE 24641:2023, Annex A and Annex E. MBSSE adoption and application often includes a set of milestones of capabilities and maturity enhancement.

D.3 MBSSE as a practice

MBSSE utilises modelling environments to model and express engineering data in the execution of the processes detailed in the software engineering framework, which is elicited in the processes and relationships covered in [5.7](#) and [5.8](#). In addition to integrating concepts, requirements, architecture and design data, the models also support measurement, project assessment and control, system analysis, decision management, verification, and validation processes.

Models should follow some basic principles:

- a) Every model should have a defined purpose.
- b) Every model should identify its subject, and context.
- c) Every model should be developed for an identified audience.
- d) Every model should have a documented set of conventions to aid its audience in understanding.

This modelling approach results in the following.

- A single source of truth for system definition that accelerates data maturity, integration, and utility. This single source of truth allows more aggressive development schedules to be pursued. For example, as

stated in [5.2](#), a system element in one context can be an SoI in another. In MBSSE, a model of the SoI can be effectively integrated with models of enabling software systems or interoperating systems to provide a model of a complete solution.

- The ability to support multiple stakeholder viewpoints and therefore, multiple views. As described in [5.2](#), an SoI may need to be represented as a hierarchical decomposition or a network. By using tailored model element relationships in MBSSE, both are achievable using the same dataset comprising the SoI.
- The ability to rapidly and thoroughly analyse an integrated model-based dataset by query helps ensure model completeness and accuracy, and greatly reduces the time required and errors introduced through manual data verification. This feature greatly facilitates and provides the formality and rigor described in [6.4.6](#).

D.4 Benefits of executing system life cycle processes in an MBSSE environment

The use of MBSSE does accelerate the achievement of life cycle process outcomes. One key value of MBSSE comes to fruition during verification and validation (see [6.4.9](#) and [6.4.11](#)). The MBSSE dataset also provides valuable information for service support of the software system in operation. A list of benefits provided by incorporation of MBSSE support activities in system life cycle processes is provided in [Table D.1](#).

Table D.1 — Life cycle processes supported by an MBSSE environment

MBSSE benefit	MBSSE support activity description	Reference in this document
More thorough understanding of the concept and system definition	Delivering software engineering data within a model enhances specificity, clarity, and accurate interpretation of the data.	6.4.1 to 6.4.5
Better ability to perform trade-offs	Through data query and analyses, modelling allows comparison of solution alternatives much more expeditiously, comprehensively, and accurately than can be executed through manual review of data. Solution alternatives can be compared by query and analyses against a common baseline, e.g. a set of requirements, functions, and key performance parameters.	6.3.3 6.4.6
More effective and efficient measurement	Modelling allows more expeditious, comprehensive, and accurate collection of data to support measures and perform analyses of proposed SoI and system element solutions than can be executed through manual reviews of data. Proposed solutions can be analysed by query to determine to what extent solutions achieve targeted values of technical measures (e.g. measures of effectiveness, measures of performance, technical performance measures).	6.3.7
Better ability to perform impact analysis	Relationships in the integrated system definition dataset allows, by model query, rapid determination of the impact of changes to system definition, including impacts within and between system elements, at the interfaces, across the overall SoI, and possibly between the SoI and interoperating/enabling systems.	6.4.6
Help ensure consistency and completeness of requirements	Model analysis can, for instance, rapidly and thoroughly determine whether requirements, architectures, and design sufficiently specify system elements within a baseline; whether the system and system elements satisfy allocated requirements and functionality; and whether interfaces are sufficiently defined and reconciled.	6.4.1 6.4.2 6.4.3
Help ensure consistency and complete-ness of architecture and design	Through the establishment of data relationships, modelling allows more effective integration of architecture and design data. Querying the integrated data allows rapid identification of missing data elements and model construction errors.	6.4.4 6.4.5 6.4.6

Table D.1 (continued)

MBSSE benefit	MBSSE support activity description	Reference in this document
Enable reuse and adaptability	Delivering systems engineering data as models that enhance specificity, clarity, and accurate interpretation of the data and facilitates their reuse. Knowledge assets from model-based system element representations, for example, models, views, architecture, and design patterns, can be exploited across the organization. The models better facilitate capturing the integrated technical information for the reusable patterns. Additionally, the models can be used to explore alternatives and impacts, enabling the identification of alternatives that increase the adaptability.	6.2.6 6.4.4 6.4.5 6.4.6
Surrogate for verification and validation activities	Model analysis can detect requirements, architecture, and design errors through model verification and validation, in advance of producing and integrating physical system elements. Validated models allow verification and validation activities to be done via simulated and modelled representations when it is not feasible to use the actual system.	6.3.8 6.4.9 6.4.11
Reduction of risk	Models can be analysed to identify technical and programmatic risks that would not otherwise have been apparent. Additionally, system elements (requirements, architecture, design, interfaces) that are developed and integrated within the models can be leveraged to affect risk treatment decisions.	6.3.4
Early detection of interface and integration issues	Relationships in the integrated system definition dataset allows, by model query, rapid determination of missing, improperly defined and incompatible interfaces.	6.4.8 6.4.4 6.4.5
Better understanding and management of dependencies	Delivering systems engineering data as models enhances specificity, clarity, and accurate interpretation of the data so that life cycle process and technical dependencies may be assessed.	6.3.1 6.3.2

D.5 Model kinds useful in MBSSE

D.5.1 General

Models are used in MBSSE for software systems vary in their rigor and purpose:

- Prescriptive models express either the expectations in terms of needs or requirements on the acquirer side, or committed requirements on the supplier side, or as constraints in a life cycle stage. They are used to reduce risks through analysis, validation, and optimization of the system architecture models and are mathematically based to support mathematical analyses or simulation.
- Descriptive models (sometimes referred to as constructive models or system architecture models) capture the system's behaviour, structure, constraints, interfaces, requirements, and serve as a repository to define product entities and their relationships.
- Cognitive models allow a simplified representation aimed at modelling psychological or intellectual processes. They can be enriched with representation of the virtual reality to represent human factors. Cognitive models replicate human reasoning and problem-solving for predictive purposes.

Models can be further classified according to the domain that they represent, called "domain specific models". For example, models may be related to the following:

- the properties of the system, such as performance, reliability, mass property or power;
- technology implementation (e.g. electrical, mechanical, software, human);
- application domain (e.g. automotive, aerospace, medical, defence).

The level of abstraction (conceptual, logical, technical, and physical models) and formalisation vary according to the intended use of a model. Depending on the intent, the following models can be used.

- Formal models can provide verifiable evidence or directly generate system elements (i.e. hardware or software elements).
- Semi-formal models can aid specification for further description or development activities.
- Informal models are utilised when complete formal specification is impractical and can be based on heuristics or expert judgement. The modelling effort is often limited by partial knowledge of the solution and by availability of time and resources.

Using the system architecture definition process (6.4.4), models can be elaborated per stakeholder perspectives or viewpoints. They can also be elaborated to reflect aspects of the entity of interest (see ISO/IEC/IEEE 42010). In addition, practitioners can use a tool-supported modelling framework to organize models according to levels of abstraction and domain of coverage. By using this type of framework, various views of these models can be produced and packaged for stakeholder use. This approach is cost-effective, because the same model or meta-model can often be used to reflect a multitude of views (both in form and level of detail, according to stakeholder viewpoints),

D.5.2 Typical uses of models for software systems

D.5.2.1 Functional model

In a functional model, a software system is defined a set of functions, which transform inputs into outputs so that the system can achieve its mission or purpose. Every system function is associated with an interaction between the system and its environment, or between subsystems. These functions capture how the system is expected to behave when used as intended.

Typically, there are multiple ways to model and decompose the functions that contribute to the definition of multiple candidate architectures or designs. Functions and input/output flows can be used to further analyse performance and non-functional properties, system constraints, and design decisions. In a functional model, functions are associated with software system elements to determine if each element has been sufficiently specified to build or buy it.

D.5.2.2 Data model

A data model (ontological or information model) represents the data elements and their relationships and properties (attributes) that the software system handles. Logical data models use schema to reflect structural relationships between data entities which can be implemented in databases. Data models reflect different types of data (text, graphics, geographic data, images, general objects) and their use in the system functions (frequency of change, data volume, use in searching) as well as the logical relationships among data elements. Data models are applied in the development of interfaces and software services, data analysis, and data reporting. Physical data models reflect the schema for storage and retrieval of data records.

D.5.2.3 Behavioural model

A behavioural model (dynamic model) represents interactions among functions and interfaces (internal and external). It defines how the system or its elements act under required conditions to sustain the operational scenarios, including their execution sequencing, synchronization, and concurrency. It models the conditions for behavioural change and performance. Behavioural models are applicable to interactive information systems and software control systems.

A behavioural model can be described with a set of interrelated scenarios. The behavioural model includes identifying the behavioural elements, e.g. modes/states, transitions, trigger events, and operational scenarios.

D.5.2.4 Temporal model

A temporal model of the system is a representation that expresses how duration is allocated in the behaviour of the system or its elements. The temporal model presents levels of execution (e.g. strategic level, tactical level, operational monitoring level, and regulation level), corresponding to levels of decision that enable humans and program logic to monitor and control the system operations. This model includes temporal aspects (e.g. duration, frequency, response time, triggers, timeout, and stop conditions) from the operational concept and system requirements.

D.5.2.5 Structural model

A structural model of the system is a representation that shows the arrangement of elements with respect to each other and the interfaces between elements and with external entities. Such a model enables consolidating or identifying physical interfaces between system elements, as well as with external entities. Structural models can be functional decompositions or object-oriented. In object-oriented programming, a set of system elements or concepts is represented through a set of objects (classes) and their relationships (inheritance, association, and dependency), depicted as nodes and links.

D.5.2.6 Network model

A network model defines an arrangement of nodes and links to help understand how resources, e.g. information and people, traverse from one node to another. A network model can be used to determine constraints such as throughput, latency, and congestion points. A network model is sometimes modelled along with a protocol stack to understand how layers in a network interact vertically up and down the stack.

D.5.2.7 Verification and validation models

Verification and validation are supported by descriptive and prescriptive models to simulate how the system architecture, functions, and performance meet the systems requirements and stakeholder needs. Also, verification and validation of models can be performed to anticipate these activities with the system itself. These activities allow assessment of the needs and requirements, evaluation of the system definition against these requirements, and evaluation of the quality characteristics of the systems. The added value of verification and validation depends on the accuracy and fidelity of the models.

D.6 Digital engineering and software engineering

Digital engineering is the application of software engineering principles to the digital world. It extends the use of software systems and MBSSE for system conceptualization and development, which has existed since the beginnings of software. Through digital engineering, software replaces or supplements analogue controls and human monitoring of a physical system, as an integral part of systems self-actuation throughout the operational stage. A digital twin represents a physical entity with data connections that enable synchronized interaction between the digital and physical entities. Digital engineering produces and applies capabilities for modelling, simulation and visualization, analysis, and optimization for systems operations and throughout the life cycle. Digital engineering is resulting in the digital transformation of numerous industries and services. Viewing digital engineering as a discipline provides a macro perspective to understand its relationship with other disciplines, such as software engineering and systems engineering.

The disciplines of digital engineering (as an extension of software engineering), systems engineering, and traditional engineering disciplines like civil and mechanical engineering can each be viewed along a continuum from foundational knowledge to applied practice. Each dimension follows the progressive pattern of disciplines from a basis in mathematics and philosophy to science, technology science, engineering discipline, and engineering practice. [Figure D.1](#) differentiates digital engineering and systems engineering by their defining orientations: real and virtual, digitalized information or systematic, qualitative relationships.

MBSSE is a digital engineering approach to systems and software engineering, as well as an engineering practice. Within the framework, MBSSE reflects the digital transformation of systems engineering practices as well as the application of systems engineering practices to digitally generated information.

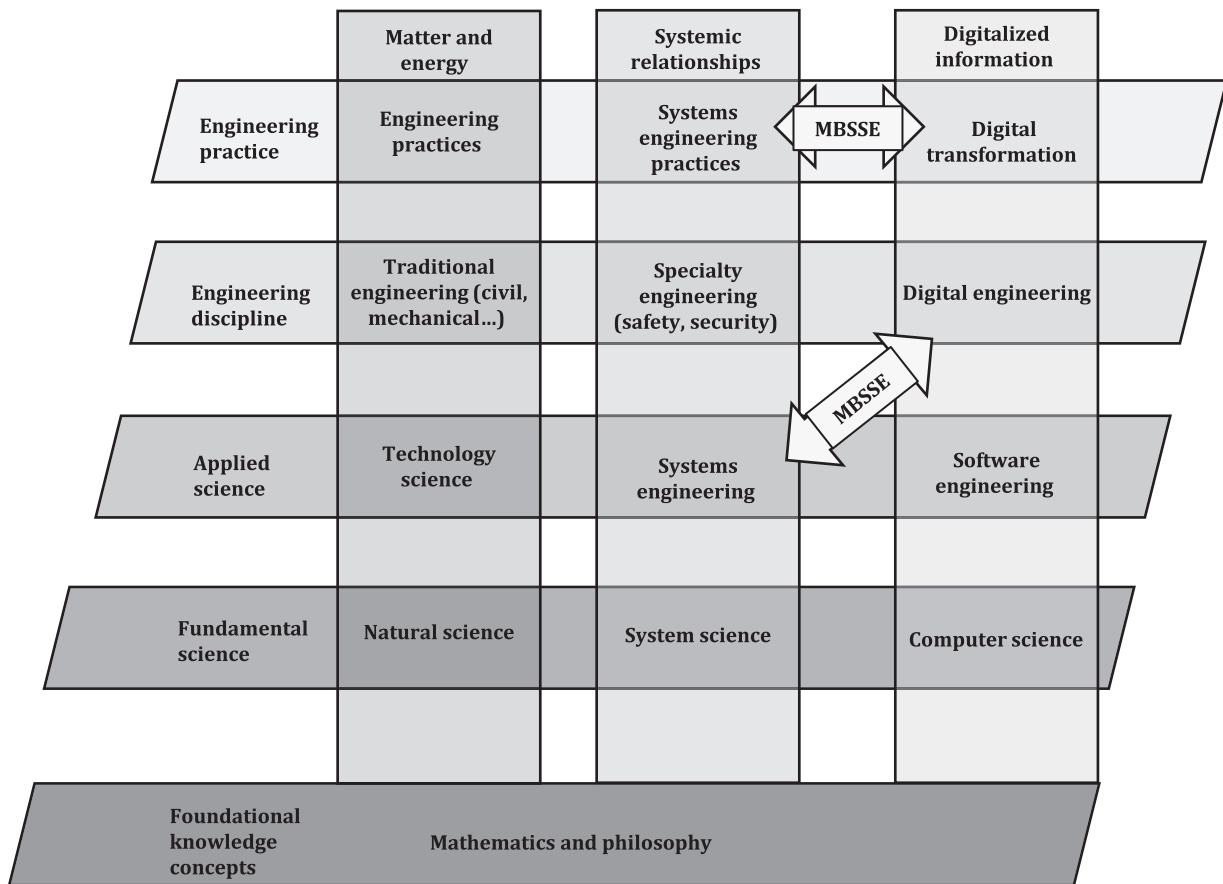


Figure D.1 — Digital engineering in a framework of engineering disciplines

As the core technology science of the digitalized information dimension, software engineering provides the foundational principles and methodologies that enable the development of digital tools and applications essential for digital engineering. These tools include software for modelling, simulation, data analysis, and system integration, which are pivotal in creating and managing digital representations of complex systems throughout their lifecycle. Conversely, digital engineering influences software engineering by introducing new requirements and challenges that drive innovation in software development practices. For instance, the need for integrated, model-based environments in digital engineering has led to advancements in MBSSE, which relies on sophisticated software tools and frameworks. Additionally, the emphasis on creating a cohesive digital thread in digital engineering necessitates robust software solutions for data management, interoperability, and real-time collaboration among diverse stakeholders. This interplay helps software engineering continuously evolve to meet the demands of increasingly complex digital engineering ecosystems, thereby enhancing the efficiency, reliability, and scalability of both fields. From a software life cycle perspective, digital engineering technologies, tools, and artefacts enhance all the software engineering processes.

Annex E **(informative)**

Assurance and assurance cases

Assurance is defined as grounds for justified confidence that a claim has been or can be achieved. This confidence is achieved by applying applicable system life cycle activities, which include a planned, systematic approach with acceptable measures of system assurance and risk management of exploitable vulnerabilities. Stakeholders need assurance prior to depending on a system, especially a system involving complexity, novelty, or technology with a history of problems. The greater the degree of dependence, the greater the need for strong assurance. System assurance claims frequently concern the functions or capabilities of the system.

Assurance, as for a variety of attributes such as safety, security, and dependability, is often required. Stakeholder concerns include achieving justified confidence that the system, while achieving its intent, does not also provide unintended behaviour or produce unintended outcomes. A claims-oriented approach to assurance serves to address the concerns that are not typically captured within the requirements that focus on intended behaviour. An assurance case can identify gaps in requirements coverage and inform the development of derived requirements to address those gaps.

Assurance is often provided through activities to construct an assurance case. An assurance case is an auditable artefact that provides a convincing and valid argument for a claim on the basis of tangible evidence under a given context. Subtle and complex arguments are necessary to organize a wide variety and huge amount of evidence and link them to the claim. Pieces of evidence can be pass/fail results of test, quantitative measurements, or qualitative evaluations. These pieces require careful review on their validity, certainty, fairness, etc., when they are integrated into a cohesive argument. Often there is not a simple direct connection between the evidence provided and the overall claim, so a structure is used to describe and connect the subclaims and the reasoning that link this overall claim and the evidence. Assurance cases focusing on specific characteristics include safety case, composed assurance package (for security), and dependability case.

Assurance activities should be integrated into life cycle processes throughout the system life cycle. Construction of assurance cases require detailed knowledge about both the SoI and the characteristics under consideration. Integration of the specific analyses in the SoI development and involving experts in the SoI development are key success factors to achieve a compliant and balanced system solution.

NOTE The ISO/IEC/IEEE 15026 series provides more information on systems and software assurance and assurance cases. Considerations for requirements on specific characteristics are provided by other documents including the IEC 61508 series (safety); ISO/IEC 27000 and ISO/IEC 15408-3 (information security); IEC 60300-1 and IEC 62741 (dependability); and ISO/IEC 25000 (systems and software quality requirements and evaluation).

Bibliography

- [1] IEC 60300-1, *Dependability management - Part 1: Guidance for management and application*
- [2] IEC 61508 (all parts), *Functional safety of electrical/electronic/programmable electronic safety-related systems*
- [3] IEC 62741, *Demonstration of dependability requirements – The dependability case*
- [4] IEEE Std 1012:2024, *IEEE Standard for System and Software Verification and Validation*
- [5] IEEE Std 7005, *IEEE Standard for Transparent Employer Data Governance*
- [6] IEEE Std 7007, *IEEE Ontological Standard for Ethically Driven Robotics and Automation Systems*
- [7] IEEE Std 7009, *IEEE Standard for Fail-Safe Design of Autonomous and Semi-Autonomous Systems*
- [8] IEEE Std 7014, *IEEE Standard for Ethical Considerations in Emulated Empathy in Autonomous and Intelligent Systems*
- [9] IEEE Std 730:2014, *IEEE Standard for Software Quality Assurance Processes*
- [10] IEEE Std 828:2012, *IEEE Standard for Configuration Management in Systems and Software Engineering*
- [11] IEEE Std 982, *IEEE Standard for Measures of the Software Aspects of Dependability*
- [12] ISO 10004:2018, *Quality management — Customer satisfaction — Guidelines for monitoring and measuring*
- [13] ISO 10007, *Quality management — Guidelines for configuration management*
- [14] ISO 13485, *Medical devices — Quality management systems — Requirements for regulatory purposes*
- [15] ISO 14001, *Environmental management systems — Requirements with guidance for use*
- [16] ISO 15704:2019, *Enterprise modelling and architecture — Requirements for enterprise-referencing architectures and methodologies*
- [17] ISO 18435 (all parts), *Industrial automation systems and integration — Diagnostics, capability assessment and maintenance applications integration*
- [18] ISO 19014-4:2020, *Earth-moving machinery — Functional safety — Part 4: Design and evaluation of software and data transmission for safety-related parts of the control system*
- [19] ISO 21511, *Work breakdown structures for project and programme management*
- [20] ISO 22400 (all parts), *Automation systems and integration — Key performance indicators (KPIs) for manufacturing operations management*
- [21] ISO 26262-1:2018, *Road vehicles — Functional safety — Part 1: Vocabulary*
- [22] ISO 26262-6, *Road vehicles — Functional safety — Part 6: Product development at the software level*
- [23] ISO 31000, *Risk management — Guidelines*
- [24] IEC 31010, *Risk management — Risk assessment techniques*
- [25] ISO 31073:2022, *Risk management — Vocabulary*
- [26] ISO 9000:—¹⁾, *Quality management systems — Fundamentals and vocabulary*

1) Under preparation.

- [27] ISO 9001:2015, *Quality management systems — Requirements*
- [28] ISO 9004, *Quality management — Quality of an organization — Guidance to achieve sustained success*
- [29] ISO 9241-210, *Ergonomics of human-system interaction — Part 210: Human-centred design for interactive systems*
- [30] ISO 9241-220, *Ergonomics of human-system interaction — Part 220: Processes for enabling, executing and assessing human-centred design within organizations*
- [31] ISO/TS 18152, *Ergonomics of human-system interaction — Specification for the process assessment of human-system issues*
- [32] ISO/IEC 5230, *Information technology — OpenChain Specification*
- [33] ISO/IEC 10746-3, *Information technology — Open distributed processing — Reference model: Architecture — Part 3:*
- [34] ISO/IEC 15408-3:2022, *Information security, cybersecurity and privacy protection — Evaluation criteria for IT security — Part 3: Security assurance components*
- [35] ISO/IEC 16350, *Information technology — Systems and software engineering — Application management*
- [36] ISO/IEC 19770 (all parts), *Information technology — IT asset management*
- [37] ISO/IEC 20000 (all parts), *Information technology — Service management*
- [38] ISO/IEC 23894, *Information technology — Artificial intelligence — Guidance on risk management*
- [39] ISO/IEC 25000, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Guide to SQuaRE*
- [40] ISO/IEC 25010, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*
- [41] ISO/IEC 25012, *Software engineering — Software product Quality Requirements and Evaluation (SQuaRE) — Data quality model*
- [42] ISO/IEC 25023, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Measurement of system and software product quality*
- [43] ISO/IEC 25030, *Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — Quality requirements framework*
- [44] ISO/IEC 25063, *Systems and software engineering — Systems and software product Quality Requirements and Evaluation (SQuaRE) — Common Industry Format (CIF) for usability: Context of use description*
- [45] ISO/IEC 26550, *Software and systems engineering — Reference model for product line engineering and management*
- [46] ISO/IEC 26580, *Software and systems engineering — Methods and tools for the feature-based approach to software and systems product line engineering*
- [47] ISO/IEC 27000, *Information technology — Security techniques — Information security management systems — Overview and vocabulary*
- [48] ISO/IEC 27002, *Information security, cybersecurity and privacy protection — Information security controls*
- [49] ISO/IEC 27034-3, *Information technology — Application security — Part 3: Application security management process*

- [50] ISO/IEC 27036 (all parts), *Cybersecurity — Supplier relationships*
- [51] ISO/IEC 33001:2015, *Information technology — Process assessment — Concepts and terminology*
- [52] ISO/IEC 33002, *Information technology — Process assessment — Requirements for performing process assessment*
- [53] ISO/IEC 33004:2015, *Information technology — Process assessment — Requirements for process reference, process assessment and maturity models*
- [54] ISO/IEC 33020:2019, *Information technology — Process assessment — Process measurement framework for assessment of process capability*
- [55] ISO/IEC 33202, *Software and systems engineering — Core agile practices*
- [56] ISO/IEC 5338, *Information technology — Artificial intelligence — AI system life cycle processes*
- [57] ISO/TR 25060, *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — General framework for Common Industry Format (CIF) for usability-related information*
- [58] ISO/IEC 29110-1-1, *Systems and software engineering — Lifecycle profiles for very small entities (VSEs) — Part 1-1: Overview*
- [59] ISO/IEC 29110-1-2, *Systems and software engineering — Lifecycle profiles for Very Small Entities (VSEs) — Part 1-2: Vocabulary*
- [60] ISO/IEC/IEEE 14764, *Software engineering — Software life cycle processes — Maintenance*
- [61] ISO/IEC/IEEE 15026 (all parts), *Systems and software engineering — Systems and software assurance*
- [62] ISO/IEC/IEEE 15288, *Systems and software engineering — System life cycle processes*
- [63] ISO/IEC/IEEE 15289:2019, *Systems and software engineering — Content of life-cycle information items (documentation)*
- [64] ISO/IEC/IEEE 15939, *Systems and software engineering — Measurement process*
- [65] ISO/IEC/IEEE 16085, *Systems and software engineering — Life cycle processes — Risk management*
- [66] ISO/IEC/IEEE 16326, *Systems and software engineering — Life cycle processes — Project management*
- [67] ISO/IEC/IEEE 21839:2019, *Systems and software engineering — System of systems (SoS) considerations in life cycle stages of a system*
- [68] ISO/IEC/IEEE 21840, *Systems and software engineering — Guidelines for the utilization of ISO/IEC/IEEE 15288 in the context of system of systems (SoS)*
- [69] ISO/IEC/IEEE 21841, *Systems and software engineering — Taxonomy of systems of systems*
- [70] ISO/IEC/IEEE 24641:2023, *Systems and Software engineering — Methods and tools for model-based systems and software engineering*
- [71] ISO/IEC/IEEE 24748-1:2024, *Systems and software engineering — Life cycle management — Part 1: Guidelines for life cycle management*
- [72] ISO/IEC/IEEE 24748-2, *Systems and software engineering — Life cycle management — Part 2: Guidelines for the application of ISO/IEC/IEEE 15288 (system life cycle processes)*
- [73] ISO/IEC/IEEE 24748-3, *Systems and software engineering — Life cycle management — Part 3: Guidelines for the application of ISO/IEC/IEEE 12207 (software life cycle processes)*
- [74] ISO/IEC/IEEE 24748-4, *Systems and software engineering — Life cycle management — Part 4: Systems engineering planning*

- [75] ISO/IEC/IEEE 24748-5, *Systems and software engineering — Life cycle management — Part 5: Software development planning*
- [76] ISO/IEC/IEEE 24748-6, *Systems and software engineering — Life cycle management — Part 6: System and software integration*
- [77] ISO/IEC/IEEE 24748-8, *Systems and software engineering — Life cycle management — Part 8: Technical reviews and audits on defense programs*
- [78] ISO/IEC/IEEE 24748-7000, *Systems and software engineering — Life cycle management — Part 7000: Standard model process for addressing ethical concerns during system design*
- [79] ISO/IEC/IEEE 24765, *Systems and software engineering — Vocabulary*
- [80] ISO/IEC/IEEE 24774:2021, *Systems and software engineering — Life cycle management — Specification for process description*
- [81] ISO/IEC/IEEE 26515, *Systems and software engineering — Developing information for users in an agile environment*
- [82] ISO/IEC/IEEE 26531, *Systems and software engineering — Content management for product life cycle, user and service management information for users*
- [83] ISO/IEC/IEEE 23612, *Software and systems engineering -- Incident management*
- [84] ISO/IEC/IEEE 29119 (all parts), *Software and systems engineering — Software testing*
- [85] ISO/IEC/IEEE 29148:2018, *Systems and software engineering — Life cycle processes — Requirements engineering*
- [86] ISO/IEC/IEEE 32675:2022, *Information technology — DevOps — Building reliable and secure systems including application build, package and deployment*
- [87] ISO/IEC/IEEE 41062, *Software engineering — Life cycle processes — Software acquisition*
- [88] ISO/IEC/IEEE 42010:2022, *Software, systems and enterprise — Architecture description*
- [89] ISO/IEC/IEEE 42020:2019, *Software, systems and enterprise — Architecture processes*
- [90] ISO/IEC/IEEE 42030, *Software, systems and enterprise — Architecture evaluation framework*
- [91] ISO/IEC/IEEE 90003, *Software engineering — Guidelines for the application of ISO 9001:2015 to computer software*
- [92] ANSI/AAMI/IEC 62304:2006/A1: 2016, *Medical device software — Software life cycle processes*
- [93] ANSI/AIAA G-043B-2018, *ANSI/AIAA Guide to the Preparation of Operational Concept Documents*
- [94] CYBERSECURITY IN MEDICAL DEVICES. Quality System Considerations and Content of Premarket Submissions, Sept. 27, 2023, <https://www.fda.gov/media/119933/download?attachment>
- [95] Framing Software Component Transparency: Establishing a Common Software Bill of Material (SBOM), NTIA Multistakeholder Process on Software Component Transparency, 2019-11-12, <https://www.fda.gov/regulatory-information/search-fda-guidance-documents/cybersecurity-medical-devices-quality-system-considerations-and-content-premarket-submissions>
- [96] IEEE. Computer Society, *Guide to the Software Engineering Body of Knowledge (SWEBOK) V4*, 2024, <ieeecs-media.computer.org/media/education/swbok/swbok-v4.pdf>
- [97] INCOSE *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, version 5.0. Hoboken, NJ, USA: John Wiley and Sons, Inc.
- [98] INCOSE TP-2003-020-01, *Technical Measurement*

- [99] INCOSE-TP-2020-002-06, *Systems Engineering and System Definitions*, Version 1.0, 08 January 2019
- [100] NATO AEP-67, *Engineering for System Assurance in NATO Programs*
- [101] NIST AI 100-1, *Artificial Intelligence Risk Management Framework*
- [102] NIST SP 800-218, *Secure Software Development Framework*
- [103] PMI (Project Management Institute), *Practice Standard for Work Breakdown Structures*
- [104] Royce, Winston. "Managing the Development of Large Software Systems", <https://www.praxisframework.org/files/royce1970.pdf>
- [105] SAE ARP4754A, *Guidelines for Development of Civil Aircraft and Systems*
- [106] 10SAE EIA-649-C-2019, *Configuration Management Standard*
- [107] 101SAE JA1011, *Evaluation Criteria for Reliability-Centered Maintenance (RCM) Processes*
- [108] STANAG 4427, *Configuration Management in System Life Cycle Management*
- [109] Tukker A.2004, Eight Types of Product Service Systems: Eight Ways to Sustainability? Business Strategy and the Environment 13, p. 246

IEEE notices and abstract

Important Notices and Disclaimers Concerning IEEE Standards Documents

IEEE Standards documents are made available for use subject to important notices and legal disclaimers. These notices and disclaimers, or a reference to this page (<https://standards.ieee.org/ipr/disclaimers>), appear in all standards and may be found under the heading “Important Notices and Disclaimers Concerning IEEE Standards Documents.”

Notice and Disclaimer of Liability Concerning the Use of IEEE Standards Documents

IEEE Standards documents are developed within IEEE Societies and subcommittees of IEEE Standards Association (IEEE SA) Board of Governors. IEEE develops its standards through an accredited consensus development process, which brings together volunteers representing varied viewpoints and interests to achieve the final product. IEEE standards are documents developed by volunteers with scientific, academic, and industry-based expertise in technical working groups. Volunteers are not necessarily members of IEEE or IEEE SA and participate without compensation from IEEE. While IEEE administers the process and establishes rules to promote fairness in the consensus development process, IEEE does not independently evaluate, test, or verify the accuracy of any of the information or the soundness of any judgments contained in its standards.

IEEE makes no warranties or representations concerning its standards, and expressly disclaims all warranties, express or implied, concerning this standard, including but not limited to the warranties of merchantability, fitness for a particular purpose and non-infringement. IEEE Standards documents do not guarantee safety, security, health, or environmental protection, or guarantee against interference with or from other devices or networks. In addition, IEEE does not warrant or represent that the use of the material contained in its standards is free from patent infringement. IEEE Standards documents are supplied “AS IS” and “WITH ALL FAULTS.”

Use of an IEEE standard is wholly voluntary. The existence of an IEEE standard does not imply that there are no other ways to produce, test, measure, purchase, market, or provide other goods and services related to the scope of the IEEE standard. Furthermore, the viewpoint expressed at the time a standard is approved and issued is subject to change brought about through developments in the state of the art and comments received from users of the standard.

In publishing and making its standards available, IEEE is not suggesting or rendering professional or other services for, or on behalf of, any person or entity, nor is IEEE undertaking to perform any duty owed by any other person or entity to another. Any person utilizing any IEEE Standards document, should rely upon their own independent judgment in the exercise of reasonable care in any given circumstances or, as appropriate, seek the advice of a competent professional in determining the appropriateness of a given IEEE standard.

IN NO EVENT SHALL IEEE BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO: THE NEED TO PROCURE SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE PUBLICATION, USE OF, OR RELIANCE UPON ANY STANDARD, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE AND REGARDLESS OF WHETHER SUCH DAMAGE WAS FORESEEABLE.

Translations

The IEEE consensus balloting process involves the review of documents in English only. In the event that an IEEE standard is translated, only the English version published by IEEE is the approved IEEE standard.

Official statements

A statement, written or oral, that is not processed in accordance with the IEEE SA Standards Board Operations Manual shall not be considered or inferred to be the official position of IEEE or any of its committees and shall not be considered to be, nor be relied upon as, a formal position of IEEE. At lectures, symposia, seminars, or educational courses, an individual presenting information on IEEE standards shall make it clear that the presenter's views should be considered the personal views of that individual rather than the formal position of IEEE, IEEE SA, the Standards Committee, or the Working Group. Statements made by volunteers may not represent the formal position of their employer(s) or affiliation(s).

Comments on standards

IEEE does not provide interpretations, consulting information, or advice pertaining to IEEE Standards documents

Suggestions for changes in documents should be in the form of a proposed change of text, together with appropriate supporting comments. Since IEEE standards represent a consensus of concerned interests, it is important that any responses to comments and questions also receive the concurrence of a balance of interests. For this reason, IEEE and the members of its Societies and subcommittees of the IEEE SA Board of Governors are not able to provide an instant response to comments or questions except in those cases where the matter has previously been addressed. For the same reason, IEEE does not respond to interpretation requests. Any person who would like to participate in evaluating comments or in revisions to an IEEE standard is welcome to join the relevant IEEE working group. You can indicate interest in a working group using the Interests tab in the Manage Profile and Interests area of the [IEEE SA myProject system](#).²⁾ An IEEE Account is needed to access the application.

Comments on standards should be submitted using the [Contact Us](#) form.³⁾

Laws and regulations

Users of IEEE Standards documents should consult all applicable laws and regulations. Compliance with the provisions of any IEEE Standards document does not constitute compliance to any applicable regulatory requirements. Implementers of the standard are responsible for observing or referring to the applicable regulatory requirements. IEEE does not, by the publication of its standards, intend to urge action that is not in compliance with applicable laws, and these documents may not be construed as doing so.

Data privacy

Users of IEEE Standards documents should evaluate the standards for considerations of data privacy and data ownership in the context of assessing and using the standards in compliance with applicable laws and regulations.

Copyrights

IEEE draft and approved standards are copyrighted by IEEE under US and international copyright laws. They are made available by IEEE and are adopted for a wide variety of both public and private uses. These include both use, by reference, in laws and regulations, and use in private self-regulation, standardization, and the promotion of engineering practices and methods. By making these documents available for use and adoption by public authorities and private users, neither IEEE nor its licensors waive any rights in copyright to the documents.

2) Available at: <https://development.standards.ieee.org/myproject-web/public/view.html>.

3) Available at: <https://standards.ieee.org/thank-you/contact/>.

Photocopies

Subject to payment of the appropriate licensing fees, IEEE will grant users a limited, non-exclusive license to photocopy portions of any individual standard for company or organizational internal use or individual, non-commercial use only. To arrange for payment of licensing fees, please contact Copyright Clearance Center, Customer Service, 222 Rosewood Drive, Danvers, MA 01923 USA; +1 978 750 8400; <https://www.copyright.com/>. Permission to photocopy portions of any individual standard for educational classroom use can also be obtained through the Copyright Clearance Center.

Updating of IEEE Standards documents

Users of IEEE Standards documents should be aware that these documents may be superseded at any time by the issuance of new editions or may be amended from time to time through the issuance of amendments, corrigenda, or errata. An official IEEE document at any point in time consists of the current edition of the document together with any amendments, corrigenda, or errata then in effect.

Every IEEE standard is subjected to review at least every 10 years. When a document is more than 10 years old and has not undergone a revision process, it is reasonable to conclude that its contents, although still of some value, do not wholly reflect the present state of the art. Users are cautioned to check to determine that they have the latest edition of any IEEE standard.

In order to determine whether a given document is the current edition and whether it has been amended through the issuance of amendments, corrigenda, or errata, visit [IEEE Xplore](#) or [contact IEEE](#).⁴⁾ For more information about the IEEE SA or IEEE's standards development process, visit the IEEE SA Website.

Errata

Errata, if any, for all IEEE standards can be accessed on the [IEEE SA Website](#).⁵⁾ Search for standard number and year of approval to access the web page of the published standard. Errata links are located under the Additional Resources Details section. Errata are also available in [IEEE Xplore](#). Users are encouraged to periodically check for errata.

Patents

IEEE standards are developed in compliance with the [IEEE SA Patent Policy](#).⁶⁾

Attention is called to the possibility that implementation of this standard may require use of subject matter covered by patent rights. By publication of this standard, no position is taken by the IEEE with respect to the existence or validity of any patent rights in connection therewith. If a patent holder or patent applicant has filed a statement of assurance via an Accepted Letter of Assurance, then the statement is listed on the IEEE SA Website at <https://standards.ieee.org/about/sasb/patcom/patents>. Letters of Assurance may indicate whether the Submitter is willing or unwilling to grant licenses under patent rights without compensation or under reasonable rates, with reasonable terms and conditions that are demonstrably free of any unfair discrimination to applicants desiring to obtain such licenses.

Essential Patent Claims may exist for which a Letter of Assurance has not been received. The IEEE is not responsible for identifying Essential Patent Claims for which a license may be required, for conducting inquiries into the legal validity or scope of Patents Claims, or determining whether any licensing terms or conditions provided in connection with the submission of a Letter of Assurance, if any, or in any licensing agreements are reasonable or non-discriminatory. Users of this standard are expressly advised that determination of the validity of any patent rights, and the risk of infringement of such rights, is entirely their own responsibility. Further information may be obtained from the IEEE Standards Association.

4) Available at: <https://standards.ieee.org/about/contact>.

5) Available at: <https://standards.ieee.org/standard/>

6) Available at: <https://standards.ieee.org/about/sasb/patcom/materials>.

IMPORTANT NOTICE

Technologies, application of technologies, and recommended procedures in various industries evolve over time. The IEEE standards development process allows participants to review developments in industries, technologies, and practices, and to determine what, if any, updates should be made to the IEEE standard. During this evolution, the technologies and recommendations in IEEE standards may be implemented in ways not foreseen during the standard's development. IEEE standards development activities consider research and information presented to the standards development group in developing any safety recommendations. Other information about safety practices, changes in technology or technology implementation, or impact by peripheral systems also may be pertinent to safety considerations during implementation of the standard. Implementers and users of IEEE Standards documents are responsible for determining and complying with all appropriate safety, security, environmental, health, and interference protection practices and all applicable laws and regulations.

Abstract

This document establishes a common process framework for describing the full life cycle of software systems from conception through retirement. It applies to the acquisition or development of software products and services whether performed internally or externally to an organization. Those aspects of software system definition needed to provide the context for software products and services are included. The document also supports the definition, control, assessment, and improvement of these processes. These processes can be applied concurrently, iteratively, and recursively to a software system and its elements throughout the life cycle of a software system. Users of the document can apply the processes and terminology to construct a suitable life cycle model, composed of stages that use selected processes. The process purposes and outcomes in ISO/IEC/IEEE 15288:2023 (systems life cycle processes) and this document are fully aligned so that one may select appropriate activities from either document for use in systems with substantial software content.

Keywords: acquisition, agreement, architecture, design, assessment, audit, configuration management, decision management, development, disposal, enabling system, implementation, information management, infrastructure, integration, life cycle, life cycle model, life cycle stages, maintenance, measurement, operation, planning, process, process improvement, process reference model, process tailoring, process view, product, portfolio, quality management, requirements, retirement, risk management, service, software, software system, system-of-interest, stages, stakeholder requirements, supply, system, system structure, tailoring, transition, validation, verification



ICS 35.080

ISBN 979-8-8557-3262-7 STD28729 (PDF); 979-8-8557-3263-4 STDPD28729 (Print)

Price based on 140 pages

© ISO/IEC 2026 – All rights reserved

© IEEE 2026 – All rights reserved

Authorized licensed use limited to: National Institute of Technology- Delhi. Downloaded on May 05,2026 at 07:25:56 UTC from IEEE Xplore. Restrictions apply.

iso.org